Namespace ElectricDrill.AstraHealth.Barrier

Interfaces

IBarrierContainer

Contract for entities that hold a barrier (temporary hit points) that absorbs incoming damage before health.

Implement this interface on any component that manages a barrier pool so that pipeline steps and external packages can interact with the barrier without depending on `EntityHealth` directly.

`EntityHealth` implements this interface. Future packages (e.g. modifier steps that grant barrier on-hit) should accept IBarrierContainer rather than the concrete type.

Interface IBarrierContainer

Namespace: ElectricDrill.AstraHealth.Barrier

Assembly: com.electricdrill.astra-health.Runtime.dll

Contract for entities that hold a barrier (temporary hit points) that absorbs incoming damage before health.

Implement this interface on any component that manages a barrier pool so that pipeline steps and external packages can interact with the barrier without depending on `EntityHealth` directly.

`EntityHealth` implements this interface. Future packages (e.g. modifier steps that grant barrier on-hit) should accept IBarrierContainer rather than the concrete type.

```
public interface IBarrierContainer
```

Properties

Barrier

Current barrier points. Zero means no barrier is active.

```
long Barrier { get; }
```

Property Value

long

Methods

AddBarrier(long)

Adds the specified amount to the entity's barrier pool.

```
void AddBarrier(long amount)
```

Parameters

`amount` long

Positive amount to add.

RemoveBarrier(long)

Removes the specified amount from the entity's barrier pool, flooring at zero.

```
void RemoveBarrier(long amount)
```

Parameters

`amount` long

Positive amount to consume.

Namespace ElectricDrill.AstraHealth.Conditions

Classes

BarrierThresholdCondition

Condition that compares the resolved target entity's barrier (temporary HP) against a threshold.

When ElectricDrill.AstraRpgFramework.Conditions.ThresholdMode.Absolute, compares barrier directly against the configured value.
When ElectricDrill.AstraRpgFramework.Conditions.ThresholdMode.Percentage, compares barrier as a percentage of MaxHP (Barrier / MaxHP × 100, clamped [0–100]) against the configured percentage threshold. Returns `false` if MaxHP is zero.

Use ElectricDrill.AstraRpgFramework.Conditions.ComparisonMode.Greater with threshold 0 to check "has any barrier".

DamageAmountThresholdCondition

Condition that compares a damage amount against a threshold.

RawAmount: the raw pre-modifier amount from Amount.
FinalAmount: the final applied amount; returns `false` when damage was prevented.

When ElectricDrill.AstraRpgFramework.Conditions.ThresholdMode.Percentage, the amount is expressed as a percentage of the target's MaxHP (amount / MaxHP × 100). Returns `false` if MaxHP is zero or target cannot be resolved.

DamageIsCriticalCondition

Condition that returns `true` when the damage payload is flagged as a critical hit. Supports PreDamage Context, DamageResolutionContext, and EntityDiedContext payloads.

DamageOutcomeCondition

Condition that returns `true` when the damage resolution outcome matches the configured value. Supports DamageResolutionContext and EntityDiedContext payloads. Returns `false` for PreDamage Context (resolution not yet available).

DamagePreventionReasonCondition

Condition that checks whether the damage resolution context's prevention reasons include any or all of the configured flags. Supports DamageResolutionContext and EntityDiedContext payloads. Returns `false` for PreDamageContext or when no prevention reasons match.

DamageSourceMatchCondition

Condition that returns `true` when the damage payload's source matches the configured reference. Supports PreDamageContext, DamageResolutionContext, and EntityDiedContext payloads.

DamageTypeMatchCondition

Condition that returns `true` when the damage payload's type matches the configured reference. Supports PreDamageContext, DamageResolutionContext, and EntityDiedContext payloads.

EntityIsAliveCondition

Condition that returns `true` when the resolved target entity is alive. Returns `false` if the entity or its EntityHealth component cannot be resolved.

EntityIsDeadCondition

Condition that returns `true` when the resolved target entity is dead. Returns `false` if the entity or its EntityHealth component cannot be resolved.

EntityIsImmuneCondition

Condition that returns `true` when the resolved target entity is immune to all damage. Returns `false` if the entity or its EntityHealth component cannot be resolved.

HealAmountThresholdCondition

Condition that compares a heal amount against a threshold.

RawAmount: raw requested heal amount.
AfterModifierAmount: after stat/modifier adjustments.
NetAmount: actual health gained (capped at remaining capacity).
OverhealAmount: AfterModifierAmount − NetAmount (≥ 0); returns `false` from PreHealContext.

When ElectricDrill.AstraRpgFramework.Conditions.ThresholdMode.Percentage, the amount is expressed as a percentage of the target's MaxHP. Returns `false` if MaxHP is zero or target cannot be resolved.

HealIsCriticalCondition

Condition that returns `true` when the heal payload is flagged as a critical heal. Supports PreHealContext, ReceivedHealContext, and ResurrectionContext payloads.

HealSourceMatchCondition

Condition that returns `true` when the heal payload's source matches the configured reference. Supports PreHealContext, ReceivedHealContext, and ResurrectionContext payloads.

HealthRatioThresholdTransitionCondition

Condition that detects when an HP ratio metric crosses a threshold in a HealthRatioChangedContext payload.

Fires on the `HealthRatioChanged` event, which is raised whenever HP or MaxHP changes. This makes it reliable for threshold crossing detection regardless of whether HP, MaxHP, or both changed.

Supported components:

- CurrentHpPercentage — HP / MaxHP × 100; uses percentage threshold.
- MissingHpAbsolute — MaxHP − HP; uses absolute threshold.
- MissingHpPercentage — (MaxHP − HP) / MaxHP × 100; uses percentage threshold.

Returns `false` when MaxHP is zero for percentage-based components.

HealthThresholdCondition
: Condition that compares the resolved target entity's current HP against a threshold.

When ElectricDrill.AstraRpgFramework.Conditions.ThresholdMode.Absolute, compares HP directly against the configured value.
When ElectricDrill.AstraRpgFramework.Conditions.ThresholdMode.Percentage, compares HP% (HP / MaxHP × 100, clamped [0–100]) against the configured percentage threshold. Returns `false` if MaxHP is zero.

MaxHealthThresholdCondition
: Condition that compares the resolved target entity's maximum HP against an absolute threshold.

MissingHealthThresholdCondition
: Condition that compares the resolved target entity's missing HP (MaxHP − HP) against a threshold.

When ElectricDrill.AstraRpgFramework.Conditions.ThresholdMode.Absolute, compares missing HP directly against the configured value.
When ElectricDrill.AstraRpgFramework.Conditions.ThresholdMode.Percentage, compares missing HP% ((MaxHP − HP) / MaxHP × 100, clamped [0–100]) against the configured percentage threshold. Returns `false` if MaxHP is zero.

PayloadIsReactableCondition
: Condition that returns `true` when the event payload implements IReactable and its IsReactable flag is `true`.

Use this to restrict reactive mechanics to first-order events only (direct damage, skills, abilities), preventing infinite chain reactions from secondary effects.

Returns `false` when the payload does not implement IReactable.

Enums

DamageAmountComponent
: Selects which damage amount to extract from a damage payload.

HealAmountComponent
: Selects which heal amount to extract from a heal payload.

HealthRatioComponent

Selects which HP ratio component to evaluate in a HealthRatioThresholdTransitionCondition.

ReasonMatchMode

Determines how multiple DamagePreventionReason flags are matched.

Class BarrierThresholdCondition

Namespace: ElectricDrill.AstraHealth.Conditions

Assembly: com.electricdrill.astra-health.Runtime.dll

Condition that compares the resolved target entity's barrier (temporary HP) against a threshold.

When ElectricDrill.AstraRpgFramework.Conditions.ThresholdMode.Absolute, compares barrier directly against the configured value.
When ElectricDrill.AstraRpgFramework.Conditions.ThresholdMode.Percentage, compares barrier as a percentage of MaxHP (Barrier / MaxHP × 100, clamped [0–100]) against the configured percentage threshold. Returns `false` if MaxHP is zero.

Use ElectricDrill.AstraRpgFramework.Conditions.ComparisonMode.Greater with threshold 0 to check "has any barrier".

```
[Serializable]
[TypeSelectableMenu("Leaf/Entity")]
public class BarrierThresholdCondition : Condition
```

Inheritance

object ← Condition ← BarrierThresholdCondition

Methods

Evaluate(EvaluationContext)

Evaluates this condition against the provided context.

```
public override bool Evaluate(EvaluationContext ctx)
```

Parameters

`ctx` EvaluationContext

Contextual data available to the condition (holder, performer, payload).

Returns

bool

`true` if the condition is satisfied; otherwise `false`.

Enum DamageAmountComponent

Namespace: ElectricDrill.AstraHealth.Conditions

Assembly: com.electricdrill.astra-health.Runtime.dll

Selects which damage amount to extract from a damage payload.

```
public enum DamageAmountComponent
```

Fields

`FinalAmount = 1`

The final applied damage amount from Amounts.Current. Only available when damage was actually applied (Outcome == Applied). Returns false when damage was prevented.

`RawAmount = 0`

The raw damage amount from Amount. Available from PreDamageContext, DamageResolutionContext (via PreDamageContext), and EntityDiedContext.

Class DamageAmountThresholdCondition

Namespace: ElectricDrill.AstraHealth.Conditions

Assembly: com.electricdrill.astra-health.Runtime.dll

Condition that compares a damage amount against a threshold.

RawAmount: the raw pre-modifier amount from Amount.
FinalAmount: the final applied amount; returns `false` when damage was prevented.

When ElectricDrill.AstraRpgFramework.Conditions.ThresholdMode.Percentage, the amount is expressed as a percentage of the target's MaxHP (amount / MaxHP × 100). Returns `false` if MaxHP is zero or target cannot be resolved.

```
[Serializable]
[TypeSelectableMenu("Payload/Damage")]
[ConditionPayload(typeof(PreDamageContext))]
[ConditionPayload(typeof(DamageResolutionContext))]
[ConditionPayload(typeof(EntityDiedContext))]
public class DamageAmountThresholdCondition : Condition
```

Inheritance

object ← Condition ← DamageAmountThresholdCondition

Methods

Evaluate(EvaluationContext)

Evaluates this condition against the provided context.

```
public override bool Evaluate(EvaluationContext ctx)
```

Parameters

`ctx` EvaluationContext

Contextual data available to the condition (holder, performer, payload).

Returns

bool

`true` if the condition is satisfied; otherwise `false`.

Class DamageIsCriticalCondition

Namespace: ElectricDrill.AstraHealth.Conditions

Assembly: com.electricdrill.astra-health.Runtime.dll

Condition that returns `true` when the damage payload is flagged as a critical hit. Supports PreDamageContext, DamageResolutionContext, and EntityDiedContext payloads.

```
[Serializable]
[TypeSelectableMenu("Payload/Damage")]
[ConditionPayload(typeof(PreDamageContext))]
[ConditionPayload(typeof(DamageResolutionContext))]
[ConditionPayload(typeof(EntityDiedContext))]
public class DamageIsCriticalCondition : Condition
```

Inheritance

object ← Condition ← DamageIsCriticalCondition

Methods

Evaluate(EvaluationContext)

Evaluates this condition against the provided context.

```
public override bool Evaluate(EvaluationContext ctx)
```

Parameters

`ctx` EvaluationContext

Contextual data available to the condition (holder, performer, payload).

Returns

bool

`true` if the condition is satisfied; otherwise `false`.

Class DamageOutcomeCondition

Namespace: ElectricDrill.AstraHealth.Conditions

Assembly: com.electricdrill.astra-health.Runtime.dll

Condition that returns `true` when the damage resolution outcome matches the configured value. Supports DamageResolutionContext and EntityDiedContext payloads. Returns `false` for PreDamage Context (resolution not yet available).

```
[Serializable]
[TypeSelectableMenu("Payload/Damage")]
[ConditionPayload(typeof(DamageResolutionContext))]
[ConditionPayload(typeof(EntityDiedContext))]
public class DamageOutcomeCondition : Condition
```

Inheritance

object ← Condition ← DamageOutcomeCondition

Methods

Evaluate(EvaluationContext)

Evaluates this condition against the provided context.

```
public override bool Evaluate(EvaluationContext ctx)
```

Parameters

`ctx` EvaluationContext

Contextual data available to the condition (holder, performer, payload).

Returns

bool

`true` if the condition is satisfied; otherwise `false`.

Class DamagePreventionReasonCondition

Namespace: ElectricDrill.AstraHealth.Conditions

Assembly: com.electricdrill.astra-health.Runtime.dll

Condition that checks whether the damage resolution context's prevention reasons include any or all of the configured flags. Supports DamageResolutionContext and EntityDiedContext payloads. Returns `false` for PreDamageContext or when no prevention reasons match.

```
[Serializable]
[TypeSelectableMenu("Payload/Damage")]
[ConditionPayload(typeof(DamageResolutionContext))]
[ConditionPayload(typeof(EntityDiedContext))]
public class DamagePreventionReasonCondition : Condition
```

Inheritance

object ← Condition ← DamagePreventionReasonCondition

Methods

Evaluate(EvaluationContext)

Evaluates this condition against the provided context.

```
public override bool Evaluate(EvaluationContext ctx)
```

Parameters

`ctx` EvaluationContext

Contextual data available to the condition (holder, performer, payload).

Returns

bool

`true` if the condition is satisfied; otherwise `false`.

Class DamageSourceMatchCondition

Namespace: ElectricDrill.AstraHealth.Conditions

Assembly: com.electricdrill.astra-health.Runtime.dll

Condition that returns `true` when the damage payload's source matches the configured reference. Supports PreDamageContext, DamageResolutionContext, and EntityDiedContext payloads.

```
[Serializable]
[TypeSelectableMenu("Payload/Damage")]
[ConditionPayload(typeof(PreDamageContext))]
[ConditionPayload(typeof(DamageResolutionContext))]
[ConditionPayload(typeof(EntityDiedContext))]
public class DamageSourceMatchCondition : Condition
```

Inheritance

object ← Condition ← DamageSourceMatchCondition

Methods

Evaluate(EvaluationContext)

Evaluates this condition against the provided context.

```
public override bool Evaluate(EvaluationContext ctx)
```

Parameters

`ctx` EvaluationContext

Contextual data available to the condition (holder, performer, payload).

Returns

bool

`true` if the condition is satisfied; otherwise `false`.

Class DamageTypeMatchCondition

Namespace: ElectricDrill.AstraHealth.Conditions

Assembly: com.electricdrill.astra-health.Runtime.dll

Condition that returns `true` when the damage payload's type matches the configured reference. Supports PreDamageContext, DamageResolutionContext, and EntityDiedContext payloads.

```
[Serializable]
[TypeSelectableMenu("Payload/Damage")]
[ConditionPayload(typeof(PreDamageContext))]
[ConditionPayload(typeof(DamageResolutionContext))]
[ConditionPayload(typeof(EntityDiedContext))]
public class DamageTypeMatchCondition : Condition
```

Inheritance

object ← Condition ← DamageTypeMatchCondition

Methods

Evaluate(EvaluationContext)

Evaluates this condition against the provided context.

```
public override bool Evaluate(EvaluationContext ctx)
```

Parameters

`ctx` EvaluationContext

Contextual data available to the condition (holder, performer, payload).

Returns

bool

`true` if the condition is satisfied; otherwise `false`.

Class EntityIsAliveCondition

Namespace: ElectricDrill.AstraHealth.Conditions

Assembly: com.electricdrill.astra-health.Runtime.dll

Condition that returns `true` when the resolved target entity is alive. Returns `false` if the entity or its Entity Health component cannot be resolved.

```
[Serializable]
[TypeSelectableMenu("Leaf/Entity")]
public class EntityIsAliveCondition : Condition
```

Inheritance

object ← Condition ← EntityIsAliveCondition

Methods

Evaluate(EvaluationContext)

Evaluates this condition against the provided context.

```
public override bool Evaluate(EvaluationContext ctx)
```

Parameters

`ctx` EvaluationContext

Contextual data available to the condition (holder, performer, payload).

Returns

bool

`true` if the condition is satisfied; otherwise `false`.

Class EntityIsDeadCondition

Namespace: ElectricDrill.AstraHealth.Conditions

Assembly: com.electricdrill.astra-health.Runtime.dll

Condition that returns `true` when the resolved target entity is dead. Returns `false` if the entity or its EntityHealth component cannot be resolved.

```
[Serializable]
[TypeSelectableMenu("Leaf/Entity")]
public class EntityIsDeadCondition : Condition
```

Inheritance

object ← Condition ← EntityIsDeadCondition

Methods

Evaluate(EvaluationContext)

Evaluates this condition against the provided context.

```
public override bool Evaluate(EvaluationContext ctx)
```

Parameters

`ctx` EvaluationContext

Contextual data available to the condition (holder, performer, payload).

Returns

bool

`true` if the condition is satisfied; otherwise `false`.

Class EntityIsImmuneCondition

Namespace: ElectricDrill.AstraHealth.Conditions

Assembly: com.electricdrill.astra-health.Runtime.dll

Condition that returns `true` when the resolved target entity is immune to all damage. Returns `false` if the entity or its EntityHealth component cannot be resolved.

```
[Serializable]
[TypeSelectableMenu("Leaf/Entity")]
public class EntityIsImmuneCondition : Condition
```

Inheritance

object ← Condition ← EntityIsImmuneCondition

Methods

Evaluate(EvaluationContext)

Evaluates this condition against the provided context.

```
public override bool Evaluate(EvaluationContext ctx)
```

Parameters

`ctx` EvaluationContext

Contextual data available to the condition (holder, performer, payload).

Returns

bool

`true` if the condition is satisfied; otherwise `false`.

Enum HealAmountComponent

Namespace: ElectricDrill.AstraHealth.Conditions

Assembly: com.electricdrill.astra-health.Runtime.dll

Selects which heal amount to extract from a heal payload.

```
public enum HealAmountComponent
```

Fields

`AfterModifierAmount = 1`

The heal amount after applying modifier/stat adjustments (AfterModifierAmount). Available from ReceivedHealContext and ResurrectionContext.

`NetAmount = 2`

The actual health added to the entity, capped at the remaining HP capacity (NetAmount). Available from ReceivedHealContext and ResurrectionContext.

`OverhealAmount = 3`

The amount of heal that exceeded the entity's max HP (AfterModifierAmount - NetAmount, clamped to ≥ 0). Available from ReceivedHealContext and ResurrectionContext. Returns false when evaluated against PreHealContext (net amount unknown pre-application).

`RawAmount = 0`

The raw requested heal amount before any modifiers (RawAmount). Available from ReceivedHealContext and ResurrectionContext.

Class HealAmountThresholdCondition

Namespace: ElectricDrill.AstraHealth.Conditions

Assembly: com.electricdrill.astra-health.Runtime.dll

Condition that compares a heal amount against a threshold.

RawAmount: raw requested heal amount.
AfterModifierAmount: after stat/modifier adjustments.
NetAmount: actual health gained (capped at remaining capacity).
OverhealAmount: AfterModifierAmount − NetAmount (≥ 0); returns `false` from PreHealContext.

When ElectricDrill.AstraRpgFramework.Conditions.ThresholdMode.Percentage, the amount is expressed as a percentage of the target's MaxHP. Returns `false` if MaxHP is zero or target cannot be resolved.

```
[Serializable]
[TypeSelectableMenu("Payload/Heal")]
[ConditionPayload(typeof(ReceivedHealContext))]
[ConditionPayload(typeof(ResurrectionContext))]
public class HealAmountThresholdCondition : Condition
```

Inheritance

object ← Condition ← HealAmountThresholdCondition

Methods

Evaluate(EvaluationContext)

Evaluates this condition against the provided context.

```
public override bool Evaluate(EvaluationContext ctx)
```

Parameters

`ctx` EvaluationContext

Contextual data available to the condition (holder, performer, payload).

Returns

bool

`true` if the condition is satisfied; otherwise `false`.

Class HealIsCriticalCondition

Namespace: ElectricDrill.AstraHealth.Conditions

Assembly: com.electricdrill.astra-health.Runtime.dll

Condition that returns `true` when the heal payload is flagged as a critical heal. Supports PreHealContext, ReceivedHealContext, and ResurrectionContext payloads.

```
[Serializable]
[TypeSelectableMenu("Payload/Heal")]
[ConditionPayload(typeof(PreHealContext))]
[ConditionPayload(typeof(ReceivedHealContext))]
[ConditionPayload(typeof(ResurrectionContext))]
public class HealIsCriticalCondition : Condition
```

Inheritance

object ← Condition ← HealIsCriticalCondition

Methods

Evaluate(EvaluationContext)

Evaluates this condition against the provided context.

```
public override bool Evaluate(EvaluationContext ctx)
```

Parameters

`ctx` EvaluationContext

Contextual data available to the condition (holder, performer, payload).

Returns

bool

`true` if the condition is satisfied; otherwise `false`.

Class HealSourceMatchCondition

Namespace: ElectricDrill.AstraHealth.Conditions

Assembly: com.electricdrill.astra-health.Runtime.dll

Condition that returns `true` when the heal payload's source matches the configured reference. Supports PreHealContext, ReceivedHealContext, and ResurrectionContext payloads.

```
[Serializable]
[TypeSelectableMenu("Payload/Heal")]
[ConditionPayload(typeof(PreHealContext))]
[ConditionPayload(typeof(ReceivedHealContext))]
[ConditionPayload(typeof(ResurrectionContext))]
public class HealSourceMatchCondition : Condition
```

Inheritance

object ← Condition ← HealSourceMatchCondition

Methods

Evaluate(EvaluationContext)

Evaluates this condition against the provided context.

```
public override bool Evaluate(EvaluationContext ctx)
```

Parameters

`ctx` EvaluationContext

Contextual data available to the condition (holder, performer, payload).

Returns

bool

`true` if the condition is satisfied; otherwise `false`.

Enum HealthRatioComponent

Namespace: ElectricDrill.AstraHealth.Conditions

Assembly: com.electricdrill.astra-health.Runtime.dll

Selects which HP ratio component to evaluate in a HealthRatioThresholdTransitionCondition.

```
public enum HealthRatioComponent
```

Fields

`CurrentHpPercentage = 0`

HP as a percentage of MaxHP (HP / MaxHP × 100, clamped to [0, 100]). Returns false when MaxHP is zero.

`MissingHpAbsolute = 1`

Missing HP as an absolute value (MaxHP − HP).

`MissingHpPercentage = 2`

Missing HP as a percentage of MaxHP ((MaxHP − HP) / MaxHP × 100, clamped to [0, 100]). Returns false when MaxHP is zero.

Class HealthRatioThresholdTransitionCondition

Namespace: ElectricDrill.AstraHealth.Conditions

Assembly: com.electricdrill.astra-health.Runtime.dll

Condition that detects when an HP ratio metric crosses a threshold in a HealthRatioChangedContext payload.

Fires on the `HealthRatioChanged` event, which is raised whenever HP or MaxHP changes. This makes it reliable for threshold crossing detection regardless of whether HP, MaxHP, or both changed.

Supported components:

- CurrentHpPercentage — HP / MaxHP × 100; uses percentage threshold.
- MissingHpAbsolute — MaxHP − HP; uses absolute threshold.
- MissingHpPercentage — (MaxHP − HP) / MaxHP × 100; uses percentage threshold.

Returns `false` when MaxHP is zero for percentage-based components.

```
[Serializable]
[TypeSelectableMenu("Payload/Health")]
[ConditionPayload(typeof(HealthRatioChangedContext))]
public class HealthRatioThresholdTransitionCondition : Condition
```

Inheritance

object ← Condition ← HealthRatioThresholdTransitionCondition

Methods

Evaluate(EvaluationContext)

Evaluates this condition against the provided context.

```
public override bool Evaluate(EvaluationContext ctx)
```

Parameters

`ctx` EvaluationContext

Contextual data available to the condition (holder, performer, payload).

Returns

bool

`true` if the condition is satisfied; otherwise `false`.

Class HealthThresholdCondition

Namespace: ElectricDrill.AstraHealth.Conditions

Assembly: com.electricdrill.astra-health.Runtime.dll

Condition that compares the resolved target entity's current HP against a threshold.

When ElectricDrill.AstraRpgFramework.Conditions.ThresholdMode.Absolute, compares HP directly against the configured value.
When ElectricDrill.AstraRpgFramework.Conditions.ThresholdMode.Percentage, compares HP% (HP / MaxHP × 100, clamped [0–100]) against the configured percentage threshold. Returns `false` if MaxHP is zero.

```
[Serializable]
[TypeSelectableMenu("Leaf/Entity")]
public class HealthThresholdCondition : Condition
```

Inheritance

object ← Condition ← HealthThresholdCondition

Methods

Evaluate(EvaluationContext)

Evaluates this condition against the provided context.

```
public override bool Evaluate(EvaluationContext ctx)
```

Parameters

`ctx` EvaluationContext

Contextual data available to the condition (holder, performer, payload).

Returns

bool

`true` if the condition is satisfied; otherwise `false`.

Class MaxHealthThresholdCondition

Namespace: ElectricDrill.AstraHealth.Conditions

Assembly: com.electricdrill.astra-health.Runtime.dll

Condition that compares the resolved target entity's maximum HP against an absolute threshold.

```
[Serializable]
[TypeSelectableMenu("Leaf/Entity")]
public class MaxHealthThresholdCondition : Condition
```

Inheritance

object ← Condition ← MaxHealthThresholdCondition

Methods

Evaluate(EvaluationContext)

Evaluates this condition against the provided context.

```
public override bool Evaluate(EvaluationContext ctx)
```

Parameters

`ctx` EvaluationContext

Contextual data available to the condition (holder, performer, payload).

Returns

bool

`true` if the condition is satisfied; otherwise `false`.

Class MissingHealthThresholdCondition

Namespace: ElectricDrill.AstraHealth.Conditions

Assembly: com.electricdrill.astra-health.Runtime.dll

Condition that compares the resolved target entity's missing HP (MaxHP − HP) against a threshold.

When ElectricDrill.AstraRpgFramework.Conditions.ThresholdMode.Absolute, compares missing HP directly against the configured value.
When ElectricDrill.AstraRpgFramework.Conditions.ThresholdMode.Percentage, compares missing HP% ((MaxHP − HP) / MaxHP × 100, clamped [0–100]) against the configured percentage threshold. Returns `false` if MaxHP is zero.

```
[Serializable]
[TypeSelectableMenu("Leaf/Entity")]
public class MissingHealthThresholdCondition : Condition
```

Inheritance

object ← Condition ← MissingHealthThresholdCondition

Methods

Evaluate(EvaluationContext)

Evaluates this condition against the provided context.

```
public override bool Evaluate(EvaluationContext ctx)
```

Parameters

`ctx` EvaluationContext

Contextual data available to the condition (holder, performer, payload).

Returns

bool

`true` if the condition is satisfied; otherwise `false`.

Class PayloadIsReactableCondition

Namespace: ElectricDrill.AstraHealth.Conditions

Assembly: com.electricdrill.astra-health.Runtime.dll

Condition that returns `true` when the event payload implements IReactable and its IsReactable flag is `true`.

Use this to restrict reactive mechanics to first-order events only (direct damage, skills, abilities), preventing infinite chain reactions from secondary effects.

Returns `false` when the payload does not implement IReactable.

```
[Serializable]
[TypeSelectableMenu("Payload/Common")]
[ConditionPayload(typeof(IReactable))]
public class PayloadIsReactableCondition : Condition
```

Inheritance

object ← Condition ← PayloadIsReactableCondition

Methods

Evaluate(EvaluationContext)

Evaluates this condition against the provided context.

```
public override bool Evaluate(EvaluationContext ctx)
```

Parameters

`ctx` EvaluationContext

Contextual data available to the condition (holder, performer, payload).

Returns

bool

`true` if the condition is satisfied; otherwise `false`.

Enum ReasonMatchMode

Namespace: ElectricDrill.AstraHealth.Conditions

Assembly: com.electricdrill.astra-health.Runtime.dll

Determines how multiple DamagePreventionReason flags are matched.

```
public enum ReasonMatchMode
```

Fields

`All = 1`

The payload reasons must include all of the specified flags.

`Any = 0`

The payload reasons must include at least one of the specified flags.

Namespace ElectricDrill.AstraHealth.Config

Classes

AstraHealthConfigProvider

AstraHealthConfigSO

AstraHealthGlobalSettingsSO

Structs

HealthRoundingSettings

Configures how fractional health-combat values are rounded to integer gameplay amounts.

Interfaces

IAstraHealthConfig

Read-only view of the health system configuration used by runtime components. Consumers should read through this interface; mutation is reserved for the concrete AstraHealthConfigSO asset (edited in the Inspector or by package-internal code via its `internal` setters).

Enums

LifestealStatSource

Selects which entity's stats feed the lifesteal rate lookup when the entity dealing damage resolves to a different entity than the one credited for it (see LifestealAttribution).

Class AstraHealthConfigProvider

Namespace: ElectricDrill.AstraHealth.Config

Assembly: com.electricdrill.astra-health.Runtime.dll

```
public static class AstraHealthConfigProvider
```

Inheritance

object ← AstraHealthConfigProvider

Properties

Instance

```
public static IAstraHealthConfig Instance { get; }
```

Property Value

IAstraHealthConfig

Methods

Reset()

```
public static void Reset()
```

WarmUp()

```
public static void WarmUp()
```

Class AstraHealthConfigSO

Namespace: ElectricDrill.AstraHealth.Config

Assembly: com.electricdrill.astra-health.Runtime.dll

```
public class AstraHealthConfigSO : ScriptableObject, IAstraHealthConfig, ITaggable
```

Inheritance

object ← Object ← ScriptableObject ← AstraHealthConfigSO

Implements

IAstraHealthConfig, ITaggable

Properties

DamageStatsAttribution

Experimental — this API may still change. Gets the policy used to resolve the fallback owner whose stats supplement PerformerStats when the performer doesn't define a stat, through the framework's Electric Drill.AstraRpgFramework.EntityCore.Owner edge (see ElectricDrill.AstraRpgFramework.Ownership.Entity Ownership). Default is ElectricDrill.AstraRpgFramework.Ownership.EntityAttribution.Direct, which preserves the pre-ownership behavior (only the performer's own stats are read) for every existing project.

```
public EntityAttribution DamageStatsAttribution { get; }
```

Property Value

EntityAttribution

DefaultDamageCalculationCalculationStrategy

Gets the default strategy used for calculating net damage when none is explicitly specified in the target entity.

```
public DamageCalculationStrategySO DefaultDamageCalculationCalculationStrategy { get; }
```

Property Value

DamageCalculationStrategySO

DefaultExpCollectionStrategy

Gets the default strategy used for determining when and how experience is collected from kills. Individual ExpCollector components can override this with a custom strategy.

```
public ExpCollectionStrategySO DefaultExpCollectionStrategy { get; }
```

Property Value

ExpCollectionStrategySO

DefaultOnDeathGameAction

Gets the default ElectricDrill.AstraRpgFramework.GameActions.GameAction<TContext> to execute when an entity dies, if no specific strategy is defined in the dying entity.

```
public GameAction<IHasEntity> DefaultOnDeathGameAction { get; }
```

Property Value

GameAction<IHasEntity>

DefaultOnResurrectionGameAction

Gets the default ElectricDrill.AstraRpgFramework.GameActions.GameAction<TContext> to execute when an entity is resurrected, if no specific strategy is defined in the resurrected entity.

```
public GameAction<IHasEntity> DefaultOnResurrectionGameAction { get; }
```

Property Value

GameAction<IHasEntity>

DefaultResurrectionSource

Gets the heal source used for resurrections heals.

```
public HealSourceSO DefaultResurrectionSource { get; }
```

Property Value

HealSourceSO

GenericFlatDamageModificationStat

Gets the stat that provides flat damage modification, regardless of type or source. Applied after percentage modifications. Stacks additively with other flat modifications.

```
public StatSO GenericFlatDamageModificationStat { get; }
```

Property Value

StatSO

GenericFlatHealAmountModifierStat

Gets the stat that provides a flat heal amount modification, applied before percentage modifiers.

```
public StatSO GenericFlatHealAmountModifierStat { get; }
```

Property Value

StatSO

GenericLifesteal

Gets the generic lifesteal configuration applied regardless of damage type. When the LifestealStat is set, all damage dealt by an entity triggers lifesteal based on this config, stacking with any type-specific lifesteal from the DamageTypeSO.

```
public LifestealStatConfig GenericLifesteal { get; }
```

Property Value

LifestealStatConfig

GenericPercentageDamageModificationStat

Gets the stat that modifies any damage received with percentage, regardless of type or source. Stacks additively with other percentage damage modifications.

```
public StatSO GenericPercentageDamageModificationStat { get; }
```

Property Value

StatSO

GenericPercentageHealAmountModifierStat

Gets the stat that modifies the amount of health healed with a percentage (e.g., 25 for +25% healing). Percentage heal modifiers are applied after flat heal modifications.

```
public StatSO GenericPercentageHealAmountModifierStat { get; }
```

Property Value

StatSO

GlobalDamageResolutionEvent

Gets the global event raised after damage is fully resolved for any entity (whether applied or prevented). This event is required: all EntityHealth instances use it as the primary damage-resolution channel, and it drives lifesteal calculations.

```
public DamageResolutionGameEvent GlobalDamageResolutionEvent { get; }
```

Property Value

DamageResolutionGameEvent

GlobalEntityDiedEvent

Gets the global event raised when any entity dies. This event is required: all EntityHealth instances use it as the primary death-notification channel.

```
public EntityDiedGameEvent GlobalEntityDiedEvent { get; }
```

Property Value

EntityDiedGameEvent

GlobalEntityHealedEvent

Gets the global event raised after a heal is successfully applied to any entity. Optional — leave null to skip this notification.

```
public EntityHealedGameEvent GlobalEntityHealedEvent { get; }
```

Property Value

EntityHealedGameEvent

GlobalEntityResurrectedEvent

Gets the global event raised when any entity is resurrected. Optional — leave null to skip this notification.

```
public EntityResurrectedGameEvent GlobalEntityResurrectedEvent { get; }
```

Property Value

EntityResurrectedGameEvent

GlobalHealthChangedEvent

Gets the global event raised when any entity's health changes (gained or lost). Optional — leave null to skip this notification.

```
public EntityHealthChangedGameEvent GlobalHealthChangedEvent { get; }
```

Property Value

EntityHealthChangedGameEvent

GlobalHealthRatioChangedEvent

Gets the global event raised whenever any entity's HP/MaxHP ratio changes (fires on both HP changes and MaxHP changes, providing a single reliable signal for HP-ratio-based conditions). Optional — leave null to skip this notification.

```
public HealthRatioChangedGameEvent GlobalHealthRatioChangedEvent { get; }
```

Property Value

HealthRatioChangedGameEvent

GlobalMaxHealthChangedEvent

Gets the global event raised when any entity's maximum health changes. Optional — leave null to skip this notification.

```
public EntityMaxHealthChangedGameEvent GlobalMaxHealthChangedEvent { get; }
```

Property Value

EntityMaxHealthChangedGameEvent

GlobalPreDamageInfoEvent

Gets the global event raised before damage is applied to any entity, allowing listeners to inspect or react to incoming damage before resolution. This event is required: all EntityHealth instances use it as the primary pre-damage channel.

```
public PreDamageGameEvent GlobalPreDamageInfoEvent { get; }
```

Property Value

PreDamageGameEvent

GlobalPreHealEvent

Gets the global event raised before a heal is applied to any entity. Optional — leave null to skip this notification.

```
public PreHealGameEvent GlobalPreHealEvent { get; }
```

Property Value

PreHealGameEvent

HealthAttributesScaling

Gets the attributes scaling component that determines how health scales based on attributes.

```
public AttributesScalingComponentSO HealthAttributesScaling { get; }
```

Property Value

AttributesScalingComponentSO

HealthRegenerationSource

Gets the heal source used for health regeneration effects.

```
public HealSourceSO HealthRegenerationSource { get; }
```

Property Value

HealSourceSO

KillCreditAttribution

Experimental — this API may still change. Gets the policy used to resolve which entity is **credited with a kill** from the damage performer, through the framework's ElectricDrill.AstraRpgFramework.EntityCore.Owner edge (see ElectricDrill.AstraRpgFramework.Ownership.EntityOwnership). Consulted for Performer and by DirectKillExpStrategySO. Default is ElectricDrill.AstraRpgFramework.Ownership.EntityAttribution.Direct, which preserves the pre-ownership behavior for every existing project.

```
public EntityAttribution KillCreditAttribution { get; }
```

Property Value

EntityAttribution

LifestealAttribution

Experimental — this API may still change. Gets the policy used to resolve the lifesteal **beneficiary** from the damage performer, through the framework's ElectricDrill.AstraRpgFramework.EntityCore.Owner edge (see ElectricDrill.AstraRpgFramework.Ownership.EntityOwnership). Consulted by ElectricDrill.AstraHealth.Heal.LifestealResolver. Default is ElectricDrill.AstraRpgFramework.Ownership.EntityAttribution.Direct, which preserves the pre-ownership behavior (the performer is always used as-is) for every existing project.

```
public EntityAttribution LifestealAttribution { get; }
```

Property Value

EntityAttribution

LifestealStatSource

Gets which entity's stats feed the lifesteal rate lookup when LifestealAttribution resolves the damage performer to a different entity. Default is Performer, which preserves pre-ownership behavior.

```
public LifestealStatSource LifestealStatSource { get; }
```

Property Value

LifestealStatSource

ManualHealthRegenerationStat

Gets or sets the stat that determines manual health regeneration amount. Use the API to trigger manual regeneration ticks programmatically.

```
public StatSO ManualHealthRegenerationStat { get; }
```

Property Value

StatSO

PassiveHealthRegenerationInterval

Gets the interval (in seconds) between passive health regeneration ticks. Regenerated health is calculated based on the Passive Health Regeneration Stat.

```
public float PassiveHealthRegenerationInterval { get; }
```

Property Value

float

PassiveHealthRegenerationStat

Gets the stat that determines passive health regeneration amount. Stat value represents health regenerated per 10 seconds.

```
public StatSO PassiveHealthRegenerationStat { get; }
```

Property Value

StatSO

RecordDamageStepTrace

Gets whether the damage calculation pipeline records a per-step amount trace (Records). The trace backs Step lifesteal, is readable by user code from any damage-resolution listener, and feeds the negative-amount pipeline diagnostic — at the cost of one small list allocation per damage instance. Defaults to `true` (identical in the editor and in builds); turn it off only after confirming nothing reads Records and no step-based lifesteal is configured.

```
public bool RecordDamageStepTrace { get; }
```

Property Value

bool

RoundingSettings

Gets the rounding policy used by fractional damage, mitigation, critical and lifesteal calculations.

```
public HealthRoundingSettings RoundingSettings { get; }
```

Property Value

HealthRoundingSettings

SuppressLifestealEvents

Gets whether to suppress PreHeal and ReceivedHeal events for lifesteal healing. When true, lifesteal will not raise heal events, improving performance when many entities deal repeatedly damage with lifesteal. Default is false (events are raised).

```
public bool SuppressLifestealEvents { get; }
```

Property Value

bool

SuppressManualRegenerationEvents

Gets whether to suppress PreHeal and ReceivedHeal events for manual health regeneration. When true, manual regeneration will not raise heal events, improving performance in case of frequent calls. Default is false (events are raised).

```
public bool SuppressManualRegenerationEvents { get; }
```

Property Value

bool

SuppressPassiveRegenerationEvents

Gets whether to suppress PreHeal and ReceivedHeal events for passive health regeneration. When true, passive regeneration will not raise heal events, improving performance for frequent regeneration ticks. Default is false (events are raised).

```
public bool SuppressPassiveRegenerationEvents { get; }
```

Property Value

bool

Tags

The tags associated with this object.

```
public GameTagSet Tags { get; }
```

Property Value

UnifyLifestealHeals

When true, generic and type-specific lifesteal amounts are summed into a single heal. The HealSource of the damage type takes precedence over the generic one (falls back to the generic source if only generic lifesteal is active or type has no source set). When false (default), each configured lifesteal fires an independent heal with its own HealSource.

```
public bool UnifyLifestealHeals { get; }
```

Property Value

bool

Class AstraHealthGlobalSettingsSO

Namespace: ElectricDrill.AstraHealth.Config

Assembly: com.electricdrill.astra-health.Runtime.dll

```
public class AstraHealthGlobalSettingsSO : ScriptableObject
```

Inheritance

object ← Object ← ScriptableObject ← AstraHealthGlobalSettingsSO

Fields

AssetPath

Full asset path for the global settings asset (used in Editor)

```
public const string AssetPath = "Assets/Resources/AstraHealthGlobalSettings.asset"
```

Field Value

string

DefaultConfigResourceName

Resource name for the default/fallback health config

```
public const string DefaultConfigResourceName = "Astra Health Config"
```

Field Value

string

ResourcePath

Resource path (without extension) for the global settings asset

```
public const string ResourcePath = "AstraHealthGlobalSettings"
```

Field Value

string

Properties

ActiveConfig

Gets or sets the active Astra Health configuration profile used by the system.

```
public AstraHealthConfigSO ActiveConfig { get; set; }
```

Property Value

AstraHealthConfigSO

Struct HealthRoundingSettings

Namespace: ElectricDrill.AstraHealth.Config

Assembly: com.electricdrill.astra-health.Runtime.dll

Configures how fractional health-combat values are rounded to integer gameplay amounts.

```
[Serializable]
public struct HealthRoundingSettings
```

Constructors

HealthRoundingSettings(RoundingMode, RoundingMode, RoundingMode, RoundingMode, RoundingMode)

Initializes a new instance of HealthRoundingSettings.

```
public HealthRoundingSettings(RoundingMode percentageDamageModifierRoundingMode,
RoundingMode defensePenetrationRoundingMode, RoundingMode damageMitigationRoundingMode,
RoundingMode criticalDamageMultiplierRoundingMode, RoundingMode lifestealRoundingMode)
```

Parameters

`percentageDamageModifierRoundingMode` RoundingMode

Rounding applied to percentage damage modifier results.

`defensePenetrationRoundingMode` RoundingMode

Rounding applied to effective defense after penetration.

`damageMitigationRoundingMode` RoundingMode

Rounding applied to damage after mitigation.

`criticalDamageMultiplierRoundingMode` RoundingMode

Rounding applied to damage after critical multipliers.

`lifestealRoundingMode` RoundingMode

Rounding applied to lifesteal heal amounts.

Properties

CriticalDamageMultiplierRoundingMode

Gets the rounding mode applied after critical damage multipliers.

```
public RoundingMode CriticalDamageMultiplierRoundingMode { get; }
```

Property Value

RoundingMode

DamageMitigationRoundingMode

Gets the rounding mode applied to the final damage amount returned by damage mitigation.

```
public RoundingMode DamageMitigationRoundingMode { get; }
```

Property Value

RoundingMode

Default

Default rounding settings used when no health configuration is available.

```
public static HealthRoundingSettings Default { get; }
```

Property Value

HealthRoundingSettings

DefensePenetrationRoundingMode

Gets the rounding mode applied to the effective defense value after defense penetration.

```
public RoundingMode DefensePenetrationRoundingMode { get; }
```

Property Value

RoundingMode

LifestealRoundingMode

Gets the rounding mode applied to lifesteal heal amounts.

```
public RoundingMode LifestealRoundingMode { get; }
```

Property Value

RoundingMode

PercentageDamageModifierRoundingMode

Gets the rounding mode applied after percentage damage modifiers are combined and applied.

```
public RoundingMode PercentageDamageModifierRoundingMode { get; }
```

Property Value

RoundingMode

Interface IAstraHealthConfig

Namespace: ElectricDrill.AstraHealth.Config

Assembly: com.electricdrill.astra-health.Runtime.dll

Read-only view of the health system configuration used by runtime components. Consumers should read through this interface; mutation is reserved for the concrete AstraHealthConfigSO asset (edited in the Inspector or by package-internal code via its `internal` setters).

```
public interface IAstraHealthConfig
```

Properties

DamageStatsAttribution

Experimental — this API may still change. Gets the policy used to resolve the fallback owner whose stats supplement PerformerStats when the performer doesn't define a stat, through the framework's Electric Drill.AstraRpgFramework.EntityCore.Owner edge (see ElectricDrill.AstraRpgFramework.Ownership.Entity Ownership). Default is ElectricDrill.AstraRpgFramework.Ownership.EntityAttribution.Direct, which preserves the pre-ownership behavior (only the performer's own stats are read) for every existing project.

```
EntityAttribution DamageStatsAttribution { get; }
```

Property Value

EntityAttribution

DefaultDamageCalculationCalculationStrategy

Gets the default strategy used for calculating net damage when none is explicitly specified in the target entity.

```
DamageCalculationStrategySO DefaultDamageCalculationCalculationStrategy { get; }
```

Property Value

DamageCalculationStrategySO

DefaultExpCollectionStrategy

Gets the default strategy used for determining when and how experience is collected from kills. Individual ExpCollector components can override this with a custom strategy.

```
ExpCollectionStrategySO DefaultExpCollectionStrategy { get; }
```

Property Value

ExpCollectionStrategySO

DefaultOnDeathGameAction

Gets the default ElectricDrill.AstraRpgFramework.GameActions.GameAction<TContext> to execute when an entity dies, if no specific strategy is defined in the dying entity.

```
GameAction<IHasEntity> DefaultOnDeathGameAction { get; }
```

Property Value

GameAction<IHasEntity>

DefaultOnResurrectionGameAction

Gets the default ElectricDrill.AstraRpgFramework.GameActions.GameAction<TContext> to execute when an entity is resurrected, if no specific strategy is defined in the resurrected entity.

```
GameAction<IHasEntity> DefaultOnResurrectionGameAction { get; }
```

Property Value

GameAction<IHasEntity>

DefaultResurrectionSource

Gets the heal source used for resurrections heals.

```
HealSourceSO DefaultResurrectionSource { get; }
```

Property Value

HealSourceSO

GenericFlatDamageModificationStat

Gets the stat that provides flat damage modification, regardless of type or source. Applied after percentage modifications. Stacks additively with other flat modifications.

```
StatSO GenericFlatDamageModificationStat { get; }
```

Property Value

StatSO

GenericFlatHealAmountModifierStat

Gets the stat that provides a flat heal amount modification, applied before percentage modifiers.

```
StatSO GenericFlatHealAmountModifierStat { get; }
```

Property Value

StatSO

GenericLifesteal

Gets the generic lifesteal configuration applied regardless of damage type. When the LifestealStat is set, all damage dealt by an entity triggers lifesteal based on this config, stacking with any type-specific lifesteal from the DamageTypeSO.

```
LifestealStatConfig GenericLifesteal { get; }
```

Property Value

LifestealStatConfig

GenericPercentageDamageModificationStat

Gets the stat that modifies any damage received with percentage, regardless of type or source. Stacks additively with other percentage damage modifications.

```
StatSO GenericPercentageDamageModificationStat { get; }
```

Property Value

StatSO

GenericPercentageHealAmountModifierStat

Gets the stat that modifies the amount of health healed with a percentage (e.g., 25 for +25% healing). Percentage heal modifiers are applied after flat heal modifications.

```
StatSO GenericPercentageHealAmountModifierStat { get; }
```

Property Value

StatSO

GlobalDamageResolutionEvent

Gets the global event raised after damage is fully resolved for any entity (whether applied or prevented). This event is required: all EntityHealth instances use it as the primary damage-resolution channel, and it drives lifesteal calculations.

```
DamageResolutionGameEvent GlobalDamageResolutionEvent { get; }
```

Property Value

DamageResolutionGameEvent

GlobalEntityDiedEvent

Gets the global event raised when any entity dies. This event is required: all EntityHealth instances use it as the primary death-notification channel.

```
EntityDiedGameEvent GlobalEntityDiedEvent { get; }
```

Property Value

EntityDiedGameEvent

GlobalEntityHealedEvent

Gets the global event raised after a heal is successfully applied to any entity. Optional — leave null to skip this notification.

```
EntityHealedGameEvent GlobalEntityHealedEvent { get; }
```

Property Value

EntityHealedGameEvent

GlobalEntityResurrectedEvent

Gets the global event raised when any entity is resurrected. Optional — leave null to skip this notification.

```
EntityResurrectedGameEvent GlobalEntityResurrectedEvent { get; }
```

Property Value

EntityResurrectedGameEvent

GlobalHealthChangedEvent

Gets the global event raised when any entity's health changes (gained or lost). Optional — leave null to skip this notification.

```
EntityHealthChangedGameEvent GlobalHealthChangedEvent { get; }
```

Property Value

EntityHealthChangedGameEvent

GlobalHealthRatioChangedEvent

Gets the global event raised whenever any entity's HP/MaxHP ratio changes (fires on both HP changes and MaxHP changes, providing a single reliable signal for HP-ratio-based conditions). Optional — leave null to skip this notification.

```
HealthRatioChangedGameEvent GlobalHealthRatioChangedEvent { get; }
```

Property Value

HealthRatioChangedGameEvent

GlobalMaxHealthChangedEvent

Gets the global event raised when any entity's maximum health changes. Optional — leave null to skip this notification.

```
EntityMaxHealthChangedGameEvent GlobalMaxHealthChangedEvent { get; }
```

Property Value

EntityMaxHealthChangedGameEvent

GlobalPreDamageInfoEvent

Gets the global event raised before damage is applied to any entity, allowing listeners to inspect or react to incoming damage before resolution. This event is required: all EntityHealth instances use it as the primary pre-damage channel.

```
PreDamageGameEvent GlobalPreDamageInfoEvent { get; }
```

Property Value

PreDamageGameEvent

GlobalPreHealEvent

Gets the global event raised before a heal is applied to any entity. Optional — leave null to skip this notification.

```
PreHealGameEvent GlobalPreHealEvent { get; }
```

Property Value

PreHealGameEvent

HealthAttributesScaling

Gets the attributes scaling component that determines how health scales based on attributes.

```
AttributesScalingComponentSO HealthAttributesScaling { get; }
```

Property Value

AttributesScalingComponentSO

HealthRegenerationSource

Gets the heal source used for health regeneration effects.

```
HealSourceSO HealthRegenerationSource { get; }
```

Property Value

HealSourceSO

KillCreditAttribution

Experimental — this API may still change. Gets the policy used to resolve which entity is **credited with a kill** from the damage performer, through the framework's ElectricDrill.AstraRpgFramework.EntityCore.Owner edge (see ElectricDrill.AstraRpgFramework.Ownership.EntityOwnership). Consulted for Performer and by DirectKillExpStrategySO. Default is ElectricDrill.AstraRpgFramework.Ownership.EntityAttribution.Direct, which preserves the pre-ownership behavior for every existing project.

```
EntityAttribution KillCreditAttribution { get; }
```

Property Value

EntityAttribution

LifestealAttribution

Experimental — this API may still change. Gets the policy used to resolve the lifesteal **beneficiary** from the damage performer, through the framework's ElectricDrill.AstraRpgFramework.EntityCore.Owner edge (see ElectricDrill.AstraRpgFramework.Ownership.EntityOwnership). Consulted by ElectricDrill.AstraHealth.Heal.LifestealResolver. Default is ElectricDrill.AstraRpgFramework.Ownership.EntityAttribution.Direct, which preserves the pre-ownership behavior (the performer is always used as-is) for every existing project.

```
EntityAttribution LifestealAttribution { get; }
```

Property Value

EntityAttribution

LifestealStatSource

Gets which entity's stats feed the lifesteal rate lookup when LifestealAttribution resolves the damage performer to a different entity. Default is Performer, which preserves pre-ownership behavior.

```
LifestealStatSource LifestealStatSource { get; }
```

Property Value

LifestealStatSource

ManualHealthRegenerationStat

Gets the stat that determines manual health regeneration amount. Use the API to trigger manual regeneration ticks programmatically. Useful for turn-based systems.

```
StatSO ManualHealthRegenerationStat { get; }
```

Property Value

StatSO

PassiveHealthRegenerationInterval

Gets the interval (in seconds) between passive health regeneration ticks. Regenerated health is calculated based on the Passive Health Regeneration Stat.

```
float PassiveHealthRegenerationInterval { get; }
```

Property Value

float

PassiveHealthRegenerationStat

Gets the stat that determines passive health regeneration amount. Stat value represents health regenerated per 10 seconds.

```
StatSO PassiveHealthRegenerationStat { get; }
```

Property Value

StatSO

RecordDamageStepTrace

Gets whether the damage calculation pipeline records a per-step amount trace (Records). The trace backs Step lifesteal, is readable by user code from any damage-resolution listener, and feeds the negative-amount pipeline diagnostic — at the cost of one small list allocation per damage instance. Defaults to `true` (identical in the editor and in builds); turn it off only after confirming nothing reads Records and no step-based lifesteal is configured.

```
bool RecordDamageStepTrace { get; }
```

Property Value

bool

RoundingSettings

Gets the rounding policy used by fractional damage, mitigation, critical and lifesteal calculations.

```
HealthRoundingSettings RoundingSettings { get; }
```

Property Value

HealthRoundingSettings

SuppressLifestealEvents

Gets whether to suppress PreHeal and ReceivedHeal events for lifesteal healing. When true, lifesteal will not raise heal events, improving performance when many entities deal repeatedly damage with lifesteal. Default is false (events are raised).

```
bool SuppressLifestealEvents { get; }
```

Property Value

bool

SuppressManualRegenerationEvents

Gets whether to suppress PreHeal and ReceivedHeal events for manual health regeneration. When true, manual regeneration will not raise heal events, improving performance in case of frequent calls. Default is false (events are raised).

```
bool SuppressManualRegenerationEvents { get; }
```

Property Value

bool

SuppressPassiveRegenerationEvents

Gets whether to suppress PreHeal and ReceivedHeal events for passive health regeneration. When true, passive regeneration will not raise heal events, improving performance for frequent regeneration ticks. Default is false (events are raised).

```
bool SuppressPassiveRegenerationEvents { get; }
```

Property Value

bool

UnifyLifestealHeals

When true, generic and type-specific lifesteal amounts are summed into a single heal. The HealSource of the damage type takes precedence over the generic one (falls back to the generic source if only generic lifesteal is active or type has no source set). When false (default), each configured lifesteal fires an independent heal with its own HealSource.

```
bool UnifyLifestealHeals { get; }
```

Property Value

bool

Enum LifestealStatSource

Namespace: ElectricDrill.AstraHealth.Config

Assembly: com.electricdrill.astra-health.Runtime.dll

Selects which entity's stats feed the lifesteal rate lookup when the entity dealing damage resolves to a different entity than the one credited for it (see LifestealAttribution).

```
public enum LifestealStatSource
```

Fields

`Beneficiary = 1`

Use the lifesteal rate stat from the resolved beneficiary only (the entity attribution resolves to, e.g. the ship owning the weapon).

`Performer = 0`

Use the lifesteal rate stat from the damage performer only (the direct dealer, e.g. a weapon entity). Preserves the pre-ownership behavior.

`PerformerThenBeneficiary = 2`

Use the performer's lifesteal rate stat if present, otherwise fall back to the beneficiary's.

`Sum = 3`

Add the performer's and the beneficiary's lifesteal rate stats together.

Namespace ElectricDrill.AstraHealth.Core

Classes

EntityHealth

Manages the health, damage, and healing mechanics for an entity. Handles damage calculation, health regeneration, barriers (temporary HP), and death events.

EntityPassiveHpRegeneration

Adds passive health regeneration to an entity over time. Attach this component to any entity that should automatically regenerate health. The regeneration amount per tick is driven by the Passive HealthRegenerationStat and the tick interval by PassiveHealthRegenerationInterval. Entities that do not require passive regeneration should not carry this component.

Interfaces

IReactable

Marks a context as carrying a reactability flag that signals whether the receiver of the associated event should act on it.

First-order events (direct attacks, skills, abilities) set IsReactable to `true`. Secondary events produced by counter-damage actions, passive modifiers, or other reactive effects set it to `false` to prevent further downstream reactions and break potential infinite chains.

Enums

EntityHealth.HpBehaviourOnMaxHpDecrease

EntityHealth.HpBehaviourOnMaxHpIncrease

Class EntityHealth

Namespace: ElectricDrill.AstraHealth.Core

Assembly: com.electricdrill.astra-health.Runtime.dll

Manages the health, damage, and healing mechanics for an entity. Handles damage calculation, health regeneration, barriers (temporary HP), and death events.

```
[RequireComponent(typeof(EntityCore))]
public class EntityHealth : MonoBehaviour, IDamageable, IHealable, IResurrectable,
IHasEntity, IBarrierContainer, IEventRegistrar
```

Inheritance

object ← Object ← Component ← Behaviour ← MonoBehaviour ← EntityHealth

Implements

IDamageable, IHealable, IResurrectable, IHasEntity, IBarrierContainer, IEventRegistrar

Remarks

Internal serialized fields — access contract

Several fields in this class are marked `[SerializeField, HideInInspector] internal`. This combination encodes a deliberate three-way contract:

- `[SerializeField]` — Unity persists the value to the scene/prefab asset.
- `[HideInInspector]` — the default Inspector hides the field; the custom `EntityHealthEditor` owns the entire UI and reads/writes values exclusively via `SerializedProperty` (e.g., `FindProperty("_hp")`).
- `internal` — accessible to package-internal helpers (ElectricDrill.AstraHealth.Core.MaxHpCalculator, `LifestealResolver`, etc.) that live in the same Runtime assembly, and to test assemblies declared in `AssemblyInfo.cs` via `InternalsVisibleTo`.

Any direct write to these fields from outside `EntityHealth` or its designated package-internal helpers is a bug. The only legitimate write paths are:

1. Unity deserialization (scene/prefab load).
2. `EntityHealthEditor` via `SerializedProperty` (Inspector).
3. Package-internal helpers (`MaxHpCalculator`, etc.) during runtime logic.
4. Test setup code in the sibling test assembly.

Properties

Barrier

Gets the current barrier points (temporary HP) of the entity.

```
public long Barrier { get; }
```

Property Value

long

BaseMaxHp

The constant base max HP, before flat/percentage/scaling modifiers. Setting it overwrites the value and recomputes total max HP, topping current HP up to the new max when it grows. The LongRef is kept in constant mode, so the value stays private to this instance and never aliases a shared ElectricDrill.AstraRpgFramework.Utils.LongVarSO. Intended for stamping a shared prefab into per-type variants at spawn (e.g. enemy kinds); ignored while ElectricDrill.AstraHealth.Core.EntityHealth._useClassMaxHp is set, since the class then owns the base value.

```
public long BaseMaxHp { get; set; }
```

Property Value

long

Class

Gets or sets the EntityClass component associated with this entity.

```
public EntityClass Class { get; set; }
```

Property Value

EntityClass

Entity

The primary ElectricDrill.AstraRpgFramework.EntityCore associated with this object. For entity components this is the owning entity; for context payloads this is the primary affected subject (e.g. the event target).

```
public EntityCore Entity { get; }
```

Property Value

EntityCore

EntityAttributes

Gets or sets the EntityAttributes component associated with this entity.

```
public EntityAttributes EntityAttributes { get; set; }
```

Property Value

EntityAttributes

EntityCore

Gets or sets the EntityCore component associated with this entity.

```
public EntityCore EntityCore { get; set; }
```

Property Value

EntityCore

EntityStats

Gets or sets the EntityStats component associated with this entity.

```
public virtual EntityStats EntityStats { get; set; }
```

Property Value

EntityStats

HealthCanBeNegative

Gets or sets whether the entity's health can go below zero. When set to true, a death threshold must be defined.

```
public bool HealthCanBeNegative { get; set; }
```

Property Value

bool

Hp

Gets or sets the current health points of the entity.

```
public long Hp { get; set; }
```

Property Value

long

Remarks

The setter is a raw assignment: it raises no health, damage or resurrection event, applies no modifier, and does not touch Barrier. It exists for restoring an entity's state wholesale — chiefly recycling a pooled entity into a new life — where every one of those side effects would be wrong. Ordinary gameplay changes health through `Heal`, `TakeDamage` and `Resurrect`, which do all of the above on purpose.

Assigning on a dead entity is legal and is the point: `_isDead` is recomputed from HP in `OnEnable`, so a dead entity restored to full HP while deactivated comes back alive on its next activation. `Resurrect` would instead route through `Heal` — applying healing modifiers, raising the resurrection channel and allocating contexts — none of which a recycled entity wants.

IsImmune

Gets or sets whether the entity is immune to all damage.

```
public bool IsImmune { get; set; }
```

Property Value

bool

MaxHp

Gets the maximum health points of the entity.

```
public long MaxHp { get; }
```

Property Value

long

OverrideOnDeathGameAction

Gets or sets the override ElectricDrill.AstraRpgFramework.GameActions.GameAction<TContext> for handling entity death. This game action takes precedence over the default game action defined in the config. If null, the default game action from the AstraHealthConfigSO configuration will be used.

```
public GameAction<IHasEntity> OverrideOnDeathGameAction { get; set; }
```

Property Value

GameAction<IHasEntity>

OverrideOnResurrectionGameAction

Gets or sets the override ElectricDrill.AstraRpgFramework.GameActions.GameAction<TContext> for handling entity resurrection. This game action takes precedence over the default game action defined in

the config. If null, the default game action from the AstraHealthConfigSO configuration will be used.

```
public GameAction<IHasEntity> OverrideOnResurrectionGameAction { get; set; }
```

Property Value

GameAction<IHasEntity>

Methods

AddBarrier(long)

Adds the specified amount to the entity's barrier pool.

```
public void AddBarrier(long amount)
```

Parameters

`amount` long

Positive amount to add.

AddMaxHpFlatModifier(long, HpBehaviourOnMaxHpIncrease, HpBehaviourOnMaxHpDecrease)

Adds a flat modifier to the entity's maximum health.

```
public void AddMaxHpFlatModifier(long amount, EntityHealth.HpBehaviourOnMaxHpIncrease
onMaxHpIncrease = HpBehaviourOnMaxHpIncrease.None, EntityHealth.HpBehaviourOnMaxHpDecrease
onMaxHpDecrease = HpBehaviourOnMaxHpDecrease.RemoveHealthUpToMaxHp)
```

Parameters

`amount` long

The amount to add to the flat modifier. Can be negative.

`onMaxHpIncrease` EntityHealth.HpBehaviourOnMaxHpIncrease

Behavior to apply when max HP increases.

`onMaxHpDecrease` EntityHealth.HpBehaviourOnMaxHpDecrease

Behavior to apply when max HP decreases.

AddMaxHpPercentageModifier(Percentage, HpBehaviourOnMaxHpIncrease, HpBehaviourOnMaxHpDecrease)

Adds a percentage modifier to the entity's maximum health.

```
public void AddMaxHpPercentageModifier(Percentage amount,
EntityHealth.HpBehaviourOnMaxHpIncrease onMaxHpIncrease = HpBehaviourOnMaxHpIncrease.None,
EntityHealth.HpBehaviourOnMaxHpDecrease onMaxHpDecrease =
HpBehaviourOnMaxHpDecrease.RemoveHealthUpToMaxHp)
```

Parameters

`amount` Percentage

The percentage to add to the modifier.

`onMaxHpIncrease` EntityHealth.HpBehaviourOnMaxHpIncrease

Behavior to apply when max HP increases.

`onMaxHpDecrease` EntityHealth.HpBehaviourOnMaxHpDecrease

Behavior to apply when max HP decreases.

AddMaxHpScaling(StatsScalingComponentSO, HpBehaviourOnMaxHpIncrease, HpBehaviourOnMaxHpDecrease)

Adds a stats-based scaling component that dynamically affects maximum health. Subscribes to stat changes if this is the first scaling added.

```
public void AddMaxHpScaling(StatsScalingComponentSO scaling,
EntityHealth.HpBehaviourOnMaxHpIncrease onMaxHpIncrease = HpBehaviourOnMaxHpIncrease.None,
```

```
EntityHealth.HpBehaviourOnMaxHpDecrease onMaxHpDecrease =
HpBehaviourOnMaxHpDecrease.RemoveHealthUpToMaxHp)
```

Parameters

`scaling` StatsScalingComponentSO

The scaling component to add.

`onMaxHpIncrease` EntityHealth.HpBehaviourOnMaxHpIncrease

Behavior to apply when max HP increases because of the scaling.

`onMaxHpDecrease` EntityHealth.HpBehaviourOnMaxHpDecrease

Behavior to apply when max HP decreases because of the scaling.

ClearMaxHpScalings(HpBehaviourOnMaxHpIncrease, HpBehaviourOnMaxHpDecrease)

Removes all stats-based scaling components from maximum health calculations.

```
public void ClearMaxHpScalings(EntityHealth.HpBehaviourOnMaxHpIncrease onMaxHpIncrease =
HpBehaviourOnMaxHpIncrease.None, EntityHealth.HpBehaviourOnMaxHpDecrease onMaxHpDecrease =
HpBehaviourOnMaxHpDecrease.RemoveHealthUpToMaxHp)
```

Parameters

`onMaxHpIncrease` EntityHealth.HpBehaviourOnMaxHpIncrease

`onMaxHpDecrease` EntityHealth.HpBehaviourOnMaxHpDecrease

GetHpPortion(double)

Returns the amount of current HP represented by the given portion.

```
public long GetHpPortion(double portion)
```

Parameters

`portion` double

The portion of current HP (e.g., 0.2 means 20%).

Returns

long

GetMaxHpPortion(double)

Returns the amount of max HP represented by the given portion.

```
public long GetMaxHpPortion(double portion)
```

Parameters

`portion` double

The portion of max HP (e.g., 0.2 means 20%).

Returns

long

GetMissingHpPortion(double)

Returns the amount of missing HP represented by the given portion.

```
public long GetMissingHpPortion(double portion)
```

Parameters

`portion` double

The portion of missing HP (e.g., 0.2 means 20%).

Returns

long

Heal(PreHealContext)

Heals the entity by the specified amount, applying critical multipliers and heal modifiers. The actual health gained may be less than the heal amount if the entity is at or near maximum health. Throws DeadEntityException if the entity is dead.

Heal Calculation:

1. Apply critical multiplier if the heal is flagged as critical.
2. Apply generic heal modifier stat (if configured).
3. Apply heal source-specific modifier stat (if configured and heal source matches).
4. Add the final calculated heal amount to current health (capped at max HP).

Heal modifiers stack additively (e.g., +25% generic heal + +10% source-specific = +35% total).

```
public ReceivedHealContext Heal(PreHealContext context)
```

Parameters

`context` PreHealContext

The pre-heal information including amount, source, and healer.

Returns

ReceivedHealContext

Information about the heal received including the actual health gained.

Exceptions

DeadEntityException

Thrown when attempting to heal a dead entity.

Heal(PreHealContext, bool)

Heals the entity by the specified amount, applying critical multipliers and heal modifiers. The actual health gained may be less than the heal amount if the entity is at or near maximum health. Throws Dead

EntityException if the entity is dead.

Event behaviour:

- When `suppressHealEvents` is `true`, the pre-heal event (`GlobalPreHealInfoEvent`) and the entity-healed event (`GlobalEntityHealedEvent`) are **not** raised.
- The HP-change events (`GlobalHealthChangedEvent` and `GlobalHealthRatioChangedEvent`) are **always** raised when health actually changes, regardless of this flag. UI components tracking entity health depend on these events to stay in sync.

Use `suppressHealEvents: true` when the heal is an implementation detail (e.g. passive regeneration, lifesteal) and raising the full heal event pipeline would cause unwanted side-effects or noise.

```
public ReceivedHealContext Heal(PreHealContext context, bool suppressHealEvents)
```

Parameters

`context` PreHealContext

The pre-heal information including amount, source, and healer.

`suppressHealEvents` bool

When `true`, suppresses the pre-heal and entity-healed event channels. HP-change channels are always raised.

Returns

ReceivedHealContext

Information about the heal received including the actual health gained.

Exceptions

DeadEntityException

Thrown when attempting to heal a dead entity.

IsAlive()

Checks if the entity's current health is above the death threshold.

```
public bool IsAlive()
```

Returns

bool

true if current health is > death threshold, false otherwise

IsDead()

Checks if the entity's current health is at or below the death threshold. Returns the cached death state for performance.

```
public bool IsDead()
```

Returns

bool

true if current health is <= death threshold, false otherwise

ManualHealthRegenerationTick()

Method to be called to trigger manual health regeneration (e.g., if you are using a turn-based system, call this method at the start/end of each turn).

```
public void ManualHealthRegenerationTick()
```

RemoveBarrier(long)

Removes the specified amount from the entity's barrier pool, flooring at zero.

```
public void RemoveBarrier(long amount)
```

Parameters

`amount` long

Positive amount to consume.

RemoveMaxHpScaling(StatsScalingComponentSO, HpBehaviourOnMaxHpIncrease, HpBehaviourOnMaxHpDecrease)

Removes a previously added stats-based scaling component from maximum health calculations. Unsubscribes from stat changes if no scalings remain.

```
public void RemoveMaxHpScaling(StatsScalingComponentSO scaling,
EntityHealth.HpBehaviourOnMaxHpIncrease onMaxHpIncrease = HpBehaviourOnMaxHpIncrease.None,
EntityHealth.HpBehaviourOnMaxHpDecrease onMaxHpDecrease =
HpBehaviourOnMaxHpDecrease.RemoveHealthUpToMaxHp)
```

Parameters

`scaling` StatsScalingComponentSO

The scaling component to remove.

`onMaxHpIncrease` EntityHealth.HpBehaviourOnMaxHpIncrease

Behavior to apply when max HP increases because of the scaling removal.

`onMaxHpDecrease` EntityHealth.HpBehaviourOnMaxHpDecrease

Behavior to apply when max HP decreases because of the scaling removal.

Resurrect(PreHealContext)

Resurrects the entity using the provided PreHealContext, routing through Heal(PreHealContext) so that all heal modifiers (generic and HealSourceSO-specific) are applied. The Amount is the **base** heal amount before modifiers; the actual HP after resurrection may differ. If the modifiers reduce the effective HP to or below the death threshold, a fallback of `deathThreshold + 1` is applied to guarantee the entity is always resurrected alive. Throws System.InvalidOperationException if the entity is already alive.

```
public void Resurrect(PreHealContext preHealContext)
```

Parameters

`preHealContext` PreHealContext

The pre-heal context describing amount, source and healer for the resurrection heal.

Exceptions

InvalidOperationException

Thrown when attempting to resurrect an entity that is already alive.

Resurrect(Percentage)

Convenience overload. Resurrects the entity with a percentage of its maximum HP. Builds a PreHeal Context using the DefaultResurrectionSource and no healer entity. For full control over source and healer use Resurrect(PreHealContext). Throws System.InvalidOperationException if the entity is already alive.

```
public void Resurrect(Percentage withHpPercent)
```

Parameters

`withHpPercent` Percentage

The percentage of maximum HP to restore on resurrection (before heal modifiers). Expected range is [0, 100] (i.e. 0% to 100%). Values outside this range trigger a UnityEngine.Debug.LogWarning(object) and are clamped to [0, 100] before use.

Exceptions

InvalidOperationException

Thrown when attempting to resurrect an entity that is already alive.

Resurrect(long)

Convenience overload. Resurrects the entity with a base amount of HP. Builds a PreHealContext using the DefaultResurrectionSource and no healer entity. For full control over source and healer use Resurrect(PreHealContext). Throws System.InvalidOperationException if the entity is already alive.

```
public void Resurrect(long withHp)
```

Parameters

`withHp` long

The base HP amount to restore on resurrection (before heal modifiers).

Exceptions

InvalidOperationException

Thrown when attempting to resurrect an entity that is already alive.

SetHpToMax()

Sets the entity's current health to its maximum health value. Throws DeadEntityException if the entity is dead.

```
public void SetHpToMax()
```

Exceptions

DeadEntityException

Thrown when attempting to set HP to max on a dead entity.

SetupMaxHp(HpBehaviourOnMaxHpIncrease, HpBehaviourOnMaxHpDecrease)

Recalculates and updates the entity's maximum health based on base value, modifiers, and scaling. Raises the max health changed event if the value differs from the previous maximum. If the entity is dead, only recalculates max HP and raises events without adjusting current HP.

```
public void SetupMaxHp(EntityHealth.HpBehaviourOnMaxHpIncrease onMaxHpIncrease,
EntityHealth.HpBehaviourOnMaxHpDecrease onMaxHpDecrease =
HpBehaviourOnMaxHpDecrease.RemoveHealthUpToMaxHp)
```

Parameters

`onMaxHpIncrease` EntityHealth.HpBehaviourOnMaxHpIncrease

Behavior to apply when max HP increases.

`onMaxHpDecrease` EntityHealth.HpBehaviourOnMaxHpDecrease

Behavior to apply when max HP decreases.

Subscribe<TEvent>(TEvent)

Registers `gameEvent` on the channel matching `TEvent`.

```
public void Subscribe<TEvent>(TEvent gameEvent) where TEvent : ScriptableObject
```

Parameters

`gameEvent` TEvent

Type Parameters

`TEvent`

Exceptions

ArgumentException

Thrown when no channel is registered for `TEvent`.

TakeDamage(PreDamageContext)

Applies damage to the entity through the damage calculation pipeline. This is the primary method for dealing damage in the Astra Health system.

Execution Order:

1. **Death State Check**: If the entity is already dead, the damage is marked as prevented with EntityDead reason and returned immediately. Dead entities cannot take damage.

2. **Pre-Damage Event**: Raises the PreDamageGameEvent to notify listeners of incoming damage. This allows systems to react before any damage calculations occur.
3. **DamageInfo Creation**: Converts the PreDamageContext into a DamageInfo object that will track the damage through the calculation pipeline.
4. **Immunity Check**: If the entity is immune (IsImmune is true), the damage is marked as prevented with EntityImmune reason. This happens before the pipeline to short-circuit processing.
5. **Damage Calculation Pipeline**: If damage is not already prevented, the configured DamageCalculationStrategy processes the damage. The pipeline can apply:
    - Damage mitigation based on defensive stats
    - Barrier consumption (temporary HP absorption)
    - Critical hit calculations
    - General damage and/or damage type and damage source modifiers
    - Custom pipeline stages defined in the strategy

   The pipeline can also mark damage as prevented for various reasons.
6. **Prevention Check**: If the damage was prevented (either by immunity or during the pipeline):
    - Creates a DamageResolutionContext.Prevented result with all prevention reasons
    - Raises the DamageResolutionGameEvent
    - Returns immediately without affecting health
7. **Health Reduction**: If damage was not prevented, the final calculated damage amount is subtracted from the entity's current health via RemoveHealth(). This may trigger the HealthChangedGameEvent.
8. **Damage Resolution Event**: Raises the DamageResolutionGameEvent with the applied damage information. This allows systems to react to successful damage (e.g., lifesteal, on-hit effects).
9. **Death Check**: If the entity's health is now at or below the death threshold:
    - Raises the EntityDiedGameEvent with this EntityHealth and the DamageResolutionContext
    - Executes the on-death Game Action (either the entity's override or the config default)
    - The death strategy handles the actual death behavior (e.g., destroy, return ot the object pool, disable, ragdoll)
10. **Return**: Returns a DamageResolutionContext.Applied result containing all damage information for the caller to process if needed.

Strategy Selection:

The method uses the first available DamageCalculationStrategy in this priority order:

1. OverrideDamageCalculationStrategy (if set on this EntityHealth)
2. CustomDamageCalculationStrategy (if set on this EntityHealth)
3. DefaultDamageCalculationStrategy (from the AstraRpgHealthConfig)

```
public DamageResolutionContext TakeDamage(PreDamageContext preDamage)
```

Parameters

`preDamage` PreDamageContext

The pre-damage information including base amount, damage type, dealer entity, and additional context. This is used by the damage calculation pipeline to compute the final damage to apply.

Returns

DamageResolutionContext

A DamageResolutionContext indicating whether damage was applied or prevented. If prevented, includes the reasons for prevention. If applied, includes the final damage amounts and calculation details.

Unsubscribe<TEvent>(TEvent)

Removes `gameEvent` from the channel matching `TEvent`.

```
public void Unsubscribe<TEvent>(TEvent gameEvent) where TEvent : ScriptableObject
```

Parameters

`gameEvent` TEvent

Type Parameters

`TEvent`

Exceptions

ArgumentException

Thrown when no channel is registered for `TEvent`.

Enum EntityHealth.HpBehaviourOnMaxHpDecrease

Namespace: ElectricDrill.AstraHealth.Core

Assembly: com.electricdrill.astra-health.Runtime.dll

```
public enum EntityHealth.HpBehaviourOnMaxHpDecrease
```

Fields

`RemoveHealthAnyway = 3`

`RemoveHealthUpTo1 = 2`

`RemoveHealthUpToMaxHp = 0`

`RemoveHealthUpToMaxHpAndConvertRemovedToBarrier = 1`

Enum EntityHealth.HpBehaviourOnMaxHpIncrease

Namespace: ElectricDrill.AstraHealth.Core

Assembly: com.electricdrill.astra-health.Runtime.dll

```
public enum EntityHealth.HpBehaviourOnMaxHpIncrease
```

Fields

`AddHealthAndConvertExcessToBarrier = 2`

`AddHealthUpToMaxHp = 1`

`None = 0`

Class EntityPassiveHpRegeneration

Namespace: ElectricDrill.AstraHealth.Core

Assembly: com.electricdrill.astra-health.Runtime.dll

Adds passive health regeneration to an entity over time. Attach this component to any entity that should automatically regenerate health. The regeneration amount per tick is driven by the PassiveHealthRegenerationStat and the tick interval by PassiveHealthRegenerationInterval. Entities that do not require passive regeneration should not carry this component.

```
[RequireComponent(typeof(EntityHealth))]
public class EntityPassiveHpRegeneration : MonoBehaviour
```

Inheritance

object ← Object ← Component ← Behaviour ← MonoBehaviour ← EntityPassiveHpRegeneration

Interface IReactable

Namespace: ElectricDrill.AstraHealth.Core

Assembly: com.electricdrill.astra-health.Runtime.dll

Marks a context as carrying a reactability flag that signals whether the receiver of the associated event should act on it.

First-order events (direct attacks, skills, abilities) set IsReactable to `true`. Secondary events produced by counter-damage actions, passive modifiers, or other reactive effects set it to `false` to prevent further downstream reactions and break potential infinite chains.

```
public interface IReactable
```

Properties

IsReactable

When `true`, the event is a first-order event and reactive systems (such as CounterDamageGameActionSO<TContext>) may respond to it. When `false`, reactive systems should skip execution to avoid chain reactions.

```
bool IsReactable { get; }
```

Property Value

bool

Namespace ElectricDrill.AstraHealth.Damage

Classes

DamageAmountContext

Holds numeric details related to an attempt to apply damage, allowing intermediate values to be recorded for each phase of the calculation pipeline (e.g. reductions, resistances, absorptions).

DamagePreventionReasonExtensions

Extension helpers for DamagePreventionReason.

DamageResolutionContext

The immutable result returned by damage application attempts. Encapsulates whether the damage was applied or prevented, the reasons for prevention, and the concrete damage information that was ultimately applied (if any).

DamageSourceSO

Defines a damage source asset that identifies how damage was produced (for example: a spell, trap, environmental hazard, system event, etc.). Create instances via Assets -> Create -> Astra Health / Damage Source.

DamageTypeSO

Scriptable asset that describes a damage category (for example: Physical, Fire, Poison). A DamageType declares which stat reduces this damage, which mitigation/penetration functions to use, whether a stat pierces its defensive stats and whether it ignores barriers. Create instances via Assets -> Create -> Astra Health / DamageType.

PreDamageContext

Mutable pre-damage context raised via the pre-damage game event before the damage pipeline runs. Listeners may mutate Amount, Type, IsCritical, CriticalMultiplier and Ignore to transform the incoming damage — e.g. convert magical damage above a threshold into physical, apply a flat reduction, or negate it entirely by setting Ignore. The pipeline reads these fields after the event is dispatched. Use Builder to construct instances via the fluent step builder.

PreDamageContext.PreDamageInfoStepBuilder

Concrete step builder implementing the fluent interfaces. After all required fields are set, optional fields (dealer, critical, multiplier, instigator, ignore) can be configured before calling Build().

PreDamageContextConfig

Serializable configuration that fully describes a damage application and can build a PreDamage Context at runtime. Designed to be embedded in ScriptableObjects or MonoBehaviours.

Structs

DamageAmountContext.StepAmountRecord

Immutable record representing the damage value before and after a single pipeline phase (identified by the phase type).

Interfaces

IDamageable

Implemented by entities that can receive damage. The TakeDamage(PreDamageContext) method accepts a PreDamageContext record describing a pending damage attempt and returns a DamageResolutionContext describing whether the damage was applied or prevented and additional details.

IHasAmount

Represents a context that carries an associated numeric amount. The value is nullable to represent cases where no amount was resolved — for damage contexts, `null` means the damage was prevented or not applied.

PreDamageContext.DamageInfoAmount

Fluent builder step: specify the damage amount.

PreDamageContext.DamageInfoSource

Fluent builder step: specify the damage source category.

PreDamageContext.DamageInfoTarget

Fluent builder step: specify the target entity.

PreDamageContext.DamageInfoType

Fluent builder step: specify the damage type.

Enums

ContextDealerRole

Selects the entity that will act as the damage dealer, including the option of no dealer (system damage).

ContextEntityRole

Selects one of the two entities involved in a damage context.

DamageAmountMode

Determines how the damage amount is calculated.

DamageOutcome

The overall result of attempting to apply damage to a target. Use this in DamageResolutionContext to indicate whether any damage was applied or if the attempt was prevented.

DamagePreventionReason

Flags that explain why a damage attempt was prevented. Multiple reasons may be combined. These flags are used in Reasons when an attempt is prevented.

Enum ContextDealerRole

Namespace: ElectricDrill.AstraHealth.Damage

Assembly: com.electricdrill.astra-health.Runtime.dll

Selects the entity that will act as the damage dealer, including the option of no dealer (system damage).

```
public enum ContextDealerRole
```

Fields

`ContextSource = 0`

The entity that originally dealt the damage (context Source).

`ContextTarget = 1`

The entity that originally received the damage (context Target).

`None = 2`

No dealer entity — system-initiated damage with no attribution.

Enum ContextEntityRole

Namespace: ElectricDrill.AstraHealth.Damage

Assembly: com.electricdrill.astra-health.Runtime.dll

Selects one of the two entities involved in a damage context.

```
public enum ContextEntityRole
```

Fields

`ContextSource = 0`

The entity that originally dealt the damage (context Source).

`ContextTarget = 1`

The entity that originally received the damage (context Target).

Class DamageAmountContext

Namespace: ElectricDrill.AstraHealth.Damage

Assembly: com.electricdrill.astra-health.Runtime.dll

Holds numeric details related to an attempt to apply damage, allowing intermediate values to be recorded for each phase of the calculation pipeline (e.g. reductions, resistances, absorptions).

```
public class DamageAmountContext
```

Inheritance

object ← DamageAmountContext

Remarks

This class is mutable for Current and keeps a list of DamageAmountContext.StepAmountRecord entries that describe the value before and after each step. It is used to trace how the initial raw value is transformed into the final damage applied.

Constructors

DamageAmountContext(long)

Creates a new instance of DamageAmountContext with the provided initial value.

```
public DamageAmountContext(long initialAmount)
```

Parameters

`initialAmount` long

Raw starting damage value.

Properties

Current

The current damage value during calculation; updated by the various phases. The value is clamped to a minimum of 0. Setting a negative value is a pipeline misconfiguration: a warning is emitted in development builds (`UNITY_ASSERTIONS`) to aid debugging.

```
public long Current { get; set; }
```

Property Value

long

InitialAmount

The raw initial value provided to the damage calculation pipeline.

```
public long InitialAmount { get; }
```

Property Value

long

Records

Read-only list of the recorded phase entries in order. Empty when the step trace is disabled (StepTrace Enabled).

```
public IReadOnlyList<DamageAmountContext.StepAmountRecord> Records { get; }
```

Property Value

IReadOnlyList<DamageAmountContext.StepAmountRecord>

StepTraceEnabled

Whether the damage calculation pipeline appends per-step records to Records for this instance. Set from RecordDamageStepTrace by DamageInfo; defaults to `true` for directly constructed instances (e.g. unit tests).

```
public bool StepTraceEnabled { get; }
```

Property Value

bool

Methods

FromRaw(long)

Convenience factory that builds a DamageAmountContext from a raw value; equivalent to using the public constructor.

```
public static DamageAmountContext FromRaw(long raw)
```

Parameters

`raw` long

Raw damage value.

Returns

DamageAmountContext

A new DamageAmountContext initialized to `raw`.

GetStepAmount(Type)

Retrieves the record associated with a given phase identified by its type.

```
public DamageAmountContext.StepAmountRecord GetStepAmount(Type stepType)
```

Parameters

`stepType` Type

The phase type to look up.

Returns

DamageAmountContext.StepAmountRecord

The corresponding DamageAmountContext.StepAmountRecord or the default if not found.

Remarks

If no matching record exists the default value of DamageAmountContext.StepAmountRecord is returned (with StepType possibly `null`).

Struct DamageAmountContext.StepAmountRecord

Namespace: ElectricDrill.AstraHealth.Damage

Assembly: com.electricdrill.astra-health.Runtime.dll

Immutable record representing the damage value before and after a single pipeline phase (identified by the phase type).

```
public readonly struct DamageAmountContext.StepAmountRecord
```

Constructors

StepAmountRecord(Type, long, long)

Creates a new record for a phase with the associated before/after values.

```
public StepAmountRecord(Type stepType, long pre, long post)
```

Parameters

`stepType` Type

Phase type.

`pre` long

Value before the phase.

`post` long

Value after the phase.

Properties

Post

Damage value immediately after the phase executes.

```
public long Post { get; }
```

Property Value

long

Pre

Damage value immediately before the phase executes.

```
public long Pre { get; }
```

Property Value

long

StepType

The type that represents the phase (for example, a reduction class). May be `null` for uninitialized or default records.

```
public Type StepType { get; }
```

Property Value

Type

Methods

ToString()

Compact textual representation of the record, useful for logging.

```
public override string ToString()
```

Returns

string

String in the format "{TypeName}: {Pre} -> {Post}".

Enum DamageAmountMode

Namespace: ElectricDrill.AstraHealth.Damage

Assembly: com.electricdrill.astra-health.Runtime.dll

Determines how the damage amount is calculated.

```
public enum DamageAmountMode
```

Fields

`DamageFromContext = 2`

A percentage of the damage amount carried by the incoming DamageResolutionContext (e.g. 0.1 = 10% of received damage).

`Fixed = 0`

A fixed value provided via a ElectricDrill.AstraRpgFramework.Utils.LongRef.

`ScalingFormula = 1`

Computed at runtime via a ScalingFormula asset.

Enum DamageOutcome

Namespace: ElectricDrill.AstraHealth.Damage

Assembly: com.electricdrill.astra-health.Runtime.dll

The overall result of attempting to apply damage to a target. Use this in DamageResolutionContext to indicate whether any damage was applied or if the attempt was prevented.

```
public enum DamageOutcome
```

Fields

`Applied = 0`

The damage was applied to the target's health/defenses. See FinalDamageInfo for details.

`Prevented = 1`

The damage attempt was prevented. See Reasons for prevention reasons.

Enum DamagePreventionReason

Namespace: ElectricDrill.AstraHealth.Damage

Assembly: com.electricdrill.astra-health.Runtime.dll

Flags that explain why a damage attempt was prevented. Multiple reasons may be combined. These flags are used in Reasons when an attempt is prevented.

```
[Flags]
public enum DamagePreventionReason
```

Extension Methods
DamagePreventionReasonExtensions.IsTerminal(DamagePreventionReason) , DamagePreventionReasonExtensions.ToDetailedString(DamagePreventionReason)

Fields

`AllDamageImmune = 2`

The target is immune to all damage because of its generic damage mitigation stat value.

`BarrierAbsorbed = 16`

The target's barrier fully absorbed the damage.

`DamageSourceImmune = 8`

The target is immune to the specific DamageSourceSO that originated the damage.

`DamageTypeImmune = 4`

The target is immune to this specific DamageTypeSO.

`DefenseAbsorbed = 32`

The target's stat defense for the damage type fully absorbed the damage.

`EntityDead = 512`

The target was already dead when the damage would have been applied.

`EntityImmune = 1`

The target entity is immune to all damage (global immunity).

`None = 0`

No prevention; damage can proceed.

`PipelineReducedToZero = 256`

After reductions, a pipeline step reduced the damage to zero and nothing was applied.

`PrePhaseIgnored = 64`

The pre-processing phase explicitly ignored the damage (for example by a passive ability that intercepted the PreDmgGameEvent and instructed the PreDamageContext to ignore it).

`PrePhaseZeroAmount = 128`

The damage amount was zero in the pre-phase and therefore not applied.

Class DamagePreventionReasonExtensions

Namespace: ElectricDrill.AstraHealth.Damage

Assembly: com.electricdrill.astra-health.Runtime.dll

Extension helpers for DamagePreventionReason.

```
public static class DamagePreventionReasonExtensions
```

Inheritance

object ← DamagePreventionReasonExtensions

Methods

IsTerminal(DamagePreventionReason)

Returns true when the provided set of reasons indicates a terminal prevention state (i.e. any prevention reason is present). False when None.

```
public static bool IsTerminal(this DamagePreventionReason reasons)
```

Parameters

`reasons` DamagePreventionReason

Returns

bool

ToDetailedString(DamagePreventionReason)

Returns a detailed, ordered description for all active prevention reasons.

```
public static string ToDetailedString(this DamagePreventionReason reasons)
```

Parameters

`reasons` DamagePreventionReason

Returns

string

Class DamageResolutionContext

Namespace: ElectricDrill.AstraHealth.Damage

Assembly: com.electricdrill.astra-health.Runtime.dll

The immutable result returned by damage application attempts. Encapsulates whether the damage was applied or prevented, the reasons for prevention, and the concrete damage information that was ultimately applied (if any).

```
public sealed class DamageResolutionContext : IHasEntity, IHasTarget, IHasPerformer,
IHasAmount, IHasInstigator, IReactable
```

Inheritance

object ← DamageResolutionContext

Implements

IHasEntity, IHasTarget, IHasPerformer, IHasAmount, IHasInstigator, IReactable

Properties

Entity

The entity that received damage. Same as Target in this case.

```
public EntityCore Entity { get; }
```

Property Value

EntityCore

FinalDamageInfo

The final computed damage information that was applied to the target. This is non-null only when Outcome is Applied. The contained DamageInfo includes applied amount, barrier/health split, termination step type and any additional diagnostics produced by the pipeline.

```
public DamageInfo FinalDamageInfo { get; }
```

Property Value

DamageInfo

Instigator

The specific thing that caused this damage — propagated from the originating PreDamageContext. May be `null` if unspecified.

```
public IEffectInstigator Instigator { get; }
```

Property Value

IEffectInstigator

IsReactable

Propagated from IsReactable. When `false`, reactive systems should not respond to this damage resolution event.

```
public bool IsReactable { get; }
```

Property Value

bool

Outcome

The high-level outcome of the attempt: either Applied or Prevented.

```
public DamageOutcome Outcome { get; }
```

Property Value

DamageOutcome

Performer

```
public EntityCore Performer { get; }
```

Property Value

EntityCore

PreDamageContext

The original PreDamageContext that triggered this resolution. Useful for logging and diagnostic correlation.

```
public PreDamageContext PreDamageContext { get; }
```

Property Value

PreDamageContext

Reasons

When Outcome is Prevented, this contains the combined DamagePreventionReason flags explaining why. It will be None when Outcome is Applied.

```
public DamagePreventionReason Reasons { get; }
```

Property Value

DamagePreventionReason

Target

```
public EntityCore Target { get; }
```

Property Value

EntityCore

TerminationStepType

If the pipeline execution terminated early, this will contain the System.Type of the pipeline termination step that decided to stop processing. May be null when the pipeline completed normally.

```
public Type TerminationStepType { get; }
```

Property Value

Type

Methods

Applied(DamageInfo, PreDamageContext)

Create a DamageResolutionContext representing an applied damage attempt. The `info` parameter contains the concrete damage values applied.

```
public static DamageResolutionContext Applied(DamageInfo info, PreDamageContext pre)
```

Parameters

`info` DamageInfo

Final damage info that was applied.

`pre` PreDamageContext

Original pre-damage request.

Returns

DamageResolutionContext

A configured DamageResolutionContext with Outcome == Applied.

Prevented(DamagePreventionReason, PreDamageContext, DamageInfo)

Create a DamageResolutionContext representing a prevented damage attempt. `reasons` should explain why the attempt didn't apply. Optionally the `info` parameter may contain a partial DamageInfo produced before prevention.

```
public static DamageResolutionContext Prevented(DamagePreventionReason reasons,
PreDamageContext pre, DamageInfo info = null)
```

Parameters

`reasons` DamagePreventionReason

Flags explaining the prevention.

`pre` PreDamageContext

Original pre-damage request.

`info` DamageInfo

Optional partial damage info produced before prevention.

Returns

DamageResolutionContext

A configured DamageResolutionContext with Outcome == Prevented.

Class DamageSourceSO

Namespace: ElectricDrill.AstraHealth.Damage

Assembly: com.electricdrill.astra-health.Runtime.dll

Defines a damage source asset that identifies how damage was produced (for example: a spell, trap, environmental hazard, system event, etc.). Create instances via Assets -> Create -> Astra Health / Damage Source.

```
public class DamageSourceSO : ScriptableObject, ITaggable
```

Inheritance

object ← Object ← ScriptableObject ← DamageSourceSO

Implements

ITaggable

Properties

FlatDamageModificationStat

The stat representing the flat damage accumulator for this source. This stat serves as an accumulator for multiple flat modifiers (positive or negative) that affect damage from this source linearly.

```
public StatSO FlatDamageModificationStat { get; }
```

Property Value

StatSO

PercentageDamageModificationStat

The stat representing the percentage damage accumulator for this source. This stat serves as an accumulator for multiple percentage modifiers (positive or negative) that affect damage from this source.

```
public StatSO PercentageDamageModificationStat { get; }
```

Property Value

StatSO

Tags

The tags associated with this object.

```
public GameTagSet Tags { get; }
```

Property Value

GameTagSet

Methods

ToString()

Returns the asset name of this damage source.

```
public override string ToString()
```

Returns

string

Class DamageTypeSO

Namespace: ElectricDrill.AstraHealth.Damage

Assembly: com.electricdrill.astra-health.Runtime.dll

Scriptable asset that describes a damage category (for example: Physical, Fire, Poison). A DamageType declares which stat reduces this damage, which mitigation/penetration functions to use, whether a stat pierces its defensive stats and whether it ignores barriers. Create instances via Assets -> Create -> Astra Health / DamageType.

```
public class DamageTypeSO : ScriptableObject, ITaggable
```

Inheritance

object ← Object ← ScriptableObject ← DamageTypeSO

Implements

ITaggable

Properties

DamageMitigationFn

The mitigation function used to compute how the damage amount is modified by the reducing stat (see DefensiveStat). This may implement flat, percentage, logarithmic, or custom mitigation behaviour.

```
public DamageMitigationFnSO DamageMitigationFn { get; }
```

Property Value

DamageMitigationFnSO

DefensePenetrationFn

The penetration function used to compute how the piercing stat modifies the defensive stat that reduces damage for this DamageTypeSO. This may implement flat, percentage, logarithmic, or custom penetration behaviour.

```
public DefensePenetrationFnSO DefensePenetrationFn { get; }
```

Property Value

DefensePenetrationFnSO

DefensiveStat

The defensive stat that reduces the raw damage amount for this DamageTypeSO. For example "Physical" damage may be reduced by an "Armor" stat.

```
public StatSO DefensiveStat { get; }
```

Property Value

StatSO

DefensiveStatPiercedBy

The stat that pierces the defensive stat for this damage type (for example an "Armor Penetration" stat pierces the "Armor" stat). May be null.

```
public StatSO DefensiveStatPiercedBy { get; }
```

Property Value

StatSO

FlatDamageModificationStat

The stat representing the flat damage accumulator for this type. This stat serves as an accumulator for multiple flat modifiers (positive or negative) that affect damage of this type linearly.

```
public StatSO FlatDamageModificationStat { get; }
```

Property Value

StatSO

IgnoreGenericFlatDamageModifiers

If true, this damage type ignores generic flat damage modifiers. Useful for "true damage" mechanics that bypass flat reductions/increments.

```
public bool IgnoreGenericFlatDamageModifiers { get; protected set; }
```

Property Value

bool

IgnoreGenericPercentageDamageModifiers

If true, this damage type ignores generic percentage damage modifiers. Useful for "true damage" mechanics that bypass resistances/weaknesses.

```
public bool IgnoreGenericPercentageDamageModifiers { get; protected set; }
```

Property Value

bool

IgnoresBarrier

If true, this damage type bypasses barrier mechanic and applies directly to underlying health. Typical use: certain elemental or pure damage types.

```
public bool IgnoresBarrier { get; protected set; }
```

Property Value

bool

Lifesteal

Lifesteal configuration specific to this damage type. When the LifestealStat is set, entities that deal damage of this type will steal a percentage of the damage as health, on top of any generic lifesteal configured in IAstraHealthConfig.

```
public LifestealStatConfig Lifesteal { get; }
```

Property Value

LifestealStatConfig

PercentageDamageModificationStat

The stat representing the percentage damage accumulator for this type. This stat serves as an accumulator for multiple percentage modifiers (positive or negative) that affect damage of this type.

```
public StatSO PercentageDamageModificationStat { get; }
```

Property Value

StatSO

Tags

The tags associated with this object.

```
public GameTagSet Tags { get; }
```

Property Value

GameTagSet

Methods

ToString()

Returns the asset name of this damage type.

```
public override string ToString()
```

Returns

string

Interface IDamageable

Namespace: ElectricDrill.AstraHealth.Damage

Assembly: com.electricdrill.astra-health.Runtime.dll

Implemented by entities that can receive damage. The TakeDamage(PreDamageContext) method accepts a PreDamageContext record describing a pending damage attempt and returns a DamageResolution Context describing whether the damage was applied or prevented and additional details.

```
public interface IDamageable
```

Methods

TakeDamage(PreDamageContext)

Process an incoming damage request and return a resolution object.

Implementations should run the damage processing pipeline (pre-phase, reduction, defensive checks, barriers, health application) or delegate to the centralized pipeline host used in the project. The provided `preDamage` describes the intent (amount, type, source, critical flags, target/dealer).

The returned DamageResolutionContext must reflect the final outcome:

- If damage was applied, an instance created with Applied(DamageInfo, PreDamageContext) is expected and FinalDamageInfo will contain the concrete damage values that were actually applied to the target.
- If the damage was prevented (for example due to immunity, zero amount after reductions, or an explicit ignore), an instance created with Prevented(DamagePreventionReason, PreDamageContext, DamageInfo) is expected.

Implementations SHOULD NOT throw for normal prevention cases; use the prevention reasons and the returned resolution to communicate why the damage didn't apply.

```
DamageResolutionContext TakeDamage(PreDamageContext preDamage)
```

Parameters

`preDamage` PreDamageContext

Immutable description of the pending damage attempt.

Returns

DamageResolutionContext

A DamageResolutionContext describing the final outcome and details.

Interface IHasAmount

Namespace: ElectricDrill.AstraHealth.Damage

Assembly: com.electricdrill.astra-health.Runtime.dll

Represents a context that carries an associated numeric amount. The value is nullable to represent cases where no amount was resolved — for damage contexts, `null` means the damage was prevented or not applied.

```
public interface IHasAmount
```

Properties

Amount

The numeric amount associated with this context, or `null` if not applicable.

```
long? Amount { get; }
```

Property Value

long?

Class PreDamageContext

Namespace: ElectricDrill.AstraHealth.Damage

Assembly: com.electricdrill.astra-health.Runtime.dll

Mutable pre-damage context raised via the pre-damage game event before the damage pipeline runs. Listeners may mutate Amount, Type, IsCritical, CriticalMultiplier and Ignore to transform the incoming damage — e.g. convert magical damage above a threshold into physical, apply a flat reduction, or negate it entirely by setting Ignore. The pipeline reads these fields after the event is dispatched. Use Builder to construct instances via the fluent step builder.

```
public sealed class PreDamageContext : IHasEntity, IHasTarget, IHasPerformer, IHasAmount,
IHasInstigator, IReactable
```

Inheritance

object ← PreDamageContext

Implements

IHasEntity, IHasTarget, IHasPerformer, IHasAmount, IHasInstigator, IReactable

Properties

Amount

The raw damage amount (before modifiers). Listeners may mutate this to transform the incoming damage.

```
public long Amount { get; set; }
```

Property Value

long

Builder

Entry point for the fluent builder. Example:

```
var pre = PreDamageContext.Builder
    .WithAmount(10)
    .WithType(DamageType.Physical)
    .WithSource(DamageSource.Weapon)
    .WithTarget(targetEntity)
    .WithPerformer(dealerEntity)   // optional
    .WithIsCritical(false)
    .Build();
```

```
public static PreDamageContext.DamageInfoAmount Builder { get; }
```

Property Value

PreDamageContext.DamageInfoAmount

CriticalMultiplier

Multiplier applied when this is critical damage. A value of 1.0 means no change. Values <= 0 are normalized to 1.0 by the constructor. Listeners may mutate this.

```
public double CriticalMultiplier { get; set; }
```

Property Value

double

DamageSource

The source or reason for the damage (for example an ability or environmental effect).

```
public DamageSourceSO DamageSource { get; }
```

Property Value

DamageSourceSO

Entity

The entity that will receive the damage. Same as Target in this case.

```
public EntityCore Entity { get; }
```

Property Value

EntityCore

Ignore

If true, the damage will be ignored and not applied to the target. Use this in cases where the damage should be negated (e.g., passive abilities). Listeners may mutate this.

```
public bool Ignore { get; set; }
```

Property Value

bool

Instigator

The specific thing that caused this damage — for example an ability definition, a modifier instance, or a passive. Use pattern matching to identify the concrete type. May be `null` if no specific instigator is relevant beyond Performer and DamageSource.

```
public IEffectInstigator Instigator { get; }
```

Property Value

IEffectInstigator

IsCritical

Whether this damage instance was flagged as a critical hit. Listeners may mutate this.

```
public bool IsCritical { get; set; }
```

Property Value

bool

IsReactable

When `true`, reactive systems (such as counter-damage actions) may respond to this event. When `false`, reactive systems should skip execution to prevent chain reactions. Defaults to `true`. Set to `false` when building contexts for secondary effects such as counter-damage or passive damage modifiers that react to other damage events.

```
public bool IsReactable { get; }
```

Property Value

bool

Performer

The entity that deals the damage (may be null for environmental or system-initiated damage).

```
public EntityCore Performer { get; }
```

Property Value

EntityCore

Target

The entity that will receive the damage.

```
public EntityCore Target { get; }
```

Property Value

EntityCore

Type

The type/category of the damage (e.g. physical, magical). Listeners may mutate this to convert the damage into a different type before the pipeline runs.

```
public DamageTypeSO Type { get; set; }
```

Property Value

DamageTypeSO

Methods

Create(long, DamageTypeSO, DamageSourceSO, EntityCore, EntityCore, bool, double, IEffectInstigator, bool, bool)

Allocation-free alternative to Builder: constructs a PreDamageContext directly, without the intermediate step-builder object. The fluent Builder stays the ergonomic default; prefer this overload on per-frame / high-frequency damage paths. Critical multiplier <= 0 is normalized to 1.0.

```
public static PreDamageContext Create(long amount, DamageTypeSO type, DamageSourceSO
damageSource, EntityCore target, EntityCore performer = null, bool isCritical = false,
double criticalMultiplier = 1, IEffectInstigator instigator = null, bool isReactable = true,
bool ignore = false)
```

Parameters

`amount` long

Raw damage amount, before modifiers.

`type` DamageTypeSO

Damage type/category.

`damageSource` DamageSourceSO

Damage source category.

`target` EntityCore

Entity that will receive the damage.

`performer` EntityCore

Entity dealing the damage. Pass `null` for environmental/system damage.

`isCritical` bool

Whether the hit is flagged as critical.

`criticalMultiplier` double

Multiplier applied when critical (1.0 means no change).

`instigator` IEffectInstigator

Specific cause of the damage (ability, modifier, passive). May be `null`.

`isReactable` bool

When `false`, reactive systems skip this event (use for secondary effects).

`ignore` bool

Initial value of Ignore. Listeners may still mutate it after dispatch.

Returns

PreDamageContext

Interface PreDamageContext.DamageInfoAmount

Namespace: ElectricDrill.AstraHealth.Damage

Assembly: com.electricdrill.astra-health.Runtime.dll

Fluent builder step: specify the damage amount.

```
public interface PreDamageContext.DamageInfoAmount
```

Methods

WithAmount(long)

Set the damage amount and advance to the next step.

```
PreDamageContext.DamageInfoType WithAmount(long amount)
```

Parameters

`amount` long

Returns

PreDamageContext.DamageInfoType

Interface PreDamageContext.DamageInfoSource

Namespace: ElectricDrill.AstraHealth.Damage

Assembly: com.electricdrill.astra-health.Runtime.dll

Fluent builder step: specify the damage source category.

```
public interface PreDamageContext.DamageInfoSource
```

Methods

WithSource(DamageSourceSO)

Set the damage source and advance to the next step.

```
PreDamageContext.DamageInfoTarget WithSource(DamageSourceSO damageSource)
```

Parameters

`damageSource` DamageSourceSO

Returns

PreDamageContext.DamageInfoTarget

Interface PreDamageContext.DamageInfoTarget

Namespace: ElectricDrill.AstraHealth.Damage

Assembly: com.electricdrill.astra-health.Runtime.dll

Fluent builder step: specify the target entity.

```
public interface PreDamageContext.DamageInfoTarget
```

Methods

WithTarget(EntityCore)

Set the damage target and return the concrete builder. The dealer (WithPerformer(EntityCore)) is optional and can be omitted when no specific entity performed the damage (e.g. environmental or system-initiated).

```
PreDamageContext.PreDamageInfoStepBuilder WithTarget(EntityCore target)
```

Parameters

`target` EntityCore

Returns

PreDamageContext.PreDamageInfoStepBuilder

Interface PreDamageContext.DamageInfoType

Namespace: ElectricDrill.AstraHealth.Damage

Assembly: com.electricdrill.astra-health.Runtime.dll

Fluent builder step: specify the damage type.

```
public interface PreDamageContext.DamageInfoType
```

Methods

WithType(DamageTypeSO)

Set the damage type and advance to the next step.

```
PreDamageContext.DamageInfoSource WithType(DamageTypeSO type)
```

Parameters

`type` DamageTypeSO

Returns

PreDamageContext.DamageInfoSource

Class PreDamageContext.PreDamageInfoStepBuilder

Namespace: ElectricDrill.AstraHealth.Damage

Assembly: com.electricdrill.astra-health.Runtime.dll

Concrete step builder implementing the fluent interfaces. After all required fields are set, optional fields (dealer, critical, multiplier, instigator, ignore) can be configured before calling Build().

```
public sealed class PreDamageContext.PreDamageInfoStepBuilder :
PreDamageContext.DamageInfoAmount, PreDamageContext.DamageInfoType,
PreDamageContext.DamageInfoSource, PreDamageContext.DamageInfoTarget
```

Inheritance

object ← PreDamageContext.PreDamageInfoStepBuilder

Implements

PreDamageContext.DamageInfoAmount, PreDamageContext.DamageInfoType, PreDamageContext.DamageInfoSource, PreDamageContext.DamageInfoTarget

Methods

Build()

Build the PreDamageContext instance with the previously configured values.

```
public PreDamageContext Build()
```

Returns

PreDamageContext

WithAmount(long)

Set the damage amount.

```
public PreDamageContext.DamageInfoType WithAmount(long amount)
```

Parameters

`amount` long

Returns

PreDamageContext.DamageInfoType

WithCriticalMultiplier(double)

Set the critical damage multiplier. Values <= 0 will be normalized to 1.0 by the constructor.

```
public PreDamageContext.PreDamageInfoStepBuilder WithCriticalMultiplier(double multiplier)
```

Parameters

`multiplier` double

Returns

PreDamageContext.PreDamageInfoStepBuilder

WithIgnore(bool)

Set the initial Ignore flag for the built context. Defaults to `false`. Listeners may still mutate the flag after dispatch.

```
public PreDamageContext.PreDamageInfoStepBuilder WithIgnore(bool ignore)
```

Parameters

`ignore` bool

Returns

PreDamageContext.PreDamageInfoStepBuilder

WithInstigator(IEffectInstigator)

Set the specific cause of this damage — for example an ability definition, a modifier definition or instance, or a passive. Use this to allow downstream listeners to identify the exact origin beyond entity (WithPerformer(EntityCore)) and category (WithSource(DamageSourceSO)).

```
public PreDamageContext.PreDamageInfoStepBuilder WithInstigator(IEffectInstigator
instigator)
```

Parameters

`instigator` IEffectInstigator

Returns

PreDamageContext.PreDamageInfoStepBuilder

WithIsCritical(bool)

Mark this damage as critical or not.

```
public PreDamageContext.PreDamageInfoStepBuilder WithIsCritical(bool isCritical)
```

Parameters

`isCritical` bool

Returns

PreDamageContext.PreDamageInfoStepBuilder

WithIsReactable(bool)

Set the reactability flag for the built context. Pass `false` when building a context for a secondary effect (counter-damage, passive modifier) so that reactive systems do not respond to it and infinite chains are

prevented. Defaults to `true`.

```
public PreDamageContext.PreDamageInfoStepBuilder WithIsReactable(bool isReactable)
```

Parameters

`isReactable` bool

Returns

PreDamageContext.PreDamageInfoStepBuilder

WithPerformer(EntityCore)

Set the performer entity (the entity causing the damage). Optional — pass `null` or omit entirely when no specific entity performed the damage (e.g. environmental, traps, system-initiated).

```
public PreDamageContext.PreDamageInfoStepBuilder WithPerformer(EntityCore performer)
```

Parameters

`performer` EntityCore

Returns

PreDamageContext.PreDamageInfoStepBuilder

WithSource(DamageSourceSO)

Set the damage source category.

```
public PreDamageContext.DamageInfoTarget WithSource(DamageSourceSO damageSource)
```

Parameters

`damageSource` DamageSourceSO

Returns

PreDamageContext.DamageInfoTarget

WithTarget(EntityCore)

Set the target entity that will receive the damage.

```
public PreDamageContext.PreDamageInfoStepBuilder WithTarget(EntityCore target)
```

Parameters

`target` EntityCore

Returns

PreDamageContext.PreDamageInfoStepBuilder

WithType(DamageTypeSO)

Set the damage type.

```
public PreDamageContext.DamageInfoSource WithType(DamageTypeSO type)
```

Parameters

`type` DamageTypeSO

Returns

PreDamageContext.DamageInfoSource

Class PreDamageContextConfig

Namespace: ElectricDrill.AstraHealth.Damage

Assembly: com.electricdrill.astra-health.Runtime.dll

Serializable configuration that fully describes a damage application and can build a PreDamageContext at runtime. Designed to be embedded in ScriptableObjects or MonoBehaviours.

```
[Serializable]
public class PreDamageContextConfig
```

Inheritance

object ← PreDamageContextConfig

Properties

DamageDealer

Which entity role from the incoming context deals the damage.

```
public ContextDealerRole DamageDealer { get; }
```

Property Value

ContextDealerRole

DamageTarget

Which entity role from the incoming context receives the damage.

```
public ContextEntityRole DamageTarget { get; }
```

Property Value

ContextEntityRole

Methods

Build(EntityCore, EntityCore, long?, IEffectInstigator, bool)

Builds a PreDamageContext from pre-resolved dealer and target entities. The caller is responsible for resolving the correct ElectricDrill.AstraRpgFramework.EntityCore instances from the originating event context before invoking this method.

```
public PreDamageContext Build(EntityCore dealer, EntityCore target, long?
contextDamageAmount = null, IEffectInstigator instigator = null, bool isReactable = true)
```

Parameters

`dealer` EntityCore

The entity dealing the damage. May be `null` for system-initiated damage. If `null` and ElectricDrill.AstraHealth.Damage.PreDamageContextConfig._amountMode is ScalingFormula, the formula falls back to `CalculateValue(target, target)` with a warning. If `null` and ElectricDrill.AstraHealth.Damage.PreDamageContextConfig._useCritStatForMultiplier is true, falls back to the ElectricDrill.AstraRpgFramework.Utils.Percentage multiplier.

`target` EntityCore

The entity receiving the damage. Must not be `null`.

`contextDamageAmount` long?

The final damage amount from the originating context. Required when ElectricDrill.AstraHealth.Damage.PreDamageContextConfig._amountMode is DamageFromContext; ignored otherwise.

`instigator` IEffectInstigator

Optional: the specific cause of this damage. Pass the originating modifier, ability, or game action so downstream listeners can identify and filter it (e.g., to prevent infinite counter-damage loops).

`isReactable` bool

Whether the resulting PreDamageContext is a first-order event that reactive systems (such as counter-damage actions) may respond to. Pass `false` when building a secondary effect (e.g. counter-damage, passive damage modifier) to prevent downstream chain reactions. Defaults to `true`.

Returns

PreDamageContext

A fully constructed PreDamageContext, or `null` if `target` is `null`.

Namespace ElectricDrill.AstraHealth.Damage.Bounds

Classes

DamageCap

MonoBehaviour that provides an upper bound on the damage this entity can receive per hit. Attach it alongside ElectricDrill.AstraHealth.Damage.Bounds.DamageCap.EntityHealth on any entity that needs a damage cap.

Configure via the Inspector by choosing a DamageBoundSource:

- FixedValue — a constant `long` value.
- HealthScaling — inline health metric + percentage.
- ScalingComponent — any ElectricDrill.AstraRpgFramework.Scaling.ScalingComponentSO asset.

DamageFloor

MonoBehaviour that provides a lower bound (minimum damage) this entity must receive per hit. Attach it alongside ElectricDrill.AstraHealth.Damage.Bounds.DamageFloor.EntityHealth on any entity that needs a guaranteed minimum damage.

Configure via the Inspector by choosing a DamageBoundSource:

- FixedValue — a constant `long` value.
- HealthScaling — inline health metric + percentage.
- ScalingComponent — any ElectricDrill.AstraRpgFramework.Scaling.ScalingComponentSO asset.

Interfaces

IDamageCap

Contract for components that provide an upper bound (cap) on the damage an entity can receive from a single hit.

Attach a component implementing this interface to an entity so that ApplyDamageCapStep can query it during the damage calculation pipeline.

IDamageFloor

Contract for components that provide a lower bound (floor) on the damage an entity receives from a single hit.

Attach a component implementing this interface to an entity so that ApplyDamageFloorStep can query it during the damage calculation pipeline.

Enums

DamageBoundSource

Determines how a damage bound value (cap or floor) is resolved at runtime.

Enum DamageBoundSource

Namespace: ElectricDrill.AstraHealth.Damage.Bounds

Assembly: com.electricdrill.astra-health.Runtime.dll

Determines how a damage bound value (cap or floor) is resolved at runtime.

```
public enum DamageBoundSource
```

Fields

`FixedValue = 0`

Use a fixed `long` value configured directly on the component.

`HealthScaling = 1`

Compute the bound from the entity's health using an inline health metric and percentage (e.g. 10 % of max HP). This mirrors HealthScalingComponentSO without requiring a separate ScriptableObject asset.

`ScalingComponent = 2`

Delegate calculation to an external ElectricDrill.AstraRpgFramework.Scaling.ScalingComponentSO ScriptableObject (e.g. a HealthScalingComponentSO or any user-defined scaling component).

Class DamageCap

Namespace: ElectricDrill.AstraHealth.Damage.Bounds

Assembly: com.electricdrill.astra-health.Runtime.dll

MonoBehaviour that provides an upper bound on the damage this entity can receive per hit. Attach it alongside ElectricDrill.AstraHealth.Damage.Bounds.DamageCap.EntityHealth on any entity that needs a damage cap.

Configure via the Inspector by choosing a DamageBoundSource:

- FixedValue — a constant `long` value.
- HealthScaling — inline health metric + percentage.
- ScalingComponent — any ElectricDrill.AstraRpgFramework.Scaling.ScalingComponentSO asset.

```
[RequireComponent(typeof(EntityCore))]
public class DamageCap : MonoBehaviour, IDamageCap
```

Inheritance

object ← Object ← Component ← Behaviour ← MonoBehaviour ← DamageCap

Implements

IDamageCap

Methods

GetDamageCap()

Returns the maximum damage value allowed for a single hit on this entity. The pipeline step will clamp the current damage to at most this value.

```
public long GetDamageCap()
```

Returns

long

A positive `long` representing the damage cap.

Class DamageFloor

Namespace: ElectricDrill.AstraHealth.Damage.Bounds

Assembly: com.electricdrill.astra-health.Runtime.dll

MonoBehaviour that provides a lower bound (minimum damage) this entity must receive per hit. Attach it alongside ElectricDrill.AstraHealth.Damage.Bounds.DamageFloor.EntityHealth on any entity that needs a guaranteed minimum damage.

Configure via the Inspector by choosing a DamageBoundSource:

- FixedValue — a constant `long` value.
- HealthScaling — inline health metric + percentage.
- ScalingComponent — any ElectricDrill.AstraRpgFramework.Scaling.ScalingComponentSO asset.

```
[RequireComponent(typeof(EntityCore))]
public class DamageFloor : MonoBehaviour, IDamageFloor
```

Inheritance

object ← Object ← Component ← Behaviour ← MonoBehaviour ← DamageFloor

Implements

IDamageFloor

Methods

GetDamageFloor()

Returns the minimum damage value guaranteed for a single hit on this entity. The pipeline step will raise the current damage to at least this value.

```
public long GetDamageFloor()
```

Returns

long

A positive `long` representing the damage floor.

Interface IDamageCap

Namespace: ElectricDrill.AstraHealth.Damage.Bounds

Assembly: com.electricdrill.astra-health.Runtime.dll

Contract for components that provide an upper bound (cap) on the damage an entity can receive from a single hit.

Attach a component implementing this interface to an entity so that ApplyDamageCapStep can query it during the damage calculation pipeline.

```
public interface IDamageCap
```

Methods

GetDamageCap()

Returns the maximum damage value allowed for a single hit on this entity. The pipeline step will clamp the current damage to at most this value.

```
long GetDamageCap()
```

Returns

long

A positive `long` representing the damage cap.

Interface IDamageFloor

Namespace: ElectricDrill.AstraHealth.Damage.Bounds

Assembly: com.electricdrill.astra-health.Runtime.dll

Contract for components that provide a lower bound (floor) on the damage an entity receives from a single hit.

Attach a component implementing this interface to an entity so that ApplyDamageFloorStep can query it during the damage calculation pipeline.

```
public interface IDamageFloor
```

Methods

GetDamageFloor()

Returns the minimum damage value guaranteed for a single hit on this entity. The pipeline step will raise the current damage to at least this value.

```
long GetDamageFloor()
```

Returns

long

A positive `long` representing the damage floor.

Namespace ElectricDrill.AstraHealth.Damage.CalculationPipeline

Classes

ApplyBarrierStep
: Damage step that consumes target barrier (temporary shields). It doesn't remove health.

ApplyCriticalMultiplierStep
: Damage step that applies critical hit multipliers when the current damage is flagged as critical.

ApplyDamageCapStep
: Damage step that enforces an upper bound (cap) on the current damage amount.

  At runtime the step looks for a component implementing IDamageCap on the Target. If found and the current damage exceeds the cap, the amount is clamped down. If no IDamageCap component is present, the step is a no-op.

ApplyDamageFloorStep
: Damage step that enforces a lower bound (floor) on the current damage amount.

  At runtime the step looks for a component implementing IDamageFloor on the Target. If found and the current damage is below the floor, the amount is raised. If no IDamageFloor component is present, the step is a no-op.

  Note: The base Process(DamageInfo) method skips steps when `Current <= 0` or when the damage has already been prevented. This means the floor cannot revive damage that was fully absorbed by a previous step. To guarantee a minimum damage amount, place this step *before* steps that might reduce the value to zero.

ApplyDefenseStep
: Damage step that applies defensive calculations based on the damage type's defensive and piercing stats and configured mitigation and penetration functions.

ApplyFlatDmgModifiersStep
: Damage step that applies flat damage modifiers (both positive increments and negative reductions) based on configured generic flat damage modifiers & source/type flat damage modifiers. This step should typically be applied AFTER percentage-based modifications.

ApplyPercentageDmgModifiersStep
: Damage step that applies generic and configured damage source/type modifiers (for example weaknesses and resistances) to the current damage amount.

DamageCalculationStrategySO

ScriptableObject that defines a configurable damage calculation pipeline. The pipeline is composed of ordered DamageStep instances executed sequentially.

Optionally, a **pre-strategy** and/or **post-strategy** can be configured to run another DamageCalculationStrategySO before or after this strategy's own steps. This enables composition without duplicating the default pipeline: a custom strategy can delegate to the global default pipeline (or any other strategy) and then apply additional steps on top.

DamageInfo

Container for damage pipeline state while a damage calculation is executed. Holds the amounts being transformed, the source/type metadata and accumulated prevention reasons.

DamageMitigationCalculator

Pure, stateless helpers that compute damage after defensive and damage reduction functions are applied. Callers are responsible for validating their DamageTypeSO configuration (for example, the consistency warnings in ApplyDefenseStep).

DamageStep

Base class for a single step in the damage calculation pipeline. A DamageStep performs a focused transformation on a DamageInfo instance (for example applying defenses, barriers or modifiers).

Class ApplyBarrierStep

Namespace: ElectricDrill.AstraHealth.Damage.CalculationPipeline

Assembly: com.electricdrill.astra-health.Runtime.dll

Damage step that consumes target barrier (temporary shields). It doesn't remove health.

```
[Serializable]
public class ApplyBarrierStep : DamageStep
```

Inheritance

object ← DamageStep ← ApplyBarrierStep

Inherited Members

DamageStep.Process(DamageInfo)

Properties

DisplayName

Human readable name shown in pipeline editors.

```
public override string DisplayName { get; }
```

Property Value

string

Methods

ProcessStep(DamageInfo)

Applies the target's barrier to reduce the current damage amount. If the damage type ignores barrier, or the target has no barrier, the method returns the input unchanged. When barrier is consumed the target's barrier value is reduced accordingly. If the barrier completely absorbs the damage, Pipeline ReducedToZero is added.

```
public override DamageInfo ProcessStep(DamageInfo data)
```

Parameters

`data` DamageInfo

Current damage pipeline data.

Returns

DamageInfo

The same `data` instance updated with reduced amounts and any barrier consumption.

Class ApplyCriticalMultiplierStep

Namespace: ElectricDrill.AstraHealth.Damage.CalculationPipeline

Assembly: com.electricdrill.astra-health.Runtime.dll

Damage step that applies critical hit multipliers when the current damage is flagged as critical.

```
[Serializable]
public class ApplyCriticalMultiplierStep : DamageStep
```

Inheritance

object ← DamageStep ← ApplyCriticalMultiplierStep

Inherited Members

DamageStep.Process(DamageInfo)

Properties

DisplayName

Human readable name shown in pipeline editors.

```
public override string DisplayName { get; }
```

Property Value

string

Methods

ProcessStep(DamageInfo)

Multiplies the current damage by the critical multiplier from DamageInfo when the hit is marked critical. If the multiplier is 1 or invalid, no change occurs.

```
public override DamageInfo ProcessStep(DamageInfo data)
```

Parameters

`data` DamageInfo

Current damage pipeline data.

Returns

DamageInfo

The same `data` instance updated with the multiplied amount when applicable.

Class ApplyDamageCapStep

Namespace: ElectricDrill.AstraHealth.Damage.CalculationPipeline

Assembly: com.electricdrill.astra-health.Runtime.dll

Damage step that enforces an upper bound (cap) on the current damage amount.

At runtime the step looks for a component implementing IDamageCap on the Target. If found and the current damage exceeds the cap, the amount is clamped down. If no IDamageCap component is present, the step is a no-op.

```
[Serializable]
public class ApplyDamageCapStep : DamageStep
```

Inheritance

object ← DamageStep ← ApplyDamageCapStep

Inherited Members

DamageStep.Process(DamageInfo)

Properties

DisplayName

Human-readable name shown in editors and logs for this step. Implementations should return a short descriptive label.

```
public override string DisplayName { get; }
```

Property Value

string

Methods

ProcessStep(DamageInfo)

Clamps Current to the value returned by GetDamageCap() when a provider is found on the target. A cap value <= 0 is treated as misconfiguration and ignored.

```
public override DamageInfo ProcessStep(DamageInfo data)
```

Parameters

`data` DamageInfo

Current damage pipeline data.

Returns

DamageInfo

The same `data` instance, potentially with a reduced current amount.

Class ApplyDamageFloorStep

Namespace: ElectricDrill.AstraHealth.Damage.CalculationPipeline

Assembly: com.electricdrill.astra-health.Runtime.dll

Damage step that enforces a lower bound (floor) on the current damage amount.

At runtime the step looks for a component implementing IDamageFloor on the Target. If found and the current damage is below the floor, the amount is raised. If no IDamageFloor component is present, the step is a no-op.

Note: The base Process(DamageInfo) method skips steps when `Current <= 0` or when the damage has already been prevented. This means the floor cannot revive damage that was fully absorbed by a previous step. To guarantee a minimum damage amount, place this step *before* steps that might reduce the value to zero.

```
[Serializable]
public class ApplyDamageFloorStep : DamageStep
```

Inheritance

object ← DamageStep ← ApplyDamageFloorStep

Inherited Members

DamageStep.Process(DamageInfo)

Properties

DisplayName

Human-readable name shown in editors and logs for this step. Implementations should return a short descriptive label.

```
public override string DisplayName { get; }
```

Property Value

string

Methods

ProcessStep(DamageInfo)

Raises Current to the value returned by GetDamageFloor() when a provider is found on the target and the current amount is below the floor. A floor value <= 0 is treated as misconfiguration and ignored.

```
public override DamageInfo ProcessStep(DamageInfo data)
```

Parameters

`data` DamageInfo

Current damage pipeline data.

Returns

DamageInfo

The same `data` instance, potentially with an increased current amount.

Class ApplyDefenseStep

Namespace: ElectricDrill.AstraHealth.Damage.CalculationPipeline

Assembly: com.electricdrill.astra-health.Runtime.dll

Damage step that applies defensive calculations based on the damage type's defensive and piercing stats and configured mitigation and penetration functions.

```
[Serializable]
public class ApplyDefenseStep : DamageStep
```

Inheritance

object ← DamageStep ← ApplyDefenseStep

Inherited Members

DamageStep.Process(DamageInfo)

Properties

DisplayName

Human-readable name shown in pipeline editors.

```
public override string DisplayName { get; }
```

Property Value

string

Methods

ProcessStep(DamageInfo)

Applies defense and damage reduction logic using the damage type's configuration. If the damage type is inconsistently configured the method logs a warning and skips the corresponding reduction stage.

```
public override DamageInfo ProcessStep(DamageInfo data)
```

Parameters

`data` DamageInfo

Current damage pipeline data.

Returns

DamageInfo

The same `data` instance updated with the reduced amount.

Class ApplyFlatDmgModifiersStep

Namespace: ElectricDrill.AstraHealth.Damage.CalculationPipeline

Assembly: com.electricdrill.astra-health.Runtime.dll

Damage step that applies flat damage modifiers (both positive increments and negative reductions) based on configured generic flat damage modifiers & source/type flat damage modifiers. This step should typically be applied AFTER percentage-based modifications.

```
[Serializable]
public class ApplyFlatDmgModifiersStep : DamageStep
```

Inheritance

object ← DamageStep ← ApplyFlatDmgModifiersStep

Inherited Members

DamageStep.Process(DamageInfo)

Properties

DisplayName

Human-readable name shown in pipeline editors.

```
public override string DisplayName { get; }
```

Property Value

string

Methods

ProcessStep(DamageInfo)

Applies configured flat modifiers to Amounts. Flat modifiers are summed additively and applied to the current damage amount. The final damage cannot go below 0 (negative results are clamped to 0).

```
public override DamageInfo ProcessStep(DamageInfo data)
```

Parameters

`data` DamageInfo

Current damage pipeline data.

Returns

DamageInfo

The same `data` instance updated with the modified amounts.

Class ApplyPercentageDmgModifiersStep

Namespace: ElectricDrill.AstraHealth.Damage.CalculationPipeline

Assembly: com.electricdrill.astra-health.Runtime.dll

Damage step that applies generic and configured damage source/type modifiers (for example weaknesses and resistances) to the current damage amount.

```
[Serializable]
public class ApplyPercentageDmgModifiersStep : DamageStep
```

Inheritance

object ← DamageStep ← ApplyPercentageDmgModifiersStep

Inherited Members

DamageStep.Process(DamageInfo)

Properties

DisplayName

Human readable name shown in pipeline editors.

```
public override string DisplayName { get; }
```

Property Value

string

Methods

ProcessStep(DamageInfo)

Applies configured percentage-based modifiers to Amounts. If the configured adjustments lead to immunity or a terminal prevention condition the method will mark the DamageInfo with the appropriate DamagePreventionReason.

```
public override DamageInfo ProcessStep(DamageInfo data)
```

Parameters

`data` DamageInfo

Current damage pipeline data.

Returns

DamageInfo

The same `data` instance updated with the modified amounts and any prevention reasons.

Class DamageCalculationStrategySO

Namespace: ElectricDrill.AstraHealth.Damage.CalculationPipeline

Assembly: com.electricdrill.astra-health.Runtime.dll

ScriptableObject that defines a configurable damage calculation pipeline. The pipeline is composed of ordered DamageStep instances executed sequentially.

Optionally, a **pre-strategy** and/or **post-strategy** can be configured to run another DamageCalculationStrategySO before or after this strategy's own steps. This enables composition without duplicating the default pipeline: a custom strategy can delegate to the global default pipeline (or any other strategy) and then apply additional steps on top.

```
public class DamageCalculationStrategySO : ScriptableObject, ITaggable
```

Inheritance

object ← Object ← ScriptableObject ← DamageCalculationStrategySO

Implements

ITaggable

Fields

steps

Ordered list of steps composing this calculation strategy. Steps are executed in sequence.

```
[SerializeReference]
public List<DamageStep> steps
```

Field Value

List<DamageStep>

Properties

Tags

The tags associated with this object.

```
public GameTagSet Tags { get; }
```

Property Value

GameTagSet

Methods

CalculateDamage(DamageInfo)

Executes the entire damage calculation pipeline defined in this asset, including optional pre-strategy and post-strategy delegation.

```
public virtual DamageInfo CalculateDamage(DamageInfo data)
```

Parameters

`data` DamageInfo

The initial damage information.

Returns

DamageInfo

The damage information after all steps have been applied.

Class DamageInfo

Namespace: ElectricDrill.AstraHealth.Damage.CalculationPipeline

Assembly: com.electricdrill.astra-health.Runtime.dll

Container for damage pipeline state while a damage calculation is executed. Holds the amounts being transformed, the source/type metadata and accumulated prevention reasons.

```
public class DamageInfo : IHasPerformer, IHasTarget, IHasInstigator
```

Inheritance

object ← DamageInfo

Implements

IHasPerformer, IHasTarget, IHasInstigator

Remarks

Built-in pipeline services (TargetBarrier, TargetStats, PerformerStats) are pre-resolved once in the constructor so pipeline steps avoid repeated `GetComponent` calls on the hot path.

Extensibility pattern for external package steps: follow the same approach used by `ApplyDamageCapStep` — define your own interface, implement it on a MonoBehaviour, and query it in your step via `data.Target.TryGetComponent<IYourInterface>(out var svc)`. No changes to DamageInfo are required.

Constructors

DamageInfo(PreDamageContext, IAstraHealthConfig)

Creates a new DamageInfo from the provided PreDamageContext. The constructor initializes amounts, metadata and sets pre-phase prevention reasons if applicable.

```
public DamageInfo(PreDamageContext pre, IAstraHealthConfig config = null)
```

Parameters

`pre` PreDamageContext

The immutable pre-damage description used to initialize this instance.

`config` IAstraHealthConfig

The health configuration to embed in this context so pipeline steps avoid calling the static config provider on every hot-path invocation. Pass `null` in unit tests that do not exercise config-dependent steps.

Properties

Amounts

Numeric amounts and intermediate records for the damage being processed.

```
public DamageAmountContext Amounts { get; set; }
```

Property Value

DamageAmountContext

Config

Active health configuration for this damage calculation. Provided by the originating EntityHealth so that pipeline steps never need to call the static config provider.

```
public IAstraHealthConfig Config { get; }
```

Property Value

IAstraHealthConfig

CriticalMultiplier

Critical multiplier applied when IsCritical is true (1.0 means no change).

```
public double CriticalMultiplier { get; }
```

Property Value

double

DamageSource

The origin/source of the damage (spell, environmental, trap, etc.).

```
public DamageSourceSO DamageSource { get; }
```

Property Value

DamageSourceSO

Instigator

The specific thing that caused this damage — propagated from the originating PreDamageContext. May be `null` if unspecified.

```
public IEffectInstigator Instigator { get; }
```

Property Value

IEffectInstigator

IsCritical

Indicates whether the incoming hit was marked as critical.

```
public bool IsCritical { get; }
```

Property Value

bool

IsPrevented

Returns true when accumulated reasons include a terminal prevention condition.

```
public bool IsPrevented { get; }
```

Property Value

bool

Performer

Entity that deals the damage (may be the same as target for self damage or null for environmental damage).

```
public EntityCore Performer { get; }
```

Property Value

EntityCore

PerformerStats

Stat reader for the performer (damage dealer), pre-resolved once in the constructor. `null` for environmental damage (no performer) or when Performer is null.

```
public IStatReader PerformerStats { get; }
```

Property Value

IStatReader

Remarks

Pipeline steps accessing piercing or offensive stats must guard against `null` (environmental damage has no performer).

Under DamageStatsAttribution other than ElectricDrill.AstraRpgFramework.Ownership.EntityAttribution.Direct, this is a fallback chain: the performer's own stats (e.g. a weapon's) stay authoritative, and the resolved owner's stats (e.g. a ship's) are consulted only for stats the performer doesn't define. See ElectricDrill.AstraHealth.Damage.CalculationPipeline.ChainedStatReader.

Reasons

Reasons accumulated through the pipeline that caused damage to be prevented. This is a flag enum that can contain multiple reasons.

```
public DamagePreventionReason Reasons { get; }
```

Property Value

DamagePreventionReason

RoundingSettings

Rounding policy resolved from Config once for this damage calculation.

```
public HealthRoundingSettings RoundingSettings { get; }
```

Property Value

HealthRoundingSettings

Target

Entity that is the target of this damage calculation.

```
public EntityCore Target { get; }
```

Property Value

EntityCore

TargetBarrier

Barrier container on the target, pre-resolved once in the constructor. `null` when the target has no IBarrierContainer component (e.g. non-health entities).

```
public IBarrierContainer TargetBarrier { get; }
```

Property Value

IBarrierContainer

TargetStats

Stat reader for the damage target, pre-resolved once in the constructor via the ElectricDrill.AstraRpgFramework.EntityCore → ElectricDrill.AstraRpgFramework.Stats.IStatReader implicit cast. Returns 0 for any stat not present in the target's stat set. `null` when Target is null.

```
public IStatReader TargetStats { get; }
```

Property Value

IStatReader

TerminationStepType

DamageStep that caused termination of the pipeline.

```
public Type TerminationStepType { get; }
```

Property Value

Type

Type

Configured damage type describing how the damage behaves (defenses, true, etc.).

```
public DamageTypeSO Type { get; }
```

Property Value

DamageTypeSO

Methods

AddReason(DamagePreventionReason, Type, bool)

Adds a prevention reason to the accumulated flags and optionally marks the termination step.

```
public void AddReason(DamagePreventionReason reason, Type stepType, bool terminate = true)
```

Parameters

`reason` DamagePreventionReason

Reason to add.

`stepType` Type

Type of the step that added the reason (can be null for pre-phase).

`terminate` bool

If true and the termination step is not already set, records `stepType` as the termination cause.

Class DamageMitigationCalculator

Namespace: ElectricDrill.AstraHealth.Damage.CalculationPipeline

Assembly: com.electricdrill.astra-health.Runtime.dll

Pure, stateless helpers that compute damage after defensive and damage reduction functions are applied. Callers are responsible for validating their DamageTypeSO configuration (for example, the consistency warnings in ApplyDefenseStep).

```
public static class DamageMitigationCalculator
```

Inheritance

object ← DamageMitigationCalculator

Remarks

Every method has an overload taking a ReductionContext and one without it. The ones without it use Default, so a function whose result depends on levels is evaluated at level 1: pass a context whenever the entities are known.

Methods

ApplyDamageMitigation(long, double, DamageMitigationFnSO, RoundingMode)

Applies `damageMitigationFn` and rounds fractional results with `roundingMode`, evaluating a level-dependent function at level 1.

```
public static long ApplyDamageMitigation(long amount, double reducedDefensiveStatValue,
DamageMitigationFnSO damageMitigationFn, RoundingMode roundingMode)
```

Parameters

`amount` long

The damage amount to reduce.

`reducedDefensiveStatValue` double

The defensive stat value after defense penetration was applied.

`damageMitigationFn` DamageMitigationFnSO

The damage mitigation function. May be `null`, in which case `amount` is returned unchanged.

`roundingMode` RoundingMode

Rounding mode applied by mitigation functions with fractional output.

Returns

long

The reduced damage amount.

ApplyDamageMitigation(long, double, DamageMitigationFnSO, RoundingMode, in ReductionContext)

Applies `damageMitigationFn` and rounds fractional results with `roundingMode`.

```
public static long ApplyDamageMitigation(long amount, double reducedDefensiveStatValue,
DamageMitigationFnSO damageMitigationFn, RoundingMode roundingMode, in
ReductionContext context)
```

Parameters

`amount` long

The damage amount to reduce.

`reducedDefensiveStatValue` double

The defensive stat value after defense penetration was applied.

`damageMitigationFn` DamageMitigationFnSO

The damage mitigation function. May be `null`, in which case `amount` is returned unchanged.

`roundingMode` RoundingMode

Rounding mode applied by mitigation functions with fractional output.

`context` ReductionContext

The entities involved in the hit (their levels and stats), for functions whose result depends on them.

Returns

long

The reduced damage amount.

CalculatePiercedDefense(long, long, DefensePenetrationFnSO, StatSO, in ReductionContext, bool)

Calculates the effective defensive stat value after the attacker's piercing has been applied through `defensePenetrationFn`.

```
public static double CalculatePiercedDefense(long piercingStatValue, long
defensiveStatValue, DefensePenetrationFnSO defensePenetrationFn, StatSO defensiveStat, in
ReductionContext context, bool applyClamp = true)
```

Parameters

`piercingStatValue` long

The attacker's piercing stat value.

`defensiveStatValue` long

The defender's defensive stat value.

`defensePenetrationFn` DefensePenetrationFnSO

The penetration function. May be `null`, in which case `defensiveStatValue` is returned unchanged.

`defensiveStat` StatSO

The defensive ElectricDrill.AstraRpgFramework.Stats.StatSO definition. May be `null` when `defensePenetrationFn` is `null`.

`context` ReductionContext

The entities involved in the hit (their levels and stats), for functions whose result depends on them.

`applyClamp` bool

Whether to apply stat clamping. Defaults to `true`.

Returns

double

The reduced defensive stat value.

CalculatePiercedDefense(long, long, DefensePenetrationFnSO, StatSO, bool)

Calculates the effective defensive stat value after the attacker's piercing has been applied through `defensePenetrationFn`, evaluating a level-dependent function at level 1.

```
public static double CalculatePiercedDefense(long piercingStatValue, long
defensiveStatValue, DefensePenetrationFnSO defensePenetrationFn, StatSO defensiveStat, bool
applyClamp = true)
```

Parameters

`piercingStatValue` long

The attacker's piercing stat value.

`defensiveStatValue` long

The defender's defensive stat value.

`defensePenetrationFn` DefensePenetrationFnSO

The penetration function. May be `null`, in which case `defensiveStatValue` is returned unchanged.

`defensiveStat` StatSO

The defensive ElectricDrill.AstraRpgFramework.Stats.StatSO definition. May be `null` when `defensePenetrationFn` is `null`.

`applyClamp` bool

Whether to apply stat clamping. Defaults to `true`.

Returns

double

The reduced defensive stat value.

CalculateReducedDmg(long, long, DefensePenetrationFnSO, long, StatSO, DamageMitigationFnSO, HealthRoundingSettings, in ReductionContext, bool)

Calculates the reduced damage amount using explicit rounding settings for defense penetration and damage mitigation.

```
public static long CalculateReducedDmg(long amount, long piercingStatValue,
DefensePenetrationFnSO defensePenetrationFn, long defensiveStatValue, StatSO defensiveStat,
DamageMitigationFnSO damageMitigationFn, HealthRoundingSettings roundingSettings, in
ReductionContext context, bool applyClamp = true)
```

Parameters

`amount` long

The base damage amount.

`piercingStatValue` long

The attacker's piercing stat value.

`defensePenetrationFn` DefensePenetrationFnSO

Function used to reduce the defensive stat with the piercing value. May be `null`, in which case the defensive stat is passed through unchanged.

`defensiveStatValue` long

The defender's defensive stat value.

`defensiveStat` StatSO

The defensive ElectricDrill.AstraRpgFramework.Stats.StatSO definition. May be `null` when `defensePenetrationFn` is `null`.

`damageMitigationFn` DamageMitigationFnSO

Function used to reduce the damage based on the (possibly reduced) defensive stat. May be `null`, in which case `amount` is returned unchanged.

`roundingSettings` HealthRoundingSettings

Rounding policy used for fractional penetration and mitigation values.

`context` ReductionContext

The entities involved in the hit (their levels and stats), for functions whose result depends on them.

`applyClamp` bool

Whether to apply stat clamping in defense penetration. Defaults to `true` for gameplay; pass `false` for analysis tools that want to visualize the unclamped formula.

Returns

long

The damage amount after defense and damage reduction have been applied.

CalculateReducedDmg(long, long, DefensePenetrationFnSO, long, StatSO, DamageMitigationFnSO, HealthRoundingSettings, bool)

Calculates the reduced damage amount using explicit rounding settings for defense penetration and damage mitigation, evaluating level-dependent functions at level 1.

```
public static long CalculateReducedDmg(long amount, long piercingStatValue,
DefensePenetrationFnSO defensePenetrationFn, long defensiveStatValue, StatSO defensiveStat,
DamageMitigationFnSO damageMitigationFn, HealthRoundingSettings roundingSettings, bool
applyClamp = true)
```

Parameters

`amount` long

The base damage amount.

`piercingStatValue` long

The attacker's piercing stat value.

`defensePenetrationFn` DefensePenetrationFnSO

Function used to reduce the defensive stat with the piercing value. May be `null`, in which case the defensive stat is passed through unchanged.

`defensiveStatValue` long

The defender's defensive stat value.

`defensiveStat` StatSO

The defensive ElectricDrill.AstraRpgFramework.Stats.StatSO definition. May be `null` when `defensePenetrationFn` is `null`.

`damageMitigationFn` DamageMitigationFnSO

Function used to reduce the damage based on the (possibly reduced) defensive stat. May be `null`, in which case `amount` is returned unchanged.

`roundingSettings` HealthRoundingSettings

Rounding policy used for fractional penetration and mitigation values.

`applyClamp` bool

Whether to apply stat clamping in defense penetration. Defaults to `true` for gameplay; pass `false` for analysis tools that want to visualize the unclamped formula.

Returns

long

The damage amount after defense and damage reduction have been applied.

CalculateReducedDmg(long, long, long, DamageTypeSO, HealthRoundingSettings, in ReductionContext, bool)

Convenience overload that reads reduction functions from `damageType` and uses explicit rounding settings.

```
public static long CalculateReducedDmg(long amount, long piercingStatValue, long
defensiveStatValue, DamageTypeSO damageType, HealthRoundingSettings roundingSettings, in
ReductionContext context, bool applyClamp = true)
```

Parameters

`amount` long

The base damage amount.

`piercingStatValue` long

The attacker's piercing stat value.

`defensiveStatValue` long

The defender's defensive stat value.

`damageType` DamageTypeSO

The damage type asset that supplies the reduction functions and defensive stat.

`roundingSettings` HealthRoundingSettings

Rounding policy used for fractional penetration and mitigation values.

`context` ReductionContext

The entities involved in the hit (their levels and stats), for functions whose result depends on them.

`applyClamp` bool

Whether to apply stat clamping in defense penetration. Defaults to `true` for gameplay.

Returns

long

The damage amount after defense and damage reduction have been applied.

CalculateReducedDmg(long, long, long, DamageTypeSO, HealthRoundingSettings, bool)

Convenience overload that reads reduction functions from `damageType` and uses explicit rounding settings, evaluating level-dependent functions at level 1.

```
public static long CalculateReducedDmg(long amount, long piercingStatValue, long
defensiveStatValue, DamageTypeSO damageType, HealthRoundingSettings roundingSettings, bool
applyClamp = true)
```

Parameters

`amount` long

The base damage amount.

`piercingStatValue` long

The attacker's piercing stat value.

`defensiveStatValue` long

The defender's defensive stat value.

`damageType` DamageTypeSO

The damage type asset that supplies the reduction functions and defensive stat.

`roundingSettings` HealthRoundingSettings

Rounding policy used for fractional penetration and mitigation values.

`applyClamp` bool

Whether to apply stat clamping in defense penetration. Defaults to `true` for gameplay.

Returns

long

The damage amount after defense and damage reduction have been applied.

Class DamageStep

Namespace: ElectricDrill.AstraHealth.Damage.CalculationPipeline

Assembly: com.electricdrill.astra-health.Runtime.dll

Base class for a single step in the damage calculation pipeline. A DamageStep performs a focused transformation on a DamageInfo instance (for example applying defenses, barriers or modifiers).

```
[Serializable]
public abstract class DamageStep
```

Inheritance

object ← DamageStep

Derived

ApplyBarrierStep, ApplyCriticalMultiplierStep, ApplyDamageCapStep, ApplyDamageFloorStep, ApplyDefenseStep, ApplyFlatDmgModifiersStep, ApplyPercentageDmgModifiersStep

Properties

DisplayName

Human-readable name shown in editors and logs for this step. Implementations should return a short descriptive label.

```
public abstract string DisplayName { get; }
```

Property Value

string

Methods

Process(DamageInfo)

Entry point executed by the pipeline runner. This method enforces common preconditions (skipping when already prevented or when the current amount is <= 0), invokes ProcessStep(DamageInfo), records

the before/after amounts for tracing and sets the PipelineReducedToZero reason when a step reduces a positive amount to zero or less.

```
public DamageInfo Process(DamageInfo data)
```

Parameters

`data` DamageInfo

The pipeline state being processed.

Returns

DamageInfo

The same `data` instance, potentially modified by the step.

ProcessStep(DamageInfo)

Implement this method in derived classes to perform the actual transformation of the pipeline state.

```
public abstract DamageInfo ProcessStep(DamageInfo data)
```

Parameters

`data` DamageInfo

Pipeline state to process. Implementations should mutate this instance to reflect changes.

Returns

DamageInfo

The modified `data` instance.

Remarks

Accessing external services from a step: Use pre-resolved properties on `data` for core framework dependencies (`data.TargetStats`, `data.TargetBarrier`, `data.PerformerStats`). For services provided by external packages, use `data.Target.TryGetComponent<IYourInterface>(out var svc)` — the same pattern

used by `ApplyDamageCapStep` and `ApplyDamageFloorStep`. This keeps DamageInfo lean while allowing full extensibility.

Namespace ElectricDrill.AstraHealth.DamageMitigationFunctions

Classes

DamageMitigationFnSO

Base type for damage mitigation functions. Implementations compute how an incoming damage amount is reduced using a defensive statistic value.

FlatDamageMitigationFnSO

HyperbolicDamageMitigationFnSO

Mitigates damage by `X / (X + K)`, where `X` is the defensive stat and `K` the defense that gives 50% mitigation. The formula of LoL, Dota, WoW and Diablo.

LogDamageMitigationFnSO

Legacy logarithmic mitigation. Kept so existing assets keep working; prefer HyperbolicDamageMitigationFnSO for new work. For small defense values it behaves like the hyperbolic function with `K = BaseValue / ScaleFactor`; at high values it grows far more slowly.

PercentageDamageMitigationFnSO

Class DamageMitigationFnSO

Namespace: ElectricDrill.AstraHealth.DamageMitigationFunctions

Assembly: com.electricdrill.astra-health.Runtime.dll

Base type for damage mitigation functions. Implementations compute how an incoming damage amount is reduced using a defensive statistic value.

```
public abstract class DamageMitigationFnSO : ReductionFnSO, ITaggable
```

Inheritance

object ← Object ← ScriptableObject ← ReductionFnSO ← DamageMitigationFnSO

Implements

ITaggable

Derived

FlatDamageMitigationFnSO, HyperbolicDamageMitigationFnSO, LogDamageMitigationFnSO, PercentageDamageMitigationFnSO

Inherited Members

ReductionFnSO.Tags

Methods

CalculateMitigatedDamage(long, double, RoundingMode)

Reduces the specified `amount` of damage using the provided defensive statistic value and returns the resulting damage amount rounded with `roundingMode`.

```
public abstract long CalculateMitigatedDamage(long amount, double defensiveStatValue,
RoundingMode roundingMode)
```

Parameters

`amount` long

The original incoming damage amount to be reduced.

`defensiveStatValue` double

The value of the defensive statistic used to reduce damage (semantics depend on the implementation: absolute value, percentage, etc.).

`roundingMode` RoundingMode

Rounding mode applied to fractional results.

Returns

long

The reduced damage amount as a non-negative long. Implementations should ensure the returned value is >= 0 and apply `roundingMode` to fractional results.

CalculateMitigatedDamage(long, double, RoundingMode, in ReductionContext)

Same as CalculateMitigatedDamage(long, double, RoundingMode), for functions that also depend on the entities involved (a constant that grows with the attacker's level, another stat of the target). The damage pipeline always calls this overload.

```
public virtual long CalculateMitigatedDamage(long amount, double defensiveStatValue,
RoundingMode roundingMode, in ReductionContext context)
```

Parameters

`amount` long

The original incoming damage amount to be reduced.

`defensiveStatValue` double

The value of the defensive statistic used to reduce damage.

`roundingMode` RoundingMode

Rounding mode applied to fractional results.

`context` ReductionContext

The entities involved in the hit: their levels and stats. Valid only during the call.

Returns

long

The reduced damage amount as a non-negative long.

Remarks

The default implementation ignores `context` and forwards to the abstract overload, so existing implementations keep working unchanged. Override it to use the context.

Class FlatDamageMitigationFnSO

Namespace: ElectricDrill.AstraHealth.DamageMitigationFunctions

Assembly: com.electricdrill.astra-health.Runtime.dll

```
public class FlatDamageMitigationFnSO : DamageMitigationFnSO, ITaggable
```

Inheritance

object ← Object ← ScriptableObject ← ReductionFnSO ← DamageMitigationFnSO ← FlatDamageMitigationFnSO

Implements

ITaggable

Inherited Members

DamageMitigationFnSO.CalculateMitigatedDamage(long, double, RoundingMode, in ReductionContext) , ReductionFnSO.Tags

Methods

CalculateMitigatedDamage(long, double, RoundingMode)

Subtracts `defensiveStatValue * _factor` from the incoming `amount`. The result is clamped to a minimum of 0 and rounded to the nearest whole number.

```
public override long CalculateMitigatedDamage(long amount, double defensiveStatValue,
RoundingMode roundingMode)
```

Parameters

`amount` long

Incoming damage amount.

`defensiveStatValue` double

Value of the defensive statistic used to reduce damage. This value is multiplied by ElectricDrill.AstraHealth.DamageMitigationFunctions.FlatDamageMitigationFnSO._factor before being subtracted.

`roundingMode` RoundingMode

Rounding mode applied to the final fractional damage value.

Returns

long

The reduced damage as a non-negative long.

Class HyperbolicDamageMitigationFnSO

Namespace: ElectricDrill.AstraHealth.DamageMitigationFunctions

Assembly: com.electricdrill.astra-health.Runtime.dll

Mitigates damage by `X / (X + K)`, where `X` is the defensive stat and `K` the defense that gives 50% mitigation. The formula of LoL, Dota, WoW and Diablo.

```
public class HyperbolicDamageMitigationFnSO : DamageMitigationFnSO, ITaggable
```

Inheritance

object ← Object ← ScriptableObject ← ReductionFnSO ← DamageMitigationFnSO ← HyperbolicDamageMitigationFnSO

Implements

ITaggable

Inherited Members

ReductionFnSO.Tags

Remarks

Mitigation has diminishing returns, but every point of `X` adds the same share of effective health: `effective health = health × (1 + X / K)`. `K` is `BaseValue + PerLevel × level`: a constant (LoL: 100) or a level-dependent value (WoW: 400 + 85 × level). Create instances via Assets -> Create -> Astra Health / Dmg Mitigation Functions / Hyperbolic Dmg Mitigation.

Methods

CalculateMitigatedDamage(long, double, RoundingMode)

Reduces damage with `Damage × K / (X + K)`, evaluating `K` at level 1 because no entity is known. The gameplay path uses the overload with a ReductionContext.

```
public override long CalculateMitigatedDamage(long amount, double defensiveStatValue,
RoundingMode roundingMode)
```

Parameters

`amount` long

`defensiveStatValue` double

`roundingMode` RoundingMode

Returns

long

CalculateMitigatedDamage(long, double, RoundingMode, in ReductionContext)

Reduces damage with `Damage × K / (X + K)`.

```
public override long CalculateMitigatedDamage(long amount, double defensiveStatValue,
RoundingMode roundingMode, in ReductionContext context)
```

Parameters

`amount` long

Incoming damage amount.

`defensiveStatValue` double

Defensive stat value `X`.

`roundingMode` RoundingMode

Rounding mode applied to the fractional result.

`context` ReductionContext

Levels used to evaluate `K`.

Returns

long

The reduced damage as a non-negative long.

Remarks

If `defensiveStatValue` is zero or negative the original damage is returned (negative defense does not amplify damage, as with the logarithmic function). A `K` of zero mitigates all damage. The result is non-negative and rounded with `roundingMode`.

Class LogDamageMitigationFnSO

Namespace: ElectricDrill.AstraHealth.DamageMitigationFunctions

Assembly: com.electricdrill.astra-health.Runtime.dll

Legacy logarithmic mitigation. Kept so existing assets keep working; prefer HyperbolicDamageMitigationFnSO for new work. For small defense values it behaves like the hyperbolic function with `K = BaseValue / ScaleFactor`; at high values it grows far more slowly.

```
public class LogDamageMitigationFnSO : DamageMitigationFnSO, ITaggable
```

Inheritance

object ← Object ← ScriptableObject ← ReductionFnSO ← DamageMitigationFnSO ← LogDamageMitigationFnSO

Implements

ITaggable

Inherited Members

DamageMitigationFnSO.CalculateMitigatedDamage(long, double, RoundingMode, in ReductionContext) , ReductionFnSO.Tags

Methods

CalculateMitigatedDamage(long, double, RoundingMode)

Reduces damage using a logarithmic formula that produces diminishing returns as the defensive stat increases.

```
public override long CalculateMitigatedDamage(long amount, double defensiveStatValue,
RoundingMode roundingMode)
```

Parameters

`amount` long

Incoming damage amount.

`defensiveStatValue` double

Defensive statistic value used to compute the mitigation multiplier.

`roundingMode` RoundingMode

Rounding mode applied to the final fractional damage value.

Returns

long

The reduced damage as a non-negative long.

Remarks

Formula: Reduced Damage = Damage * (BaseValue / (BaseValue + Log(1 + DefensiveStat * ScaleFactor)))

If `defensiveStatValue` is zero or negative the original damage is returned. The final result is clamped to be non-negative and rounded to the nearest integer.

Class PercentageDamageMitigationFnSO

Namespace: ElectricDrill.AstraHealth.DamageMitigationFunctions

Assembly: com.electricdrill.astra-health.Runtime.dll

```
public class PercentageDamageMitigationFnSO : DamageMitigationFnSO, ITaggable
```

Inheritance

object ← Object ← ScriptableObject ← ReductionFnSO ← DamageMitigationFnSO ← PercentageDamageMitigationFnSO

Implements

ITaggable

Inherited Members

DamageMitigationFnSO.CalculateMitigatedDamage(long, double, RoundingMode, in ReductionContext) , ReductionFnSO.Tags

Methods

CalculateMitigatedDamage(long, double, RoundingMode)

Reduces damage by a percentage determined by `defensiveStatValue`. Example: defensiveStatValue == 25 -> damage reduced by 25%.

```
public override long CalculateMitigatedDamage(long amount, double defensiveStatValue,
RoundingMode roundingMode)
```

Parameters

`amount` long

Incoming damage amount.

`defensiveStatValue` double

Percentage (0..100+). Treated as a percent reduction of the incoming damage.

`roundingMode` RoundingMode

Rounding mode applied to the final fractional damage value.

Returns

long

The reduced damage as a non-negative long.

Remarks

The computation is: reducedAmount = amount - amount * defensiveStatValue / 100.0 The result is clamped to a minimum of 0 and rounded to the nearest integer.

Namespace ElectricDrill.AstraHealth.Defense PenetrationFunctions

Classes

DefensePenetrationFnSO

Base type for defense penetration functions. Implementations compute a reduced defense value based on a piercing stat and the pierced defense stat.

FlatDefensePenetrationFnSO

Reduces defense by subtracting a scaled amount of the piercing stat from the pierced defense. The result is clamped to the stat's min/max bounds. Create instances via Assets -> Create -> Astra Health / Def Penetration Functions / Flat Def Penetration.

HyperbolicDefensePenetrationFnSO

Pierces `P / (P + K)` of the target's defense, where `P` is the piercing stat and `K` the piercing that removes half of it. The penetration counterpart of HyperbolicDamageMitigationFnSO.

LogDefensePenetrationFnSO

Applies a logarithmic divisive reduction to defense to provide diminishing returns as piercing increases. Create instances via Assets -> Create -> Astra Health / Def Penetration Functions / Log Def Penetration (Legacy).

PercentageDefensePenetrationFnSO

Reduces defense by subtracting a percentage of the pierced defense equal to the piercing stat percentage. Example: piercingStatValue = 20 -> reduces defense by 20%. Create isntances via Assets -> Create -> Astra Health / Def Mitigation Functions / Log Def Mitigation.

Class DefensePenetrationFnSO

Namespace: ElectricDrill.AstraHealth.DefensePenetrationFunctions

Assembly: com.electricdrill.astra-health.Runtime.dll

Base type for defense penetration functions. Implementations compute a reduced defense value based on a piercing stat and the pierced defense stat.

```
public abstract class DefensePenetrationFnSO : ReductionFnSO, ITaggable
```

Inheritance

object ← Object ← ScriptableObject ← ReductionFnSO ← DefensePenetrationFnSO

Implements

ITaggable

Derived

FlatDefensePenetrationFnSO, HyperbolicDefensePenetrationFnSO, LogDefensePenetrationFnSO, PercentageDefensePenetrationFnSO

Inherited Members

ReductionFnSO.Tags

Methods

CalculatePiercedDefense(long, long, StatSO, in ReductionContext, bool)

Same as CalculatePiercedDefense(long, long, StatSO, bool), for functions that also depend on the entities involved (a constant that grows with the attacker's level, another stat of the attacker). The damage pipeline always calls this overload.

```
public virtual double CalculatePiercedDefense(long piercingStatValue, long
defensiveStatValue, StatSO defensiveStat, in ReductionContext context, bool applyClamp
= true)
```

Parameters

`piercingStatValue` long

The attacker's piercing stat value used to reduce defense.

`defensiveStatValue` long

The defender's original defense stat value to be reduced.

`defensiveStat` StatSO

The defensive stat object used to clamp the result within its min/max bounds.

`context` ReductionContext

The entities involved in the hit: their levels and stats. Valid only during the call.

`applyClamp` bool

Whether to apply stat clamping to the result.

Returns

double

The reduced defense value (clamped if applyClamp is true).

Remarks

The default implementation ignores `context` and forwards to the abstract overload, so existing implementations keep working unchanged. Override it to use the context.

CalculatePiercedDefense(long, long, StatSO, bool)

Compute the reduced defense value after applying a piercing stat.

```
public abstract double CalculatePiercedDefense(long piercingStatValue, long
defensiveStatValue, StatSO defensiveStat, bool applyClamp = true)
```

Parameters

`piercingStatValue` long

The attacker's piercing stat value used to reduce defense.

`defensiveStatValue` long

The defender's original defense stat value to be reduced.

`defensiveStat` StatSO

The defensive stat object used to clamp the result within its min/max bounds.

`applyClamp` bool

Whether to apply stat clamping to the result. Default true for normal gameplay; false for analysis/graphs.

Returns

double

The reduced defense value (clamped if applyClamp is true).

Class FlatDefensePenetrationFnSO

Namespace: ElectricDrill.AstraHealth.DefensePenetrationFunctions

Assembly: com.electricdrill.astra-health.Runtime.dll

Reduces defense by subtracting a scaled amount of the piercing stat from the pierced defense. The result is clamped to the stat's min/max bounds. Create instances via Assets -> Create -> Astra Health / Def Penetration Functions / Flat Def Penetration.

```
public class FlatDefensePenetrationFnSO : DefensePenetrationFnSO, ITaggable
```

Inheritance

object ← Object ← ScriptableObject ← ReductionFnSO ← DefensePenetrationFnSO ← FlatDefensePenetrationFnSO

Implements

ITaggable

Inherited Members

DefensePenetrationFnSO.CalculatePiercedDefense(long, long, StatSO, in ReductionContext, bool) , ReductionFnSO.Tags

Methods

CalculatePiercedDefense(long, long, StatSO, bool)

Subtracts (piercingStatValue * factor) from the pierced defense and optionally clamps the result to the stat's bounds.

```
public override double CalculatePiercedDefense(long piercingStatValue, long
defensiveStatValue, StatSO defensiveStat, bool applyClamp = true)
```

Parameters

`piercingStatValue` long

The attacker's piercing stat value used to reduce defense.

`defensiveStatValue` long

The defender's original defense stat value to be reduced.

`defensiveStat` StatSO

The defensive stat object used to clamp the result.

`applyClamp` bool

Whether to apply stat clamping to the result.

Returns

double

The reduced defense value (clamped to stat's min/max bounds if applyClamp is true).

Class HyperbolicDefensePenetrationFnSO

Namespace: ElectricDrill.AstraHealth.DefensePenetrationFunctions

Assembly: com.electricdrill.astra-health.Runtime.dll

Pierces `P / (P + K)` of the target's defense, where `P` is the piercing stat and `K` the piercing that removes half of it. The penetration counterpart of HyperbolicDamageMitigationFnSO.

```
public class HyperbolicDefensePenetrationFnSO : DefensePenetrationFnSO, ITaggable
```

Inheritance

object ← Object ← ScriptableObject ← ReductionFnSO ← DefensePenetrationFnSO ← HyperbolicDefensePenetrationFnSO

Implements

ITaggable

Inherited Members

ReductionFnSO.Tags

Remarks

`K` is `BaseValue + PerLevel × level`, like on the mitigation function, with its own values. Create instances via Assets -> Create -> Astra Health / Def Penetration Functions / Hyperbolic Def Penetration.

Methods

CalculatePiercedDefense(long, long, StatSO, in ReductionContext, bool)

Reduces defense with `Defense × K / (P + K)`.

```
public override double CalculatePiercedDefense(long piercingStatValue, long
defensiveStatValue, StatSO defensiveStat, in ReductionContext context, bool applyClamp
= true)
```

Parameters

`piercingStatValue` long

The attacker's piercing stat value `P`.

`defensiveStatValue` long

The defender's original defense stat value.

`defensiveStat` StatSO

The defensive stat used to clamp the result. No clamp is applied when it is null.

`context` ReductionContext

Levels used to evaluate `K`.

`applyClamp` bool

Whether to clamp the result to the stat's bounds.

Returns

double

The reduced defense value.

Remarks

If `piercingStatValue` is zero or negative the defense is returned unchanged.

CalculatePiercedDefense(long, long, StatSO, bool)

Reduces defense with `Defense × K / (P + K)`, evaluating `K` at level 1 because no entity is known. The gameplay path uses the overload with a ReductionContext.

```
public override double CalculatePiercedDefense(long piercingStatValue, long
defensiveStatValue, StatSO defensiveStat, bool applyClamp = true)
```

Parameters

`piercingStatValue` long

`defensiveStatValue` long

`defensiveStat` StatSO

`applyClamp` bool

Returns

double

Class LogDefensePenetrationFnSO

Namespace: ElectricDrill.AstraHealth.DefensePenetrationFunctions

Assembly: com.electricdrill.astra-health.Runtime.dll

Applies a logarithmic divisive reduction to defense to provide diminishing returns as piercing increases. Create instances via Assets -> Create -> Astra Health / Def Penetration Functions / Log Def Penetration (Legacy).

```
public class LogDefensePenetrationFnSO : DefensePenetrationFnSO, ITaggable
```

Inheritance

object ← Object ← ScriptableObject ← ReductionFnSO ← DefensePenetrationFnSO ← LogDefensePenetrationFnSO

Implements

ITaggable

Inherited Members

DefensePenetrationFnSO.CalculatePiercedDefense(long, long, StatSO, in ReductionContext, bool) , ReductionFnSO.Tags

Remarks

Legacy: kept so existing assets keep working; prefer HyperbolicDefensePenetrationFnSO for new work. For small piercing values it behaves like the hyperbolic function with `K = BaseValue / ScaleFactor`; at high values it grows far more slowly.

Methods

CalculatePiercedDefense(long, long, StatSO, bool)

Reduces defense using a logarithmic divisive formula that provides diminishing returns. Formula: Reduced Defense = Defense * (BaseValue / (BaseValue + Log(1 + PiercingStat * ScaleFactor)))

```
public override double CalculatePiercedDefense(long piercingStatValue, long
defensiveStatValue, StatSO defensiveStat, bool applyClamp = true)
```

Parameters

`piercingStatValue` long

The attacker's piercing stat value.

`defensiveStatValue` long

The defender's original defense stat value.

`defensiveStat` StatSO

The defensive stat object used to clamp the result.

`applyClamp` bool

Whether to apply stat clamping to the result.

Returns

double

The reduced defense value (clamped to stat's min/max bounds if applyClamp is true)

Class PercentageDefensePenetrationFnSO

Namespace: ElectricDrill.AstraHealth.DefensePenetrationFunctions

Assembly: com.electricdrill.astra-health.Runtime.dll

Reduces defense by subtracting a percentage of the pierced defense equal to the piercing stat percentage. Example: piercingStatValue = 20 -> reduces defense by 20%. Create isntances via Assets -> Create -> Astra Health / Def Mitigation Functions / Log Def Mitigation.

```
public class PercentageDefensePenetrationFnSO : DefensePenetrationFnSO, ITaggable
```

Inheritance

object ← Object ← ScriptableObject ← ReductionFnSO ← DefensePenetrationFnSO ← PercentageDefensePenetrationFnSO

Implements

ITaggable

Inherited Members

DefensePenetrationFnSO.CalculatePiercedDefense(long, long, StatSO, in ReductionContext, bool) , ReductionFnSO.Tags

Methods

CalculatePiercedDefense(long, long, StatSO, bool)

Apply percentage-based penetration to the pierced defense and optionally clamp the result to the stat's bounds.

```
public override double CalculatePiercedDefense(long piercingStatValue, long
defensiveStatValue, StatSO defensiveStat, bool applyClamp = true)
```

Parameters

`piercingStatValue` long

Percentage to reduce (e.g. 25 means 25%).

`defensiveStatValue` long

Original defense stat.

`defensiveStat` StatSO

The defensive stat object used to clamp the result.

`applyClamp` bool

Whether to apply stat clamping to the result.

Returns

double

The reduced defense value (clamped to stat's min/max bounds if applyClamp is true).

Namespace ElectricDrill.AstraHealth.Events

Classes

DamageResolutionGameEvent

Event raised to notify the resolution of a damage attempt (applied or prevented). Payload: DamageResolutionContext.

DamageResolutionGameEventListener

Event raised to notify the resolution of a damage attempt (applied or prevented). Payload: DamageResolutionContext.

EntityDiedGameEvent

Event raised when an entity dies. Payload: EntityDiedContext.

EntityDiedGameEventListener

Event raised when an entity dies. Payload: EntityDiedContext.

EntityHealedGameEvent

Event raised when an entity got healed. Payload: ReceivedHealContext.

EntityHealedGameEventListener

Event raised when an entity got healed. Payload: ReceivedHealContext.

EntityHealthChangedGameEvent

Event raised when an entity's health changes (gained or lost). Payload: EntityHealthChangedContext.

EntityHealthChangedGameEventListener

Event raised when an entity's health changes (gained or lost). Payload: EntityHealthChangedContext.

EntityMaxHealthChangedGameEvent

Event raised when an entity's maximum health changes. Payload: EntityMaxHealthChangedContext.

EntityMaxHealthChangedGameEventListener

Event raised when an entity's maximum health changes. Payload: EntityMaxHealthChangedContext.

EntityResurrectedGameEvent

Event raised when an entity is resurrected. Payload: ResurrectionContext.

EntityResurrectedGameEventListener

Event raised when an entity is resurrected. Payload: ResurrectionContext.

HealthRatioChangedGameEvent

Event raised when an entity's HP/MaxHP ratio changes (fired on both HP changes and MaxHP changes). Payload: HealthRatioChangedContext.

HealthRatioChangedGameEventListener

PreDamageGameEvent

Event raised before damage is processed. Payload: PreDamageContext.

PreDamageGameEventListener

Event raised before damage is processed. Payload: PreDamageContext.

PreHealGameEvent

The entity is about to be healed. Payload: PreHealContext.

PreHealGameEventListener

The entity is about to be healed. Payload: PreHealContext.

Class DamageResolutionGameEvent

Namespace: ElectricDrill.AstraHealth.Events

Assembly: com.electricdrill.astra-health.Runtime.dll

Event raised to notify the resolution of a damage attempt (applied or prevented). Payload: DamageResolutionContext.

```
[CreateAssetMenu(fileName = "DamageResolution Game Event", menuName = "Astra
Health/Events/Generated (Damage)/DamageResolution")]
public class DamageResolutionGameEvent : GameEventGeneric1<DamageResolutionContext>,
IRaisable<DamageResolutionContext>
```

Inheritance

object ← Object ← ScriptableObject ← GameEventBase<GameEventListenerGeneric1<DamageResolutionContext>, Action<DamageResolutionContext>> ← GameEventGeneric1<DamageResolutionContext> ← DamageResolutionGameEvent

Implements

IRaisable<DamageResolutionContext>

Inherited Members

GameEventGeneric1<DamageResolutionContext>.OnEventRaised , GameEventGeneric1<DamageResolutionContext>.Raise(DamageResolutionContext) , GameEventGeneric1<DamageResolutionContext>.RegisterListener(GameEventListenerGeneric1<DamageResolutionContext>) , GameEventGeneric1<DamageResolutionContext>.UnregisterListener(GameEventListenerGeneric1<DamageResolutionContext>) , GameEventBase<GameEventListenerGeneric1<DamageResolutionContext>, Action<DamageResolutionContext>>.RegisterMonoListener(GameEventListenerGeneric1<DamageResolutionContext>) , GameEventBase<GameEventListenerGeneric1<DamageResolutionContext>, Action<DamageResolutionContext>>.UnregisterMonoListener(GameEventListenerGeneric1<DamageResolutionContext>) , GameEventBase<GameEventListenerGeneric1<DamageResolutionContext>, Action<DamageResolutionContext>>.RegisterCodeListener(Action<DamageResolutionContext>) ,

GameEventBase<GameEventListenerGeneric1<DamageResolutionContext>,
Action<DamageResolutionContext>>.UnregisterCodeListener(Action<DamageResolutionContext>) ,
GameEventBase<GameEventListenerGeneric1<DamageResolutionContext>,
Action<DamageResolutionContext>>.Dispatch<TInvoker>(TInvoker)

Class DamageResolutionGameEventListener

Namespace: ElectricDrill.AstraHealth.Events

Assembly: com.electricdrill.astra-health.Runtime.dll

Event raised to notify the resolution of a damage attempt (applied or prevented). Payload: DamageResolutionContext.

```
public class DamageResolutionGameEventListener :
GameEventListenerGeneric1<DamageResolutionContext>
```

Inheritance

object ← Object ← Component ← Behaviour ← MonoBehaviour ← GameEventListenerGeneric1<DamageResolutionContext> ← DamageResolutionGameEventListener

Inherited Members

GameEventListenerGeneric1<DamageResolutionContext>._event ,
GameEventListenerGeneric1<DamageResolutionContext>._response ,
GameEventListenerGeneric1<DamageResolutionContext>._ownerAwareGameActions ,
GameEventListenerGeneric1<DamageResolutionContext>.OnEventRaised(DamageResolutionContext)

Class EntityDiedGameEvent

Namespace: ElectricDrill.AstraHealth.Events

Assembly: com.electricdrill.astra-health.Runtime.dll

Event raised when an entity dies. Payload: EntityDiedContext.

```
[CreateAssetMenu(fileName = "EntityDied Game Event", menuName = "Astra
Health/Events/Generated (Health)/EntityDied")]
public class EntityDiedGameEvent : GameEventGeneric1<EntityDiedContext>,
IRaisable<EntityDiedContext>
```

Inheritance

object ← Object ← ScriptableObject ← GameEventBase<GameEventListenerGeneric1<EntityDiedContext>, Action<EntityDiedContext>> ← GameEventGeneric1<EntityDiedContext> ← EntityDiedGameEvent

Implements

IRaisable<EntityDiedContext>

Inherited Members

GameEventGeneric1<EntityDiedContext>.OnEventRaised , GameEventGeneric1<EntityDiedContext>.Raise(EntityDiedContext) , GameEventGeneric1<EntityDiedContext>.RegisterListener(GameEventListenerGeneric1<EntityDiedContext>) , GameEventGeneric1<EntityDiedContext>.UnregisterListener(GameEventListenerGeneric1<EntityDiedContext>) , GameEventBase<GameEventListenerGeneric1<EntityDiedContext>, Action<EntityDiedContext>>.RegisterMonoListener(GameEventListenerGeneric1<EntityDiedContext>) , GameEventBase<GameEventListenerGeneric1<EntityDiedContext>, Action<EntityDiedContext>>.UnregisterMonoListener(GameEventListenerGeneric1<EntityDiedContext>) , GameEventBase<GameEventListenerGeneric1<EntityDiedContext>, Action<EntityDiedContext>>.RegisterCodeListener(Action<EntityDiedContext>) , GameEventBase<GameEventListenerGeneric1<EntityDiedContext>, Action<EntityDiedContext>>.UnregisterCodeListener(Action<EntityDiedContext>) , GameEventBase<GameEventListenerGeneric1<EntityDiedContext>, Action<EntityDiedContext>>.Dispatch<TInvoker>(TInvoker)

Class EntityDiedGameEventListener

Namespace: ElectricDrill.AstraHealth.Events

Assembly: com.electricdrill.astra-health.Runtime.dll

Event raised when an entity dies. Payload: EntityDiedContext.

```
public class EntityDiedGameEventListener : GameEventListenerGeneric1<EntityDiedContext>
```

Inheritance

object ← Object ← Component ← Behaviour ← MonoBehaviour ←
GameEventListenerGeneric1<EntityDiedContext> ← EntityDiedGameEventListener

Inherited Members

GameEventListenerGeneric1<EntityDiedContext>._event ,
GameEventListenerGeneric1<EntityDiedContext>._response ,
GameEventListenerGeneric1<EntityDiedContext>._ownerAwareGameActions ,
GameEventListenerGeneric1<EntityDiedContext>.OnEventRaised(EntityDiedContext)

Class EntityHealedGameEvent

Namespace: ElectricDrill.AstraHealth.Events

Assembly: com.electricdrill.astra-health.Runtime.dll

Event raised when an entity got healed. Payload: ReceivedHealContext.

```
[CreateAssetMenu(fileName = "EntityHealed Game Event", menuName = "Astra
Health/Events/Generated (Health)/EntityHealed")]
public class EntityHealedGameEvent : GameEventGeneric1<ReceivedHealContext>,
IRaisable<ReceivedHealContext>
```

Inheritance

object ← Object ← ScriptableObject ←
GameEventBase<GameEventListenerGeneric1<ReceivedHealContext>, Action<ReceivedHealContext>> ←
GameEventGeneric1<ReceivedHealContext> ← EntityHealedGameEvent

Implements

IRaisable<ReceivedHealContext>

Inherited Members

GameEventGeneric1<ReceivedHealContext>.OnEventRaised ,
GameEventGeneric1<ReceivedHealContext>.Raise(ReceivedHealContext) ,
GameEventGeneric1<ReceivedHealContext>.RegisterListener(GameEventListenerGeneric1<ReceivedHealContext>) ,
GameEventGeneric1<ReceivedHealContext>.UnregisterListener(GameEventListenerGeneric1<ReceivedHealContext>) ,
GameEventBase<GameEventListenerGeneric1<ReceivedHealContext>, Action<ReceivedHealContext>>.RegisterMonoListener(GameEventListenerGeneric1<ReceivedHealContext>) ,
GameEventBase<GameEventListenerGeneric1<ReceivedHealContext>, Action<ReceivedHealContext>>.UnregisterMonoListener(GameEventListenerGeneric1<ReceivedHealContext>) ,
GameEventBase<GameEventListenerGeneric1<ReceivedHealContext>, Action<ReceivedHealContext>>.RegisterCodeListener(Action<ReceivedHealContext>) ,
GameEventBase<GameEventListenerGeneric1<ReceivedHealContext>, Action<ReceivedHealContext>>.UnregisterCodeListener(Action<ReceivedHealContext>) ,
GameEventBase<GameEventListenerGeneric1<ReceivedHealContext>, Action<ReceivedHealContext>>.Dispatch<TInvoker>(TInvoker)

Class EntityHealedGameEventListener

Namespace: ElectricDrill.AstraHealth.Events

Assembly: com.electricdrill.astra-health.Runtime.dll

Event raised when an entity got healed. Payload: ReceivedHealContext.

```
public class EntityHealedGameEventListener : GameEventListenerGeneric1<ReceivedHealContext>
```

Inheritance

object ← Object ← Component ← Behaviour ← MonoBehaviour ←
GameEventListenerGeneric1<ReceivedHealContext> ← EntityHealedGameEventListener

Inherited Members

GameEventListenerGeneric1<ReceivedHealContext>._event ,
GameEventListenerGeneric1<ReceivedHealContext>._response ,
GameEventListenerGeneric1<ReceivedHealContext>._ownerAwareGameActions ,
GameEventListenerGeneric1<ReceivedHealContext>.OnEventRaised(ReceivedHealContext)

Class EntityHealthChangedGameEvent

Namespace: ElectricDrill.AstraHealth.Events

Assembly: com.electricdrill.astra-health.Runtime.dll

Event raised when an entity's health changes (gained or lost). Payload: EntityHealthChangedContext.

```
[CreateAssetMenu(fileName = "EntityHealthChanged Game Event", menuName = "Astra
Health/Events/Generated (Health)/EntityHealthChanged")]
public class EntityHealthChangedGameEvent : GameEventGeneric1<EntityHealthChangedContext>,
IRaisable<EntityHealthChangedContext>
```

Inheritance

object ← Object ← ScriptableObject ←
GameEventBase<GameEventListenerGeneric1<EntityHealthChangedContext>, Action<EntityHealthChangedContext>> ←
GameEventGeneric1<EntityHealthChangedContext> ← EntityHealthChangedGameEvent

Implements

IRaisable<EntityHealthChangedContext>

Inherited Members

GameEventGeneric1<EntityHealthChangedContext>.OnEventRaised ,
GameEventGeneric1<EntityHealthChangedContext>.Raise(EntityHealthChangedContext) ,
GameEventGeneric1<EntityHealthChangedContext>.RegisterListener(GameEventListenerGeneric1<EntityHealthChangedContext>) ,
GameEventGeneric1<EntityHealthChangedContext>.UnregisterListener(GameEventListenerGeneric1<EntityHealthChangedContext>) ,
GameEventBase<GameEventListenerGeneric1<EntityHealthChangedContext>, Action<EntityHealthChangedContext>>.RegisterMonoListener(GameEventListenerGeneric1<EntityHealthChangedContext>) ,
GameEventBase<GameEventListenerGeneric1<EntityHealthChangedContext>, Action<EntityHealthChangedContext>>.UnregisterMonoListener(GameEventListenerGeneric1<EntityHealthChangedContext>) ,
GameEventBase<GameEventListenerGeneric1<EntityHealthChangedContext>, Action<EntityHealthChangedContext>>.RegisterCodeListener(Action<EntityHealthChangedContext>) ,

GameEventBase<GameEventListenerGeneric1<EntityHealthChangedContext>, Action<EntityHealthChangedContext>>.UnregisterCodeListener(Action<EntityHealthChangedContext>)
,
GameEventBase<GameEventListenerGeneric1<EntityHealthChangedContext>, Action<EntityHealthChangedContext>>.Dispatch<TInvoker>(TInvoker)

Class EntityHealthChangedGameEventListener

Namespace: ElectricDrill.AstraHealth.Events

Assembly: com.electricdrill.astra-health.Runtime.dll

Event raised when an entity's health changes (gained or lost). Payload: EntityHealthChangedContext.

```
public class EntityHealthChangedGameEventListener :
GameEventListenerGeneric1<EntityHealthChangedContext>
```

Inheritance

object ← Object ← Component ← Behaviour ← MonoBehaviour ← GameEventListenerGeneric1<EntityHealthChangedContext> ← EntityHealthChangedGameEventListener

Inherited Members

GameEventListenerGeneric1<EntityHealthChangedContext>._event , GameEventListenerGeneric1<EntityHealthChangedContext>._response , GameEventListenerGeneric1<EntityHealthChangedContext>._ownerAwareGameActions , GameEventListenerGeneric1<EntityHealthChangedContext>.OnEventRaised(EntityHealthChangedContext)

Class EntityMaxHealthChangedGameEvent

Namespace: ElectricDrill.AstraHealth.Events

Assembly: com.electricdrill.astra-health.Runtime.dll

Event raised when an entity's maximum health changes. Payload: EntityMaxHealthChangedContext.

```
[CreateAssetMenu(fileName = "EntityMaxHealthChanged Game Event", menuName = "Astra
Health/Events/Generated (Health)/EntityMaxHealthChanged")]
public class EntityMaxHealthChangedGameEvent :
GameEventGeneric1<EntityMaxHealthChangedContext>, IRaisable<EntityMaxHealthChangedContext>
```

Inheritance

object ← Object ← ScriptableObject ← GameEventBase<GameEventListenerGeneric1<EntityMaxHealthChangedContext>, Action<EntityMaxHealthChangedContext>> ← GameEventGeneric1<EntityMaxHealthChangedContext> ← EntityMaxHealthChangedGameEvent

Implements

IRaisable<EntityMaxHealthChangedContext>

Inherited Members

GameEventGeneric1<EntityMaxHealthChangedContext>.OnEventRaised , GameEventGeneric1<EntityMaxHealthChangedContext>.Raise(EntityMaxHealthChangedContext) , GameEventGeneric1<EntityMaxHealthChangedContext>.RegisterListener(GameEventListenerGeneric1<EntityMaxHealthChangedContext>) , GameEventGeneric1<EntityMaxHealthChangedContext>.UnregisterListener(GameEventListenerGeneric1<EntityMaxHealthChangedContext>) , GameEventBase<GameEventListenerGeneric1<EntityMaxHealthChangedContext>, Action<EntityMaxHealthChangedContext>>.RegisterMonoListener(GameEventListenerGeneric1<EntityMaxHealthChangedContext>) , GameEventBase<GameEventListenerGeneric1<EntityMaxHealthChangedContext>, Action<EntityMaxHealthChangedContext>>.UnregisterMonoListener(GameEventListenerGeneric1<EntityMaxHealthChangedContext>) , GameEventBase<GameEventListenerGeneric1<EntityMaxHealthChangedContext>, Action<EntityMaxHealthChangedContext>>.RegisterCodeListener(Action<EntityMaxHealthChangedContext>) ,

GameEventBase<GameEventListenerGeneric1<EntityMaxHealthChangedContext>, Action<EntityMaxHealthChangedContext>>.UnregisterCodeListener(Action<EntityMaxHealthChangedContext>) ,
GameEventBase<GameEventListenerGeneric1<EntityMaxHealthChangedContext>, Action<EntityMaxHealthChangedContext>>.Dispatch<TInvoker>(TInvoker)

Class EntityMaxHealthChangedGameEventListener

Namespace: ElectricDrill.AstraHealth.Events

Assembly: com.electricdrill.astra-health.Runtime.dll

Event raised when an entity's maximum health changes. Payload: EntityMaxHealthChangedContext.

```
public class EntityMaxHealthChangedGameEventListener :
GameEventListenerGeneric1<EntityMaxHealthChangedContext>
```

Inheritance

object ← Object ← Component ← Behaviour ← MonoBehaviour ← GameEventListenerGeneric1<EntityMaxHealthChangedContext> ← EntityMaxHealthChangedGameEventListener

Inherited Members

GameEventListenerGeneric1<EntityMaxHealthChangedContext>._event , GameEventListenerGeneric1<EntityMaxHealthChangedContext>._response , GameEventListenerGeneric1<EntityMaxHealthChangedContext>._ownerAwareGameActions , GameEventListenerGeneric1<EntityMaxHealthChangedContext>.OnEventRaised(EntityMaxHealthChangedContext)

Class EntityResurrectedGameEvent

Namespace: ElectricDrill.AstraHealth.Events

Assembly: com.electricdrill.astra-health.Runtime.dll

Event raised when an entity is resurrected. Payload: ResurrectionContext.

```
[CreateAssetMenu(fileName = "EntityResurrected Game Event", menuName = "Astra
Health/Events/Generated (Health)/EntityResurrected")]
public class EntityResurrectedGameEvent : GameEventGeneric1<ResurrectionContext>,
IRaisable<ResurrectionContext>
```

Inheritance

object ← Object ← ScriptableObject ←
GameEventBase<GameEventListenerGeneric1<ResurrectionContext>, Action<ResurrectionContext>> ←
GameEventGeneric1<ResurrectionContext> ← EntityResurrectedGameEvent

Implements

IRaisable<ResurrectionContext>

Inherited Members

GameEventGeneric1<ResurrectionContext>.OnEventRaised ,
GameEventGeneric1<ResurrectionContext>.Raise(ResurrectionContext) ,
GameEventGeneric1<ResurrectionContext>.RegisterListener(GameEventListenerGeneric1<ResurrectionContext>) ,
GameEventGeneric1<ResurrectionContext>.UnregisterListener(GameEventListenerGeneric1<ResurrectionContext>) ,
GameEventBase<GameEventListenerGeneric1<ResurrectionContext>, Action<ResurrectionContext>>.RegisterMonoListener(GameEventListenerGeneric1<ResurrectionContext>) ,
GameEventBase<GameEventListenerGeneric1<ResurrectionContext>, Action<ResurrectionContext>>.UnregisterMonoListener(GameEventListenerGeneric1<ResurrectionContext>) ,
GameEventBase<GameEventListenerGeneric1<ResurrectionContext>, Action<ResurrectionContext>>.RegisterCodeListener(Action<ResurrectionContext>) ,
GameEventBase<GameEventListenerGeneric1<ResurrectionContext>, Action<ResurrectionContext>>.UnregisterCodeListener(Action<ResurrectionContext>) ,
GameEventBase<GameEventListenerGeneric1<ResurrectionContext>, Action<ResurrectionContext>>.Dispatch<TInvoker>(TInvoker)

Class EntityResurrectedGameEventListener

Namespace: ElectricDrill.AstraHealth.Events

Assembly: com.electricdrill.astra-health.Runtime.dll

Event raised when an entity is resurrected. Payload: ResurrectionContext.

```
public class EntityResurrectedGameEventListener :
GameEventListenerGeneric1<ResurrectionContext>
```

Inheritance

object ← Object ← Component ← Behaviour ← MonoBehaviour ←
GameEventListenerGeneric1<ResurrectionContext> ← EntityResurrectedGameEventListener

Inherited Members

GameEventListenerGeneric1<ResurrectionContext>._event ,
GameEventListenerGeneric1<ResurrectionContext>._response ,
GameEventListenerGeneric1<ResurrectionContext>._ownerAwareGameActions ,
GameEventListenerGeneric1<ResurrectionContext>.OnEventRaised(ResurrectionContext)

Class HealthRatioChangedGameEvent

Namespace: ElectricDrill.AstraHealth.Events

Assembly: com.electricdrill.astra-health.Runtime.dll

Event raised when an entity's HP/MaxHP ratio changes (fired on both HP changes and MaxHP changes). Payload: HealthRatioChangedContext.

```
[CreateAssetMenu(fileName = "HealthRatioChanged Game Event", menuName = "Astra
Health/Events/Generated (Health)/HealthRatioChanged")]
public class HealthRatioChangedGameEvent : GameEventGeneric1<HealthRatioChangedContext>,
IRaisable<HealthRatioChangedContext>
```

Inheritance

object ← Object ← ScriptableObject ← GameEventBase<GameEventListenerGeneric1<HealthRatioChangedContext>, Action<HealthRatioChangedContext>> ← GameEventGeneric1<HealthRatioChangedContext> ← HealthRatioChangedGameEvent

Implements

IRaisable<HealthRatioChangedContext>

Inherited Members

GameEventGeneric1<HealthRatioChangedContext>.OnEventRaised , GameEventGeneric1<HealthRatioChangedContext>.Raise(HealthRatioChangedContext) , GameEventGeneric1<HealthRatioChangedContext>.RegisterListener(GameEventListenerGeneric1<HealthRatioChangedContext>) , GameEventGeneric1<HealthRatioChangedContext>.UnregisterListener(GameEventListenerGeneric1<HealthRatioChangedContext>) , GameEventBase<GameEventListenerGeneric1<HealthRatioChangedContext>, Action<HealthRatioChangedContext>>.RegisterMonoListener(GameEventListenerGeneric1<HealthRatioChangedContext>) , GameEventBase<GameEventListenerGeneric1<HealthRatioChangedContext>, Action<HealthRatioChangedContext>>.UnregisterMonoListener(GameEventListenerGeneric1<HealthRatioChangedContext>) , GameEventBase<GameEventListenerGeneric1<HealthRatioChangedContext>, Action<HealthRatioChangedContext>>.RegisterCodeListener(Action<HealthRatioChangedContext>) ,

GameEventBase<GameEventListenerGeneric1<HealthRatioChangedContext>,
Action<HealthRatioChangedContext>>.UnregisterCodeListener(Action<HealthRatioChangedContext>),
GameEventBase<GameEventListenerGeneric1<HealthRatioChangedContext>,
Action<HealthRatioChangedContext>>.Dispatch<TInvoker>(TInvoker)

Class HealthRatioChangedGameEventListener

Namespace: ElectricDrill.AstraHealth.Events

Assembly: com.electricdrill.astra-health.Runtime.dll

```
public class HealthRatioChangedGameEventListener :
GameEventListenerGeneric1<HealthRatioChangedContext>
```

Inheritance

object ← Object ← Component ← Behaviour ← MonoBehaviour ←
GameEventListenerGeneric1<HealthRatioChangedContext> ← HealthRatioChangedGameEventListener

Inherited Members

GameEventListenerGeneric1<HealthRatioChangedContext>._event ,
GameEventListenerGeneric1<HealthRatioChangedContext>._response ,
GameEventListenerGeneric1<HealthRatioChangedContext>._ownerAwareGameActions ,
GameEventListenerGeneric1<HealthRatioChangedContext>.OnEventRaised(HealthRatioChangedContext)

Class PreDamageGameEvent

Namespace: ElectricDrill.AstraHealth.Events

Assembly: com.electricdrill.astra-health.Runtime.dll

Event raised before damage is processed. Payload: PreDamageContext.

```
[CreateAssetMenu(fileName = "PreDamage Game Event", menuName = "Astra
Health/Events/Generated (Damage)/PreDamage")]
public class PreDamageGameEvent : GameEventGeneric1<PreDamageContext>,
IRaisable<PreDamageContext>
```

Inheritance

object ← Object ← ScriptableObject ← GameEventBase<GameEventListenerGeneric1<PreDamageContext>, Action<PreDamageContext>> ← GameEventGeneric1<PreDamageContext> ← PreDamageGameEvent

Implements

IRaisable<PreDamageContext>

Inherited Members

GameEventGeneric1<PreDamageContext>.OnEventRaised , GameEventGeneric1<PreDamageContext>.Raise(PreDamageContext) , GameEventGeneric1<PreDamageContext>.RegisterListener(GameEventListenerGeneric1<PreDamageContext>) , GameEventGeneric1<PreDamageContext>.UnregisterListener(GameEventListenerGeneric1<PreDamageContext>) , GameEventBase<GameEventListenerGeneric1<PreDamageContext>, Action<PreDamageContext>>.RegisterMonoListener(GameEventListenerGeneric1<PreDamageContext>) , GameEventBase<GameEventListenerGeneric1<PreDamageContext>, Action<PreDamageContext>>.UnregisterMonoListener(GameEventListenerGeneric1<PreDamageContext>) , GameEventBase<GameEventListenerGeneric1<PreDamageContext>, Action<PreDamageContext>>.RegisterCodeListener(Action<PreDamageContext>) , GameEventBase<GameEventListenerGeneric1<PreDamageContext>, Action<PreDamageContext>>.UnregisterCodeListener(Action<PreDamageContext>) , GameEventBase<GameEventListenerGeneric1<PreDamageContext>, Action<PreDamageContext>>.Dispatch<TInvoker>(TInvoker)

Class PreDamageGameEventListener

Namespace: ElectricDrill.AstraHealth.Events

Assembly: com.electricdrill.astra-health.Runtime.dll

Event raised before damage is processed. Payload: PreDamageContext.

```
public class PreDamageGameEventListener : GameEventListenerGeneric1<PreDamageContext>
```

Inheritance

object ← Object ← Component ← Behaviour ← MonoBehaviour ←
GameEventListenerGeneric1<PreDamageContext> ← PreDamageGameEventListener

Inherited Members

GameEventListenerGeneric1<PreDamageContext>._event ,
GameEventListenerGeneric1<PreDamageContext>._response ,
GameEventListenerGeneric1<PreDamageContext>._ownerAwareGameActions ,
GameEventListenerGeneric1<PreDamageContext>.OnEventRaised(PreDamageContext)

Class PreHealGameEvent

Namespace: ElectricDrill.AstraHealth.Events

Assembly: com.electricdrill.astra-health.Runtime.dll

The entity is about to be healed. Payload: PreHealContext.

```
[CreateAssetMenu(fileName = "PreHeal Game Event", menuName = "Astra
Health/Events/Generated (Health)/PreHeal")]
public class PreHealGameEvent : GameEventGeneric1<PreHealContext>, IRaisable<PreHealContext>
```

Inheritance

object ← Object ← ScriptableObject ←
GameEventBase<GameEventListenerGeneric1<PreHealContext>, Action<PreHealContext>> ←
GameEventGeneric1<PreHealContext> ← PreHealGameEvent

Implements

IRaisable<PreHealContext>

Inherited Members

GameEventGeneric1<PreHealContext>.OnEventRaised ,
GameEventGeneric1<PreHealContext>.Raise(PreHealContext) ,
GameEventGeneric1<PreHealContext>.RegisterListener(GameEventListenerGeneric1<PreHealContext>) ,
GameEventGeneric1<PreHealContext>.UnregisterListener(GameEventListenerGeneric1<PreHealContext>
) ,
GameEventBase<GameEventListenerGeneric1<PreHealContext>,
Action<PreHealContext>>.RegisterMonoListener(GameEventListenerGeneric1<PreHealContext>) ,
GameEventBase<GameEventListenerGeneric1<PreHealContext>,
Action<PreHealContext>>.UnregisterMonoListener(GameEventListenerGeneric1<PreHealContext>) ,
GameEventBase<GameEventListenerGeneric1<PreHealContext>,
Action<PreHealContext>>.RegisterCodeListener(Action<PreHealContext>) ,
GameEventBase<GameEventListenerGeneric1<PreHealContext>,
Action<PreHealContext>>.UnregisterCodeListener(Action<PreHealContext>) ,
GameEventBase<GameEventListenerGeneric1<PreHealContext>,
Action<PreHealContext>>.Dispatch<TInvoker>(TInvoker)

Class PreHealGameEventListener

Namespace: ElectricDrill.AstraHealth.Events

Assembly: com.electricdrill.astra-health.Runtime.dll

The entity is about to be healed. Payload: PreHealContext.

```
public class PreHealGameEventListener : GameEventListenerGeneric1<PreHealContext>
```

Inheritance

object ← Object ← Component ← Behaviour ← MonoBehaviour ←
GameEventListenerGeneric1<PreHealContext> ← PreHealGameEventListener

Inherited Members

GameEventListenerGeneric1<PreHealContext>._event ,
GameEventListenerGeneric1<PreHealContext>._response ,
GameEventListenerGeneric1<PreHealContext>._ownerAwareGameActions ,
GameEventListenerGeneric1<PreHealContext>.OnEventRaised(PreHealContext)

Namespace ElectricDrill.AstraHealth.Events.Contexts

Classes

EntityDiedContext

EntityHealthChangedContext

EntityMaxHealthChangedContext

HealthRatioChangedContext

Fired whenever the HP/MaxHP ratio changes — either because HP changed (AddHealth/RemoveHealth) or because MaxHP changed (RecalculateMaxHp). Provides a single reliable signal for HP-ratio-based conditions. Implements ElectricDrill.AstraRpgFramework.Contexts.IHasValueChange<T> for int where Previous/NewValue are HP percentages (0–100).

ResurrectionContext

Contains information about an entity resurrection event. All heal-related details (target, source, healer, amounts) are accessible via ReceivedHeal, which is the result of the internal Heal(PreHealContext) call performed during resurrection.

Class EntityDiedContext

Namespace: ElectricDrill.AstraHealth.Events.Contexts

Assembly: com.electricdrill.astra-health.Runtime.dll

```
public class EntityDiedContext : IHasEntity, IHasVictim, IHasTarget, IHasPerformer,
IHasAmount, IHasInstigator, IReactable
```

Inheritance

object ← EntityDiedContext

Implements

IHasEntity, IHasVictim, IHasTarget, IHasPerformer, IHasAmount, IHasInstigator, IReactable

Constructors

EntityDiedContext(EntityCore, EntityCore, DamageResolutionContext)

```
public EntityDiedContext(EntityCore victim, EntityCore performer,
DamageResolutionContext damageResolutionContext)
```

Parameters

`victim` EntityCore

`performer` EntityCore

`damageResolutionContext` DamageResolutionContext

Properties

DamageResolutionContext

```
public DamageResolutionContext DamageResolutionContext { get; }
```

Property Value

DamageResolutionContext

Entity

```
public EntityCore Entity { get; }
```

Property Value

EntityCore

Instigator

The specific thing that caused the killing blow — propagated from the originating DamageResolution Context. May be `null` if unspecified.

```
public IEffectInstigator Instigator { get; }
```

Property Value

IEffectInstigator

IsReactable

Propagated from the lethal IsReactable. When `false`, the killing blow was itself a secondary effect and reactive systems (such as counter-damage-on-death actions) should not respond to this death event.

```
public bool IsReactable { get; }
```

Property Value

bool

Performer

The entity that was responsible for the death of the Victim.

```
public EntityCore Performer { get; }
```

Property Value

EntityCore

Target

The killed entity. Same as Victim in this case.

```
public EntityCore Target { get; }
```

Property Value

EntityCore

Victim

The entity that died.

```
public EntityCore Victim { get; }
```

Property Value

EntityCore

Class EntityHealthChangedContext

Namespace: ElectricDrill.AstraHealth.Events.Contexts

Assembly: com.electricdrill.astra-health.Runtime.dll

```
public class EntityHealthChangedContext : IHasEntity, IHasTarget, IHasValueChange<long>
```

Inheritance

object ← EntityHealthChangedContext

Implements

IHasEntity, IHasTarget, IHasValueChange<long>

Constructors

EntityHealthChangedContext(EntityCore, long, long)

```
public EntityHealthChangedContext(EntityCore target, long previousValue, long newValue)
```

Parameters

`target` EntityCore

`previousValue` long

`newValue` long

Properties

AbsAmount

The absolute amount of change between PreviousValue and NewValue.

```
public long AbsAmount { get; }
```

Property Value

long

Entity

The entity whose health changed. Same as Target in this case.

```
public EntityCore Entity { get; }
```

Property Value

EntityCore

NewValue

```
public long NewValue { get; }
```

Property Value

long

PreviousValue

```
public long PreviousValue { get; }
```

Property Value

long

Target

The entity whose health changed.

```
public EntityCore Target { get; }
```

Property Value

EntityCore

Class EntityMaxHealthChangedContext

Namespace: ElectricDrill.AstraHealth.Events.Contexts

Assembly: com.electricdrill.astra-health.Runtime.dll

```
public class EntityMaxHealthChangedContext : IHasEntity, IHasTarget, IHasValueChange<long>
```

Inheritance

object ← EntityMaxHealthChangedContext

Implements

IHasEntity, IHasTarget, IHasValueChange<long>

Constructors

EntityMaxHealthChangedContext(EntityCore, long, long)

```
public EntityMaxHealthChangedContext(EntityCore target, long previousValue, long newValue)
```

Parameters

`target` EntityCore

`previousValue` long

`newValue` long

Properties

AbsAmount

The absolute amount of change between PreviousValue and NewValue.

```
public long AbsAmount { get; }
```

Property Value

long

Entity

The entity whose max health changed. Same as Target in this case.

```
public EntityCore Entity { get; }
```

Property Value

EntityCore

NewValue

```
public long NewValue { get; }
```

Property Value

long

PreviousValue

```
public long PreviousValue { get; }
```

Property Value

long

Target

The entity whose max health changed.

```
public EntityCore Target { get; }
```

Property Value

EntityCore

Class HealthRatioChangedContext

Namespace: ElectricDrill.AstraHealth.Events.Contexts

Assembly: com.electricdrill.astra-health.Runtime.dll

Fired whenever the HP/MaxHP ratio changes — either because HP changed (AddHealth/RemoveHealth) or because MaxHP changed (RecalculateMaxHp). Provides a single reliable signal for HP-ratio-based conditions. Implements ElectricDrill.AstraRpgFramework.Contexts.IHasValueChange<T> for int where Previous/NewValue are HP percentages (0–100).

```
public class HealthRatioChangedContext : IHasEntity, IHasTarget, IHasValueChange<int>
```

Inheritance

object ← HealthRatioChangedContext

Implements

IHasEntity, IHasTarget, IHasValueChange<int>

Constructors

HealthRatioChangedContext(EntityCore, long, long, long, long)

```
public HealthRatioChangedContext(EntityCore target, long previousHp, long newHp, long
previousMaxHp, long newMaxHp)
```

Parameters

`target` EntityCore

`previousHp` long

`newHp` long

`previousMaxHp` long

`newMaxHp` long

Properties

AbsAmount

The absolute amount of change between PreviousValue and NewValue.

```
public int AbsAmount { get; }
```

Property Value

int

Entity

The primary ElectricDrill.AstraRpgFramework.EntityCore associated with this object. For entity components this is the owning entity; for context payloads this is the primary affected subject (e.g. the event target).

```
public EntityCore Entity { get; }
```

Property Value

EntityCore

NewHp

```
public long NewHp { get; }
```

Property Value

long

NewMaxHp

```
public long NewMaxHp { get; }
```

Property Value

long

NewValue

```
public int NewValue { get; }
```

Property Value

int

PreviousHp

```
public long PreviousHp { get; }
```

Property Value

long

PreviousMaxHp

```
public long PreviousMaxHp { get; }
```

Property Value

long

PreviousValue

```
public int PreviousValue { get; }
```

Property Value

int

Target

```
public EntityCore Target { get; }
```

Property Value

EntityCore

Class ResurrectionContext

Namespace: ElectricDrill.AstraHealth.Events.Contexts

Assembly: com.electricdrill.astra-health.Runtime.dll

Contains information about an entity resurrection event. All heal-related details (target, source, healer, amounts) are accessible via ReceivedHeal, which is the result of the internal Heal(PreHealContext) call performed during resurrection.

```
public class ResurrectionContext : IHasEntity, IHasTarget, IHasValueChange<long>,
IHasInstigator
```

Inheritance

object ← ResurrectionContext

Implements

IHasEntity, IHasTarget, IHasValueChange<long>, IHasInstigator

Constructors

ResurrectionContext(long, ReceivedHealContext)

Initializes a new instance of ResurrectionContext.

```
public ResurrectionContext(long previousHp, ReceivedHealContext receivedHeal)
```

Parameters

`previousHp` long

The HP before resurrection.

`receivedHeal` ReceivedHealContext

The result of the heal applied during resurrection.

Properties

AbsAmount

The net HP gained during resurrection (NewValue - PreviousValue).

```
public long AbsAmount { get; }
```

Property Value

long

Entity

The entity that was resurrected. Same as Target in this case.

```
public EntityCore Entity { get; }
```

Property Value

EntityCore

Instigator

The specific thing that caused this resurrection — propagated from the originating ReceivedHealContext. May be `null` if unspecified.

```
public IEffectInstigator Instigator { get; }
```

Property Value

IEffectInstigator

NewValue

The HP the entity was resurrected with (after all heal modifiers), as an absolute value. Computed as PreviousValue + ReceivedHeal.HealAmount.NetAmount.

```
public long NewValue { get; }
```

Property Value

long

PreviousValue

The HP the entity had before resurrection (typically at or below the death threshold).

```
public long PreviousValue { get; }
```

Property Value

long

ReceivedHeal

The full result of the heal performed during resurrection, including amounts, source and healer.

```
public ReceivedHealContext ReceivedHeal { get; }
```

Property Value

ReceivedHealContext

Target

The entity that was resurrected. Convenience accessor for ReceivedHeal.Target.

```
public EntityCore Target { get; }
```

Property Value

EntityCore

Namespace ElectricDrill.AstraHealth.Exceptions

Classes

DeadEntityException

Exception thrown when attempting to perform health-modifying operations on a dead entity. This exception indicates a programming error where the caller attempted to heal, damage, or otherwise modify the health of an entity that has already died.

Class DeadEntityException

Namespace: ElectricDrill.AstraHealth.Exceptions

Assembly: com.electricdrill.astra-health.Runtime.dll

Exception thrown when attempting to perform health-modifying operations on a dead entity. This exception indicates a programming error where the caller attempted to heal, damage, or otherwise modify the health of an entity that has already died.

```
public class DeadEntityException : InvalidOperationException
```

Inheritance

object ← Exception ← SystemException ← InvalidOperationException ← DeadEntityException

Constructors

DeadEntityException(string, long, long, string)

Initializes a new instance of the DeadEntityException class.

```
public DeadEntityException(string entityName, long currentHp, long deathThreshold,
string attemptedOperation)
```

Parameters

`entityName` string

The name of the dead entity.

`currentHp` long

The current health points of the entity.

`deathThreshold` long

The death threshold value.

`attemptedOperation` string

The operation that was attempted.

Properties

AttemptedOperation

Gets the operation that was attempted on the dead entity.

```
public string AttemptedOperation { get; }
```

Property Value

string

CurrentHp

Gets the current health points of the dead entity.

```
public long CurrentHp { get; }
```

Property Value

long

DeathThreshold

Gets the death threshold of the entity (health value at or below which the entity is considered dead).

```
public long DeathThreshold { get; }
```

Property Value

long

EntityName

Gets the name of the entity that was dead when the operation was attempted.

```
public string EntityName { get; }
```

Property Value

string

Namespace ElectricDrill.AstraHealth.Experience

Classes

ExpCollectionStrategySO

Abstract base class for experience collection strategies. Implements the Template Method pattern to define when and how experience should be collected. Derive from this class to create custom experience collection logic.

ExpCollector

Collects experience when this entity contributes to enemy kills. Subscribes automatically to the global EntityDiedGameEvent configured in IAstraHealthConfig, so no manual event wiring is required. Uses a configurable ExpCollectionStrategySO to determine when and how experience is collected. If no custom strategy is assigned, falls back to the default strategy from IAstraHealthConfig.

Class ExpCollectionStrategySO

Namespace: ElectricDrill.AstraHealth.Experience

Assembly: com.electricdrill.astra-health.Runtime.dll

Abstract base class for experience collection strategies. Implements the Template Method pattern to define when and how experience should be collected. Derive from this class to create custom experience collection logic.

```
public abstract class ExpCollectionStrategySO : ScriptableObject, ITaggable
```

Inheritance

object ← Object ← ScriptableObject ← ExpCollectionStrategySO

Implements

ITaggable

Derived

DamageSourceKillExpStrategySO, DirectKillExpStrategySO, FirstMatchExpStrategySO

Properties

Tags

The tags associated with this object.

```
public GameTagSet Tags { get; }
```

Property Value

GameTagSet

Methods

CalculateExpAmount(IExpSource, float)

Calculates the final experience amount to collect based on the source and multiplier. Override this method to customize experience calculation logic.

```
protected virtual long CalculateExpAmount(IExpSource expSource, float multiplier)
```

Parameters

`expSource` IExpSource

The experience source providing the base amount.

`multiplier` float

The multiplier to apply to the base experience.

Returns

long

The final experience amount to add.

CollectExp(EntityCore, IExpSource, float)

Performs the actual experience collection: calculates the amount, adds it to the collector's level, and marks the source as harvested. Override this method to customize the entire collection process (e.g., for party XP sharing).

```
protected virtual bool CollectExp(EntityCore collector, IExpSource expSource,
float multiplier)
```

Parameters

`collector` EntityCore

The entity collecting the experience.

`expSource` IExpSource

The experience source to collect from.

`multiplier` float

The multiplier to apply to the base experience.

Returns

bool

True if experience was collected, false otherwise.

TryCollectExp(EntityCore, EntityCore, DamageResolutionContext)

Attempts to collect experience from a victim entity and add it to the collector's level. This is the main public interface for experience collection. Handles validation, extraction of IExpSource, calculation, adding to EntityLevel, and marking as harvested.

```
public virtual bool TryCollectExp(EntityCore collector, EntityCore victim,
DamageResolutionContext dmgResolutionContext)
```

Parameters

`collector` EntityCore

The entity attempting to collect experience.

`victim` EntityCore

The entity that died and may provide experience.

`dmgResolutionContext` DamageResolutionContext

The damage resolution that caused the death.

Returns

bool

True if experience was successfully collected, false otherwise.

TryGetExpSource(EntityCore, out IExpSource)

Attempts to extract an IExpSource from the victim's EntityCore. Override this method to customize how the IExpSource is retrieved.

```
protected virtual bool TryGetExpSource(EntityCore victim, out IExpSource expSource)
```

Parameters

`victim` EntityCore

The entity core to extract the IExpSource from.

`expSource` IExpSource

The extracted IExpSource, or null if not found.

Returns

bool

True if an IExpSource was found, false otherwise.

Validate(EntityCore, EntityCore, DamageResolutionContext, out float)

Override this method to implement the specific validation logic for the strategy.

```
protected abstract bool Validate(EntityCore collector, EntityCore victim,
DamageResolutionContext dmgResolutionContext, out float multiplier)
```

Parameters

`collector` EntityCore

The entity attempting to collect experience.

`victim` EntityCore

The entity that died and may provide experience.

`dmgResolutionContext` DamageResolutionContext

The damage resolution that caused the death.

`multiplier` float

Output multiplier to apply to the base experience (1.0 = full exp).

Returns

bool

True if the strategy's conditions are met, false otherwise.

Class ExpCollector

Namespace: ElectricDrill.AstraHealth.Experience

Assembly: com.electricdrill.astra-health.Runtime.dll

Collects experience when this entity contributes to enemy kills. Subscribes automatically to the global EntityDiedGameEvent configured in IAstraHealthConfig, so no manual event wiring is required. Uses a configurable ExpCollectionStrategySO to determine when and how experience is collected. If no custom strategy is assigned, falls back to the default strategy from IAstraHealthConfig.

```
[RequireComponent(typeof(EntityCore))]
public class ExpCollector : MonoBehaviour
```

Inheritance

object ← Object ← Component ← Behaviour ← MonoBehaviour ← ExpCollector

Properties

ActiveStrategy

Gets the active experience collection strategy. Returns the custom strategy if assigned, otherwise the default from configuration.

```
public ExpCollectionStrategySO ActiveStrategy { get; }
```

Property Value

ExpCollectionStrategySO

Namespace ElectricDrill.AstraHealth.Experience.Strategies

Classes

DamageSourceKillExpStrategySO

Experience collection strategy that grants experience when the damage source matches a configured source. Useful for environmental damage, traps, or any other specific damage source scenarios. For example: create an "Environmental" DamageSource and assign it to grant XP when enemies die from fall damage.

DirectKillExpStrategySO

Experience collection strategy that grants experience when the collector is the dealer of the killing blow. This is the default behavior - only the entity that dealt the final damage receives experience.

FirstMatchExpStrategySO

Composite experience collection strategy that evaluates a list of strategies in order. Delegates to the first strategy that successfully collects experience. Useful for implementing fallback behavior (e.g., try direct kill first, then environmental).

Class DamageSourceKillExpStrategySO

Namespace: ElectricDrill.AstraHealth.Experience.Strategies

Assembly: com.electricdrill.astra-health.Runtime.dll

Experience collection strategy that grants experience when the damage source matches a configured source. Useful for environmental damage, traps, or any other specific damage source scenarios. For example: create an "Environmental" DamageSource and assign it to grant XP when enemies die from fall damage.

```
public class DamageSourceKillExpStrategySO : ExpCollectionStrategySO, ITaggable
```

Inheritance

object ← Object ← ScriptableObject ← ExpCollectionStrategySO ← DamageSourceKillExpStrategySO

Implements

ITaggable

Inherited Members

ExpCollectionStrategySO.Tags ,
ExpCollectionStrategySO.TryCollectExp(EntityCore, EntityCore, DamageResolutionContext) ,
ExpCollectionStrategySO.CollectExp(EntityCore, IExpSource, float) ,
ExpCollectionStrategySO.CalculateExpAmount(IExpSource, float) ,
ExpCollectionStrategySO.TryGetExpSource(EntityCore, out IExpSource)

Methods

Validate(EntityCore, EntityCore, DamageResolutionContext, out float)

Override this method to implement the specific validation logic for the strategy.

```
protected override bool Validate(EntityCore collector, EntityCore victim,
DamageResolutionContext dmgResolutionContext, out float multiplier)
```

Parameters

`collector` EntityCore

The entity attempting to collect experience.

`victim` EntityCore

The entity that died and may provide experience.

`dmgResolutionContext` DamageResolutionContext

The damage resolution that caused the death.

`multiplier` float

Output multiplier to apply to the base experience (1.0 = full exp).

Returns

bool

True if the strategy's conditions are met, false otherwise.

Class DirectKillExpStrategySO

Namespace: ElectricDrill.AstraHealth.Experience.Strategies

Assembly: com.electricdrill.astra-health.Runtime.dll

Experience collection strategy that grants experience when the collector is the dealer of the killing blow. This is the default behavior - only the entity that dealt the final damage receives experience.

```
public class DirectKillExpStrategySO : ExpCollectionStrategySO, ITaggable
```

Inheritance

object ← Object ← ScriptableObject ← ExpCollectionStrategySO ← DirectKillExpStrategySO

Implements

ITaggable

Inherited Members

ExpCollectionStrategySO.Tags ,
ExpCollectionStrategySO.TryCollectExp(EntityCore, EntityCore, DamageResolutionContext) ,
ExpCollectionStrategySO.CollectExp(EntityCore, IExpSource, float) ,
ExpCollectionStrategySO.CalculateExpAmount(IExpSource, float) ,
ExpCollectionStrategySO.TryGetExpSource(EntityCore, out IExpSource)

Remarks

Kill-credit attribution is Experimental — this API may still change. "Dealer" is resolved through KillCredit Attribution: under the default ElectricDrill.AstraRpgFramework.Ownership.EntityAttribution.Direct this is the literal damage performer; under ElectricDrill.AstraRpgFramework.Ownership.EntityAttribution. Owner/ElectricDrill.AstraRpgFramework.Ownership.EntityAttribution.Root it is the performer's resolved owner (e.g. a ship credited for a kill dealt by its weapon).

Methods

Validate(EntityCore, EntityCore, DamageResolutionContext, out float)

Override this method to implement the specific validation logic for the strategy.

```
protected override bool Validate(EntityCore collector, EntityCore victim,
DamageResolutionContext dmgResolutionContext, out float multiplier)
```

Parameters

`collector` EntityCore

The entity attempting to collect experience.

`victim` EntityCore

The entity that died and may provide experience.

`dmgResolutionContext` DamageResolutionContext

The damage resolution that caused the death.

`multiplier` float

Output multiplier to apply to the base experience (1.0 = full exp).

Returns

bool

True if the strategy's conditions are met, false otherwise.

Class FirstMatchExpStrategySO

Namespace: ElectricDrill.AstraHealth.Experience.Strategies

Assembly: com.electricdrill.astra-health.Runtime.dll

Composite experience collection strategy that evaluates a list of strategies in order. Delegates to the first strategy that successfully collects experience. Useful for implementing fallback behavior (e.g., try direct kill first, then environmental).

```
public class FirstMatchExpStrategySO : ExpCollectionStrategySO, ITaggable
```

Inheritance

object ← Object ← ScriptableObject ← ExpCollectionStrategySO ← FirstMatchExpStrategySO

Implements

ITaggable

Inherited Members

ExpCollectionStrategySO.Tags , ExpCollectionStrategySO.CollectExp(EntityCore, IExpSource, float) , ExpCollectionStrategySO.CalculateExpAmount(IExpSource, float) , ExpCollectionStrategySO.TryGetExpSource(EntityCore, out IExpSource)

Methods

TryCollectExp(EntityCore, EntityCore, DamageResolutionContext)

Override TryCollectExp to delegate to the first matching sub-strategy. The winning strategy handles the complete collection workflow.

```
public override bool TryCollectExp(EntityCore collector, EntityCore victim,
DamageResolutionContext dmgResolutionContext)
```

Parameters

`collector` EntityCore

`victim` EntityCore

`dmgResolutionContext` DamageResolutionContext

Returns

bool

Validate(EntityCore, EntityCore, DamageResolutionContext, out float)

Not used in FirstMatchExpStrategy since TryCollectExp is overridden.

```
protected override bool Validate(EntityCore collector, EntityCore victim,
DamageResolutionContext dmgResolutionContext, out float multiplier)
```

Parameters

`collector` EntityCore

`victim` EntityCore

`dmgResolutionContext` DamageResolutionContext

`multiplier` float

Returns

bool

Namespace ElectricDrill.AstraHealth.GameActions.Actions

Classes

CounterDamageGameActionSO<TContext>

Abstract base game action that reacts to a resolved damage event by inflicting damage in return. Works with any context implementing ElectricDrill.AstraRpgFramework.Contexts.IHasTarget, ElectricDrill.AstraRpgFramework.Contexts.IHasPerformer and IHasAmount: compatible with DamageResolutionContext, EntityDiedContext, and others.

This class implements ElectricDrill.AstraRpgFramework.Contexts.IEffectInstigator so it can be passed as the instigator of the counter-damage it produces. Listeners can then check `ctx.Instigator == thisActionSO` to identify and filter self-triggered cycles.

DealDamageGameActionSO<TContext>

Abstract base game action that inflicts configured damage on the entity carried by the incoming context. Works with any context implementing ElectricDrill.AstraRpgFramework.Contexts.IHasEntity: the context entity is the damage target by default, while the dealer is resolved from the embedded PreDamageContextConfig (`None` → system damage, any context role → self-inflicted damage from that same entity).

Unlike CounterDamageGameActionSO<TContext> — which *reacts* to a resolved damage event — this action is meant to be a deliberate, first-order damage application (e.g. a periodic tick, an on-death explosion, an ability payload). By default the produced PreDamageContext is reactable so downstream reactive systems can respond to it; disable ElectricDrill.AstraHealth.GameActions.Actions.DealDamageGameActionSO<TContext>._isReactable when the damage is itself a secondary effect that should not spawn further reactions.

This class implements ElectricDrill.AstraRpgFramework.Contexts.IEffectInstigator so it can be passed as the instigator of the damage it produces. Listeners can then check `ctx.Instigator == thisActionSO` to identify and filter self-triggered cycles.

When `TContext` also implements IHasAmount, the carried amount is forwarded to the config so DamageFromContext keeps working; otherwise that mode falls back to `0` with a warning.

ResurrectEntityGameActionSO<TContext>

Base generic game action for resurrecting entities with EntityHealth components. Can be configured to resurrect with either a percentage of max HP or a flat HP amount. Uses the default resurrection source from config and no healer entity (system action).

Connect to any `GameEventListener` whose payload implements ElectricDrill.AstraRpgFramework.Contexts.IHasEntity, such as `EntityDiedGameEventListener`, `DamageResolutionGameEventListener`, or directly from an `EntityHealth` reference.

SetEntityOverrideOnDeathGameActionSO<TContext>

Base generic game action that assigns (or clears) the OverrideOnDeathGameAction field. Useful for one-shot mechanics such as "resurrect on first death": configure a CompositeAction that resurrects the entity and then executes this action with ElectricDrill.AstraHealth.GameActions.Actions.SetEntityOverrideOnDeathGameActionSO<TContext>._newOverrideGameAction set to null, so the override is cleared and subsequent deaths use the default config action.

Connect to any `GameEventListener` whose payload implements ElectricDrill.AstraRpgFramework.Contexts.IHasEntity, such as `EntityDiedGameEventListener`, `DamageResolutionGameEventListener`, or directly from an `EntityHealth` reference.

SetEntityOverrideOnResurrectionGameActionSO<TContext>

Base generic game action that assigns (or clears) the OverrideOnResurrectionGameAction field.

Connect to any `GameEventListener` whose payload implements ElectricDrill.AstraRpgFramework.Contexts.IHasEntity, such as `EntityDiedGameEventListener`, `EntityResurrectedGameEventListener`, or directly from an `EntityHealth` reference.

Class CounterDamageGameActionSO<TContext>

Namespace: ElectricDrill.AstraHealth.GameActions.Actions

Assembly: com.electricdrill.astra-health.Runtime.dll

Abstract base game action that reacts to a resolved damage event by inflicting damage in return. Works with any context implementing ElectricDrill.AstraRpgFramework.Contexts.IHasTarget, ElectricDrill.AstraRpgFramework.Contexts.IHasPerformer and IHasAmount: compatible with DamageResolutionContext, EntityDiedContext, and others.

This class implements ElectricDrill.AstraRpgFramework.Contexts.IEffectInstigator so it can be passed as the instigator of the counter-damage it produces. Listeners can then check `ctx.Instigator == thisActionSO` to identify and filter self-triggered cycles.

```
public abstract class CounterDamageGameActionSO<TContext> : GameAction<TContext>,
IExecutable<TContext>, ITaggable, IEffectInstigator where TContext : IHasTarget,
IHasPerformer, IHasInstigator, IHasAmount, IReactable
```

Type Parameters

`TContext`

Context type implementing ElectricDrill.AstraRpgFramework.Contexts.IHasTarget, ElectricDrill.AstraRpgFramework.Contexts.IHasPerformer, ElectricDrill.AstraRpgFramework.Contexts.IHasInstigator and IHasAmount.

Inheritance

object ← Object ← ScriptableObject ← GameActionBase ← GameAction<TContext> ← CounterDamageGameActionSO<TContext>

Implements

IExecutable<TContext>, ITaggable, IEffectInstigator

Derived

CounterDamageOnDamageGameActionSO, CounterDamageOnDeathGameActionSO, CounterDamageOnPreDamageGameActionSO

Inherited Members

GameAction<TContext>.Tags , GameAction<TContext>.DisplayName , GameAction<TContext>.ExecuteWithOwnerAsync(TContext, MonoBehaviour, CancellationToken) , GameAction<TContext>.ExecuteAsyncForUnityEvent(TContext) , GameAction<TContext>.OnOperationCanceled() , GameAction<TContext>.RunFireAndForget(TContext, MonoBehaviour)

Methods

ExecuteAsync(TContext, CancellationToken)

Executes the action asynchronously with the provided context. Override this method to implement your action logic.

```
public override Awaitable ExecuteAsync(TContext context, CancellationToken cancellationToken
= default)
```

Parameters

`context` TContext

The context object containing information needed for execution.

`cancellationToken` CancellationToken

Token to cancel the operation. MonoBehaviour owners can use their "destroyCancellationToken" to automatically cancel when destroyed.

Returns

Awaitable

An Awaitable that completes when the action finishes.

Class DealDamageGameActionSO<TContext>

Namespace: ElectricDrill.AstraHealth.GameActions.Actions

Assembly: com.electricdrill.astra-health.Runtime.dll

Abstract base game action that inflicts configured damage on the entity carried by the incoming context. Works with any context implementing ElectricDrill.AstraRpgFramework.Contexts.IHasEntity: the context entity is the damage target by default, while the dealer is resolved from the embedded PreDamage ContextConfig (`None` → system damage, any context role → self-inflicted damage from that same entity).

Unlike CounterDamageGameActionSO<TContext> — which *reacts* to a resolved damage event — this action is meant to be a deliberate, first-order damage application (e.g. a periodic tick, an on-death explosion, an ability payload). By default the produced PreDamageContext is reactable so downstream reactive systems can respond to it; disable ElectricDrill.AstraHealth.GameActions.Actions.DealDamageGameActionSO<TContext>._isReactable when the damage is itself a secondary effect that should not spawn further reactions.

This class implements ElectricDrill.AstraRpgFramework.Contexts.IEffectInstigator so it can be passed as the instigator of the damage it produces. Listeners can then check `ctx.Instigator == thisActionSO` to identify and filter self-triggered cycles.

When `TContext` also implements IHasAmount, the carried amount is forwarded to the config so DamageFromContext keeps working; otherwise that mode falls back to `0` with a warning.

```
public abstract class DealDamageGameActionSO<TContext> : GameAction<TContext>,
IExecutable<TContext>, ITaggable, IEffectInstigator where TContext : IHasEntity
```

Type Parameters

`TContext`

Context type implementing ElectricDrill.AstraRpgFramework.Contexts.IHasEntity.

Inheritance

object ← Object ← ScriptableObject ← GameActionBase ← GameAction<TContext> ← DealDamageGameActionSO<TContext>

Implements

IExecutable<TContext>, ITaggable, IEffectInstigator

Derived

DealDamageIHasEntityGameActionSO

Inherited Members
GameAction<TContext>.Tags , GameAction<TContext>.DisplayName , GameAction<TContext>.ExecuteWithOwnerAsync(TContext, MonoBehaviour, CancellationToken) , GameAction<TContext>.ExecuteAsyncForUnityEvent(TContext) , GameAction<TContext>.OnOperationCanceled() , GameAction<TContext>.RunFireAndForget(TContext, MonoBehaviour)

Methods

ExecuteAsync(TContext, CancellationToken)

Executes the action asynchronously with the provided context. Override this method to implement your action logic.

```
public override Awaitable ExecuteAsync(TContext context, CancellationToken cancellationToken
= default)
```

Parameters

`context` TContext

The context object containing information needed for execution.

`cancellationToken` CancellationToken

Token to cancel the operation. MonoBehaviour owners can use their "destroyCancellationToken" to automatically cancel when destroyed.

Returns

Awaitable

An Awaitable that completes when the action finishes.

ResolveDealer(TContext)

Resolves the entity attributed as the damage dealer. Returns `null` (system damage) when the config selects None; otherwise returns the context entity itself (self-inflicted damage).

```
protected virtual EntityCore ResolveDealer(TContext context)
```

Parameters

`context` TContext

Returns

EntityCore

ResolveTarget(TContext)

Resolves the entity that will receive the damage. Defaults to the context entity.

```
protected virtual EntityCore ResolveTarget(TContext context)
```

Parameters

`context` TContext

Returns

EntityCore

Class ResurrectEntityGameActionSO<TContext>

Namespace: ElectricDrill.AstraHealth.GameActions.Actions

Assembly: com.electricdrill.astra-health.Runtime.dll

Base generic game action for resurrecting entities with EntityHealth components. Can be configured to resurrect with either a percentage of max HP or a flat HP amount. Uses the default resurrection source from config and no healer entity (system action).

Connect to any `GameEventListener` whose payload implements ElectricDrill.AstraRpgFramework.Contexts.IHasEntity, such as `EntityDiedGameEventListener`, `DamageResolutionGameEventListener`, or directly from an `EntityHealth` reference.

```
public abstract class ResurrectEntityGameActionSO<TContext> : GameAction<TContext>,
IExecutable<TContext>, ITaggable where TContext : IHasEntity
```

Type Parameters

`TContext`

The context type that must implement ElectricDrill.AstraRpgFramework.Contexts.IHasEntity.

Inheritance

object ← Object ← ScriptableObject ← GameActionBase ← GameAction<TContext> ← ResurrectEntityGameActionSO<TContext>

Implements

IExecutable<TContext>, ITaggable

Derived

ResurrectEntityIHasEntityGameActionSO

Inherited Members

GameAction<TContext>.Tags , GameAction<TContext>.DisplayName , GameAction<TContext>.ExecuteWithOwnerAsync(TContext, MonoBehaviour, CancellationToken) , GameAction<TContext>.ExecuteAsyncForUnityEvent(TContext) , GameAction<TContext>.OnOperationCanceled() , GameAction<TContext>.RunFireAndForget(TContext, MonoBehaviour)

Fields

_flatHpToRecover

```
[SerializeField]
[Tooltip("Flat amount of HP to restore on resurrection (used when 'Use Percentage HP'
is false)")]
protected long _flatHpToRecover
```

Field Value

long

_percentageHpToRecover

```
[SerializeField]
[Tooltip("The percentage of max HP to restore (0-100)")]
[Range(0, 100)]
protected int _percentageHpToRecover
```

Field Value

int

_usePercentageHp

```
[SerializeField]
[Tooltip("If true, resurrect with a percentage of max HP. If false, use flat HP amount.")]
protected bool _usePercentageHp
```

Field Value

bool

Methods

ExecuteAsync(TContext, CancellationToken)

Executes the resurrection logic asynchronously. Attempts to get EntityHealth from the context and resurrects with configured HP settings.

```
public override Awaitable ExecuteAsync(TContext context, CancellationToken cancellationToken
= default)
```

Parameters

`context` TContext

The context, expected to be or carry an entity that has EntityHealth.

`cancellationToken` CancellationToken

Token to cancel the operation.

Returns

Awaitable

An Awaitable that completes when resurrection finishes.

Class SetEntityOverrideOnDeathGameActionSO<TContext>

Namespace: ElectricDrill.AstraHealth.GameActions.Actions

Assembly: com.electricdrill.astra-health.Runtime.dll

Base generic game action that assigns (or clears) the OverrideOnDeathGameAction field. Useful for one-shot mechanics such as "resurrect on first death": configure a CompositeAction that resurrects the entity and then executes this action with ElectricDrill.AstraHealth.GameActions.Actions.SetEntityOverrideOnDeathGameActionSO<TContext>._newOverrideGameAction set to null, so the override is cleared and subsequent deaths use the default config action.

Connect to any `GameEventListener` whose payload implements ElectricDrill.AstraRpgFramework.Contexts.IHasEntity, such as `EntityDiedGameEventListener`, `DamageResolutionGameEventListener`, or directly from an `EntityHealth` reference.

```
public abstract class SetEntityOverrideOnDeathGameActionSO<TContext> : GameAction<TContext>, IExecutable<TContext>, ITaggable where TContext : IHasEntity
```

Type Parameters

`TContext`

The context type that must implement ElectricDrill.AstraRpgFramework.Contexts.IHasEntity.

Inheritance

object ← Object ← ScriptableObject ← GameActionBase ← GameAction<TContext> ← SetEntityOverrideOnDeathGameActionSO<TContext>

Implements

IExecutable<TContext>, ITaggable

Derived

SetEntityOverrideOnDeathIHasEntityGameActionSO

Inherited Members

GameAction<TContext>.Tags , GameAction<TContext>.DisplayName , GameAction<TContext>.ExecuteWithOwnerAsync(TContext, MonoBehaviour, CancellationToken) , GameAction<TContext>.ExecuteAsyncForUnityEvent(TContext) , GameAction<TContext>.OnOperationCanceled() , GameAction<TContext>.RunFireAndForget(TContext, MonoBehaviour)

Methods

ExecuteAsync(TContext, CancellationToken)

Executes the action asynchronously with the provided context. Override this method to implement your action logic.

```
public override Awaitable ExecuteAsync(TContext context, CancellationToken cancellationToken
= default)
```

Parameters

`context` TContext

The context object containing information needed for execution.

`cancellationToken` CancellationToken

Token to cancel the operation. MonoBehaviour owners can use their "destroyCancellationToken" to automatically cancel when destroyed.

Returns

Awaitable

An Awaitable that completes when the action finishes.

Class SetEntityOverrideOnResurrectionGameActionSO<TContext>

Namespace: ElectricDrill.AstraHealth.GameActions.Actions

Assembly: com.electricdrill.astra-health.Runtime.dll

Base generic game action that assigns (or clears) the OverrideOnResurrectionGameAction field.

Connect to any `GameEventListener` whose payload implements ElectricDrill.AstraRpgFramework.Contexts.IHasEntity, such as `EntityDiedGameEventListener`, `EntityResurrectedGameEventListener`, or directly from an `EntityHealth` reference.

```
public abstract class SetEntityOverrideOnResurrectionGameActionSO<TContext> :
GameAction<TContext>, IExecutable<TContext>, ITaggable where TContext : IHasEntity
```

Type Parameters

`TContext`

The context type that must implement ElectricDrill.AstraRpgFramework.Contexts.IHasEntity.

Inheritance

object ← Object ← ScriptableObject ← GameActionBase ← GameAction<TContext> ← SetEntityOverrideOnResurrectionGameActionSO<TContext>

Implements

IExecutable<TContext>, ITaggable

Derived

SetEntityOverrideOnResurrectionIHasEntityGameActionSO

Inherited Members

GameAction<TContext>.Tags , GameAction<TContext>.DisplayName , GameAction<TContext>.ExecuteWithOwnerAsync(TContext, MonoBehaviour, CancellationToken) , GameAction<TContext>.ExecuteAsyncForUnityEvent(TContext) , GameAction<TContext>.OnOperationCanceled() , GameAction<TContext>.RunFireAndForget(TContext, MonoBehaviour)

Methods

ExecuteAsync(TContext, CancellationToken)

Executes the action asynchronously with the provided context. Override this method to implement your action logic.

```
public override Awaitable ExecuteAsync(TContext context, CancellationToken cancellationToken
= default)
```

Parameters

`context` TContext

The context object containing information needed for execution.

`cancellationToken` CancellationToken

Token to cancel the operation. MonoBehaviour owners can use their "destroyCancellationToken" to automatically cancel when destroyed.

Returns

Awaitable

An Awaitable that completes when the action finishes.

Namespace ElectricDrill.AstraHealth.GameActions.Actions.WithDamageResolutionContext

Classes

CounterDamageOnDamageGameActionSO

Reacts to a DamageResolutionContext event by inflicting configured damage. Connect to a `DamageResolutionGameEventListener`'s UnityEvent response and select `ExecuteAsyncForUnityEvent(DamageResolutionContext)` as the dynamic invocation target. Create instances via Assets → Create → Astra Health/Game Actions/Context: DamageResolutionContext/Counter Damage.

Class CounterDamageOnDamageGameActionSO

Namespace: ElectricDrill.AstraHealth.GameActions.Actions.WithDamageResolutionContext

Assembly: com.electricdrill.astra-health.Runtime.dll

Reacts to a DamageResolutionContext event by inflicting configured damage. Connect to a `DamageResolutionGameEventListener`'s UnityEvent response and select `ExecuteAsyncForUnityEvent(DamageResolutionContext)` as the dynamic invocation target. Create instances via Assets → Create → Astra Health/Game Actions/Context: DamageResolutionContext/Counter Damage.

```
[CreateAssetMenu(fileName = "Counter Damage Game Action", menuName = "Astra Health/Game
Actions/Context: DamageResolutionContext/Counter Damage")]
public class CounterDamageOnDamageGameActionSO :
CounterDamageGameActionSO<DamageResolutionContext>, IExecutable<DamageResolutionContext>,
ITaggable, IEffectInstigator
```

Inheritance

object ← Object ← ScriptableObject ← GameActionBase ← GameAction<DamageResolutionContext> ← CounterDamageGameActionSO<DamageResolutionContext> ← CounterDamageOnDamageGameActionSO

Implements

IExecutable<DamageResolutionContext>, ITaggable, IEffectInstigator

Inherited Members

CounterDamageGameActionSO<DamageResolutionContext>.ExecuteAsync(DamageResolutionContext, CancellationToken) ,
GameAction<DamageResolutionContext>.Tags ,
GameAction<DamageResolutionContext>.DisplayName ,
GameAction<DamageResolutionContext>.ExecuteWithOwnerAsync(DamageResolutionContext, MonoBehaviour, CancellationToken) ,
GameAction<DamageResolutionContext>.ExecuteAsyncForUnityEvent(DamageResolutionContext) ,
GameAction<DamageResolutionContext>.OnOperationCanceled() ,
GameAction<DamageResolutionContext>.RunFireAndForget(DamageResolutionContext, MonoBehaviour)

Namespace ElectricDrill.AstraHealth.GameActions.Actions.WithEntityDiedContext

Classes

CounterDamageOnDeathGameActionSO

Reacts to an EntityDiedContext event by inflicting configured damage. Connect to an `EntityDiedGameEventListener`'s UnityEvent response and select `ExecuteAsyncForUnityEvent(EntityDiedContext)` as the dynamic invocation target. Create instances via Assets → Create → Astra Health/Game Actions/Context: EntityDiedContext/Counter Damage.

Class CounterDamageOnDeathGameActionSO

Namespace: ElectricDrill.AstraHealth.GameActions.Actions.WithEntityDiedContext

Assembly: com.electricdrill.astra-health.Runtime.dll

Reacts to an EntityDiedContext event by inflicting configured damage. Connect to an `EntityDiedGameEventListener`'s UnityEvent response and select `ExecuteAsyncForUnityEvent(EntityDiedContext)` as the dynamic invocation target. Create instances via Assets → Create → Astra Health/Game Actions/Context: EntityDiedContext/Counter Damage.

```
[CreateAssetMenu(fileName = "Counter Damage On Death Game Action", menuName = "Astra
Health/Game Actions/Context: EntityDiedContext/Counter Damage")]
public class CounterDamageOnDeathGameActionSO :
CounterDamageGameActionSO<EntityDiedContext>, IExecutable<EntityDiedContext>,
ITaggable, IEffectInstigator
```

Inheritance

object ← Object ← ScriptableObject ← GameActionBase ← GameAction<EntityDiedContext> ← CounterDamageGameActionSO<EntityDiedContext> ← CounterDamageOnDeathGameActionSO

Implements

IExecutable<EntityDiedContext>, ITaggable, IEffectInstigator

Inherited Members

CounterDamageGameActionSO<EntityDiedContext>.ExecuteAsync(EntityDiedContext, CancellationToken) ,
GameAction<EntityDiedContext>.Tags , GameAction<EntityDiedContext>.DisplayName ,
GameAction<EntityDiedContext>.ExecuteWithOwnerAsync(EntityDiedContext, MonoBehaviour, CancellationToken) ,
GameAction<EntityDiedContext>.ExecuteAsyncForUnityEvent(EntityDiedContext) ,
GameAction<EntityDiedContext>.OnOperationCanceled() ,
GameAction<EntityDiedContext>.RunFireAndForget(EntityDiedContext, MonoBehaviour)

Namespace ElectricDrill.AstraHealth.GameActions.Actions.WithIHasEntity

Classes

DealDamageIHasEntityGameActionSO

Inflicts configured damage on the entity carried by any ElectricDrill.AstraRpgFramework.Contexts.IHasEntity payload. Provided as a ScriptableObject for easy assignment in the editor.

Connect to any `GameEventListener` whose payload implements ElectricDrill.AstraRpgFramework.Contexts.IHasEntity, such as `EntityDiedGameEventListener`, `EntityResurrectedGameEventListener`, or a `DamageResolutionGameEventListener`, and select `ExecuteAsyncForUnityEvent(IHasEntity)` as the dynamic invocation target.

The context entity is always the damage target. The embedded config's `Damage Dealer` role only distinguishes `None` (unattributed system damage) from self-inflicted damage dealt by that same entity; the `Damage Target` role is ignored. Create instances via Assets → Create → Astra Health/Game Actions/Context: Entity/Deal Damage.

EntityContextToDamageResolutionContextProjectionGameActionSO

Attempts to narrow an ElectricDrill.AstraRpgFramework.Contexts.IHasEntity payload to a more specific context type and, if the runtime type matches, forwards the original payload to the wrapped action. Use this when subscribing a broad entity-based event pipeline to actions that only make sense for richer payloads such as damage or death contexts.

EntityContextToEntityDiedContextProjectionGameActionSO

Attempts to narrow an ElectricDrill.AstraRpgFramework.Contexts.IHasEntity payload to a more specific context type and, if the runtime type matches, forwards the original payload to the wrapped action. Use this when subscribing a broad entity-based event pipeline to actions that only make sense for richer payloads such as damage or death contexts.

EntityContextToPreDamageContextProjectionGameActionSO

Attempts to narrow an ElectricDrill.AstraRpgFramework.Contexts.IHasEntity payload to a more specific context type and, if the runtime type matches, forwards the original payload to the wrapped action. Use this when subscribing a broad entity-based event pipeline to actions that only make sense for richer payloads such as damage or death contexts.

ResurrectEntityIHasEntityGameActionSO

A game action used to resurrect an entity. Can be configured to resurrect with either a percentage of max HP or a flat HP amount. Provided as a ScriptableObject for easy assignment in the editor.

Connect to any `GameEventListener` whose payload implements ElectricDrill.AstraRpgFramework.Contexts.IHasEntity, such as `EntityDiedGameEventListener`, `EntityResurrectedGameEventListener`, or directly from an `EntityHealth` reference.

Create instances via Assets → Create → Astra Health/Game Actions/Context: Entity/Resurrect.

SetEntityOverrideOnDeathIHasEntityGameActionSO

Assigns (or clears) OverrideOnDeathGameAction on the target entity. Provided as a ScriptableObject for easy assignment in the editor.

Connect to any `GameEventListener` whose payload implements ElectricDrill.AstraRpgFramework.Contexts.IHasEntity, such as `EntityDiedGameEventListener` or `DamageResolutionGameEventListener`.

Create instances via Assets → Create → Astra Health/Game Actions/Context: Entity/Set Override On Death.

SetEntityOverrideOnResurrectionIHasEntityGameActionSO

Assigns (or clears) OverrideOnResurrectionGameAction on the target entity. Provided as a ScriptableObject for easy assignment in the editor.

Connect to any `GameEventListener` whose payload implements ElectricDrill.AstraRpgFramework.Contexts.IHasEntity, such as `EntityResurrectedGameEventListener` or `EntityDiedGameEventListener`.

Create instances via Assets → Create → Astra Health/Game Actions/Context: Entity/Set Override On Resurrection.

Class DealDamageIHasEntityGameActionSO

Namespace: ElectricDrill.AstraHealth.GameActions.Actions.WithIHasEntity

Assembly: com.electricdrill.astra-health.Runtime.dll

Inflicts configured damage on the entity carried by any ElectricDrill.AstraRpgFramework.Contexts.IHas Entity payload. Provided as a ScriptableObject for easy assignment in the editor.

Connect to any `GameEventListener` whose payload implements ElectricDrill.AstraRpgFramework.Contexts. IHasEntity, such as `EntityDiedGameEventListener`, `EntityResurrectedGameEventListener`, or a `DamageResolutionGameEventListener`, and select `ExecuteAsyncForUnityEvent(IHasEntity)` as the dynamic invocation target.

The context entity is always the damage target. The embedded config's `Damage Dealer` role only distinguishes `None` (unattributed system damage) from self-inflicted damage dealt by that same entity; the `Damage Target` role is ignored. Create instances via Assets → Create → Astra Health/Game Actions/Context: Entity/Deal Damage.

```
[CreateAssetMenu(fileName = "Deal Damage Game Action", menuName = "Astra Health/Game
Actions/Context: Entity/Deal Damage")]
public class DealDamageIHasEntityGameActionSO : DealDamageGameActionSO<IHasEntity>,
IExecutable<IHasEntity>, ITaggable, IEffectInstigator
```

Inheritance

object ← Object ← ScriptableObject ← GameActionBase ← GameAction<IHasEntity> ← DealDamageGameActionSO<IHasEntity> ← DealDamageIHasEntityGameActionSO

Implements

IExecutable<IHasEntity>, ITaggable, IEffectInstigator

Inherited Members

DealDamageGameActionSO<IHasEntity>.ExecuteAsync(IHasEntity, CancellationToken) , DealDamageGameActionSO<IHasEntity>.ResolveTarget(IHasEntity) , DealDamageGameActionSO<IHasEntity>.ResolveDealer(IHasEntity) , GameAction<IHasEntity>.Tags , GameAction<IHasEntity>.DisplayName , GameAction<IHasEntity>.ExecuteWithOwnerAsync(IHasEntity, MonoBehaviour, CancellationToken) , GameAction<IHasEntity>.ExecuteAsyncForUnityEvent(IHasEntity) , GameAction<IHasEntity>.OnOperationCanceled() , GameAction<IHasEntity>.RunFireAndForget(IHasEntity, MonoBehaviour)

Class EntityContextToDamageResolutionContextProjectionGameActionSO

Namespace: ElectricDrill.AstraHealth.GameActions.Actions.WithIHasEntity

Assembly: com.electricdrill.astra-health.Runtime.dll

Attempts to narrow an ElectricDrill.AstraRpgFramework.Contexts.IHasEntity payload to a more specific context type and, if the runtime type matches, forwards the original payload to the wrapped action. Use this when subscribing a broad entity-based event pipeline to actions that only make sense for richer payloads such as damage or death contexts.

```
[CreateAssetMenu(fileName = "New Entity Context To DamageResolutionContext Projection
Action", menuName = "Astra Health/Game Actions/Context: Entity/Projections/→
DamageResolutionContext Projection")]
public sealed class EntityContextToDamageResolutionContextProjectionGameActionSO
: EntityContextCastProjectionGameAction<DamageResolutionContext>,
IExecutable<IHasEntity>, ITaggable
```

Inheritance

object ← Object ← ScriptableObject ← GameActionBase ← GameAction<IHasEntity> ← EntityContextCastProjectionGameAction<DamageResolutionContext> ← EntityContextToDamageResolutionContextProjectionGameActionSO

Implements

IExecutable<IHasEntity>, ITaggable

Inherited Members

EntityContextCastProjectionGameAction<DamageResolutionContext>.ExecuteAsync(IHasEntity, CancellationToken) , EntityContextCastProjectionGameAction<DamageResolutionContext>.ExecuteWithOwnerAsync(IHasEntity, MonoBehaviour, CancellationToken) , GameAction<IHasEntity>.Tags , GameAction<IHasEntity>.DisplayName , GameAction<IHasEntity>.ExecuteAsyncForUnityEvent(IHasEntity) , GameAction<IHasEntity>.RunFireAndForget(IHasEntity, MonoBehaviour)

Class EntityContextToEntityDiedContextProjectionGameActionSO

Namespace: ElectricDrill.AstraHealth.GameActions.Actions.WithIHasEntity

Assembly: com.electricdrill.astra-health.Runtime.dll

Attempts to narrow an ElectricDrill.AstraRpgFramework.Contexts.IHasEntity payload to a more specific context type and, if the runtime type matches, forwards the original payload to the wrapped action. Use this when subscribing a broad entity-based event pipeline to actions that only make sense for richer payloads such as damage or death contexts.

```
[CreateAssetMenu(fileName = "New Entity Context To EntityDiedContext Projection
Action", menuName = "Astra Health/Game Actions/Context: Entity/Projections/→
EntityDiedContext Projection")]
public sealed class EntityContextToEntityDiedContextProjectionGameActionSO :
EntityContextCastProjectionGameAction<EntityDiedContext>, IExecutable<IHasEntity>, ITaggable
```

Inheritance

object ← Object ← ScriptableObject ← GameActionBase ← GameAction<IHasEntity> ← EntityContextCastProjectionGameAction<EntityDiedContext> ← EntityContextToEntityDiedContextProjectionGameActionSO

Implements

IExecutable<IHasEntity>, ITaggable

Inherited Members

EntityContextCastProjectionGameAction<EntityDiedContext>.ExecuteAsync(IHasEntity, CancellationToken) ,
EntityContextCastProjectionGameAction<EntityDiedContext>.ExecuteWithOwnerAsync(IHasEntity, MonoBehaviour, CancellationToken) ,
GameAction<IHasEntity>.Tags , GameAction<IHasEntity>.DisplayName ,
GameAction<IHasEntity>.ExecuteAsyncForUnityEvent(IHasEntity) ,
GameAction<IHasEntity>.RunFireAndForget(IHasEntity, MonoBehaviour)

Class EntityContextToPreDamageContextProjectionGameActionSO

Namespace: ElectricDrill.AstraHealth.GameActions.Actions.WithIHasEntity

Assembly: com.electricdrill.astra-health.Runtime.dll

Attempts to narrow an ElectricDrill.AstraRpgFramework.Contexts.IHasEntity payload to a more specific context type and, if the runtime type matches, forwards the original payload to the wrapped action. Use this when subscribing a broad entity-based event pipeline to actions that only make sense for richer payloads such as damage or death contexts.

```
[CreateAssetMenu(fileName = "New Entity Context To PreDamageContext Projection
Action", menuName = "Astra Health/Game Actions/Context: Entity/Projections/→
PreDamageContext Projection")]
public sealed class EntityContextToPreDamageContextProjectionGameActionSO :
EntityContextCastProjectionGameAction<PreDamageContext>, IExecutable<IHasEntity>, ITaggable
```

Inheritance

object ← Object ← ScriptableObject ← GameActionBase ← GameAction<IHasEntity> ← EntityContextCastProjectionGameAction<PreDamageContext> ← EntityContextToPreDamageContextProjectionGameActionSO

Implements

IExecutable<IHasEntity>, ITaggable

Inherited Members

EntityContextCastProjectionGameAction<PreDamageContext>.ExecuteAsync(IHasEntity, CancellationToken) ,
EntityContextCastProjectionGameAction<PreDamageContext>.ExecuteWithOwnerAsync(IHasEntity, MonoBehaviour, CancellationToken) ,
GameAction<IHasEntity>.Tags , GameAction<IHasEntity>.DisplayName ,
GameAction<IHasEntity>.ExecuteAsyncForUnityEvent(IHasEntity) ,
GameAction<IHasEntity>.RunFireAndForget(IHasEntity, MonoBehaviour)

Class ResurrectEntityIHasEntityGameActionSO

Namespace: ElectricDrill.AstraHealth.GameActions.Actions.WithIHasEntity

Assembly: com.electricdrill.astra-health.Runtime.dll

A game action used to resurrect an entity. Can be configured to resurrect with either a percentage of max HP or a flat HP amount. Provided as a ScriptableObject for easy assignment in the editor.

Connect to any `GameEventListener` whose payload implements ElectricDrill.AstraRpgFramework.Contexts.IHasEntity, such as `EntityDiedGameEventListener`, `EntityResurrectedGameEventListener`, or directly from an `EntityHealth` reference.

Create instances via Assets → Create → Astra Health/Game Actions/Context: Entity/Resurrect.

```
[CreateAssetMenu(fileName = "Resurrect Game Action", menuName = "Astra Health/Game
Actions/Context: Entity/Resurrect")]
public class ResurrectEntityIHasEntityGameActionSO :
ResurrectEntityGameActionSO<IHasEntity>, IExecutable<IHasEntity>, ITaggable
```

Inheritance

object ← Object ← ScriptableObject ← GameActionBase ← GameAction<IHasEntity> ← ResurrectEntityGameActionSO<IHasEntity> ← ResurrectEntityIHasEntityGameActionSO

Implements

IExecutable<IHasEntity>, ITaggable

Inherited Members

ResurrectEntityGameActionSO<IHasEntity>._usePercentageHp , ResurrectEntityGameActionSO<IHasEntity>._percentageHpToRecover , ResurrectEntityGameActionSO<IHasEntity>._flatHpToRecover , ResurrectEntityGameActionSO<IHasEntity>.ExecuteAsync(IHasEntity, CancellationToken) , GameAction<IHasEntity>.Tags , GameAction<IHasEntity>.DisplayName , GameAction<IHasEntity>.ExecuteWithOwnerAsync(IHasEntity, MonoBehaviour, CancellationToken) , GameAction<IHasEntity>.ExecuteAsyncForUnityEvent(IHasEntity) , GameAction<IHasEntity>.OnOperationCanceled() , GameAction<IHasEntity>.RunFireAndForget(IHasEntity, MonoBehaviour)

Class SetEntityOverrideOnDeathIHasEntityGameActionSO

Namespace: ElectricDrill.AstraHealth.GameActions.Actions.WithIHasEntity

Assembly: com.electricdrill.astra-health.Runtime.dll

Assigns (or clears) OverrideOnDeathGameAction on the target entity. Provided as a ScriptableObject for easy assignment in the editor.

Connect to any `GameEventListener` whose payload implements ElectricDrill.AstraRpgFramework.Contexts.IHasEntity, such as `EntityDiedGameEventListener` or `DamageResolutionGameEventListener`.

Create instances via Assets → Create → Astra Health/Game Actions/Context: Entity/Set Override On Death.

```
[CreateAssetMenu(fileName = "Set Override On Death Game Action", menuName = "Astra
Health/Game Actions/Context: Entity/Set Override On Death")]
public class SetEntityOverrideOnDeathIHasEntityGameActionSO :
SetEntityOverrideOnDeathGameActionSO<IHasEntity>, IExecutable<IHasEntity>, ITaggable
```

Inheritance

object ← Object ← ScriptableObject ← GameActionBase ← GameAction<IHasEntity> ← SetEntityOverrideOnDeathGameActionSO<IHasEntity> ← SetEntityOverrideOnDeathIHasEntityGameActionSO

Implements

IExecutable<IHasEntity>, ITaggable

Inherited Members

SetEntityOverrideOnDeathGameActionSO<IHasEntity>.ExecuteAsync(IHasEntity, CancellationToken) , GameAction<IHasEntity>.Tags , GameAction<IHasEntity>.DisplayName , GameAction<IHasEntity>.ExecuteWithOwnerAsync(IHasEntity, MonoBehaviour, CancellationToken) , GameAction<IHasEntity>.ExecuteAsyncForUnityEvent(IHasEntity) , GameAction<IHasEntity>.OnOperationCanceled() , GameAction<IHasEntity>.RunFireAndForget(IHasEntity, MonoBehaviour)

Class SetEntityOverrideOnResurrectionIHasEntityGameActionSO

Namespace: ElectricDrill.AstraHealth.GameActions.Actions.WithIHasEntity

Assembly: com.electricdrill.astra-health.Runtime.dll

Assigns (or clears) OverrideOnResurrectionGameAction on the target entity. Provided as a ScriptableObject for easy assignment in the editor.

Connect to any `GameEventListener` whose payload implements ElectricDrill.AstraRpgFramework.Contexts.IHasEntity, such as `EntityResurrectedGameEventListener` or `EntityDiedGameEventListener`.

Create instances via Assets → Create → Astra Health/Game Actions/Context: Entity/Set Override On Resurrection.

```
[CreateAssetMenu(fileName = "Set Override On Resurrection Game Action", menuName = "Astra Health/Game Actions/Context: Entity/Set Override On Resurrection")]
public class SetEntityOverrideOnResurrectionIHasEntityGameActionSO : SetEntityOverrideOnResurrectionGameActionSO<IHasEntity>, IExecutable<IHasEntity>, ITaggable
```

Inheritance

object ← Object ← ScriptableObject ← GameActionBase ← GameAction<IHasEntity> ← SetEntityOverrideOnResurrectionGameActionSO<IHasEntity> ← SetEntityOverrideOnResurrectionIHasEntityGameActionSO

Implements

IExecutable<IHasEntity>, ITaggable

Inherited Members

SetEntityOverrideOnResurrectionGameActionSO<IHasEntity>.ExecuteAsync(IHasEntity, CancellationToken) , GameAction<IHasEntity>.Tags , GameAction<IHasEntity>.DisplayName , GameAction<IHasEntity>.ExecuteWithOwnerAsync(IHasEntity, MonoBehaviour, CancellationToken) , GameAction<IHasEntity>.ExecuteAsyncForUnityEvent(IHasEntity) , GameAction<IHasEntity>.OnOperationCanceled() , GameAction<IHasEntity>.RunFireAndForget(IHasEntity, MonoBehaviour)

Namespace ElectricDrill.AstraHealth.GameActions.Actions.WithPreDamageContext

Classes

CompositePreDamageContextGameActionSO

Executes a list of actions. Two authoring models are supported:

- **Steps** (ElectricDrill.AstraRpgFramework.GameActions.Actions.CompositeGameAction<TContext>._steps) — each ElectricDrill.AstraRpgFramework.GameActions.Actions.CompositeStep<TContext> chooses how it composes with its neighbours (await it, run it alongside the following steps, or fire it and forget it) and may carry a delay. This is the model new assets should use.
- **Legacy list** (ElectricDrill.AstraRpgFramework.GameActions.Actions.CompositeGameAction<TContext>._actions) — a plain sequential list. Kept so assets authored before steps existed keep deserializing and running unchanged.

Will gracefully terminate if the context is destroyed or cancellation is requested.

CounterDamageOnPreDamageGameActionSO

Reacts to a PreDamageContext event by inflicting configured damage. Connect to a `PreDamageContextGameEventListener`'s UnityEvent response and select `ExecuteAsyncForUnityEvent(PreDamageContext)` as the dynamic invocation target. Create instances via Assets → Create → Astra Health/Game Actions/Context: PreDamageContext/Counter Damage.

Class CompositePreDamageContextGameActionSO

Namespace: ElectricDrill.AstraHealth.GameActions.Actions.WithPreDamageContext

Assembly: com.electricdrill.astra-health.Runtime.dll

Executes a list of actions. Two authoring models are supported:

- **Steps** (ElectricDrill.AstraRpgFramework.GameActions.Actions.CompositeGameAction<TContext>._steps) — each ElectricDrill.AstraRpgFramework.GameActions.Actions.CompositeStep<TContext> chooses how it composes with its neighbours (await it, run it alongside the following steps, or fire it and forget it) and may carry a delay. This is the model new assets should use.
- **Legacy list** (ElectricDrill.AstraRpgFramework.GameActions.Actions.CompositeGameAction<TContext>._actions) — a plain sequential list. Kept so assets authored before steps existed keep deserializing and running unchanged.

Will gracefully terminate if the context is destroyed or cancellation is requested.

```
[CreateAssetMenu(fileName = "Composite PreDamageContext GameAction", menuName = "Astra
Health/Game Actions/Context: PreDamageContext/Composite")]
public sealed class CompositePreDamageContextGameActionSO :
CompositeGameAction<PreDamageContext>, IExecutable<PreDamageContext>, ITaggable
```

Inheritance

object ← Object ← ScriptableObject ← GameActionBase ← GameAction<PreDamageContext> ← CompositeGameAction<PreDamageContext> ← CompositePreDamageContextGameActionSO

Implements

IExecutable<PreDamageContext>, ITaggable

Inherited Members

CompositeGameAction<PreDamageContext>.ExecuteAsync(PreDamageContext, CancellationToken) , CompositeGameAction<PreDamageContext>.ExecuteWithOwnerAsync(PreDamageContext, MonoBehaviour, CancellationToken) , GameAction<PreDamageContext>.Tags , GameAction<PreDamageContext>.DisplayName , GameAction<PreDamageContext>.ExecuteAsyncForUnityEvent(PreDamageContext) , GameAction<PreDamageContext>.RunFireAndForget(PreDamageContext, MonoBehaviour)

Remarks

Precedence. If ElectricDrill.AstraRpgFramework.GameActions.Actions.CompositeGameAction<TContext>._actions is non-empty the legacy path runs and ElectricDrill.AstraRpgFramework.GameActions.Actions.CompositeGameAction<TContext>._steps is ignored (with a one-shot warning if both are populated). The inspector prevents reaching that ambiguous state, so the warning is a safety net rather than a normal flow. Otherwise the step path runs.

Class CounterDamageOnPreDamageGameActionSO

Namespace: ElectricDrill.AstraHealth.GameActions.Actions.WithPreDamageContext

Assembly: com.electricdrill.astra-health.Runtime.dll

Reacts to a PreDamageContext event by inflicting configured damage. Connect to a `PreDamageContextGameEventListener`'s UnityEvent response and select `ExecuteAsyncForUnityEvent(PreDamageContext)` as the dynamic invocation target. Create instances via Assets → Create → Astra Health/Game Actions/Context: PreDamageContext/Counter Damage.

```
[CreateAssetMenu(fileName = "Counter Damage On Pre Damage Game Action", menuName = "Astra Health/Game Actions/Context: PreDamageContext/Counter Damage")]
public class CounterDamageOnPreDamageGameActionSO : CounterDamageGameActionSO<PreDamageContext>, IExecutable<PreDamageContext>, ITaggable, IEffectInstigator
```

Inheritance

object ← Object ← ScriptableObject ← GameActionBase ← GameAction<PreDamageContext> ← CounterDamageGameActionSO<PreDamageContext> ← CounterDamageOnPreDamageGameActionSO

Implements

IExecutable<PreDamageContext>, ITaggable, IEffectInstigator

Inherited Members

CounterDamageGameActionSO<PreDamageContext>.ExecuteAsync(PreDamageContext, CancellationToken) ,
GameAction<PreDamageContext>.Tags , GameAction<PreDamageContext>.DisplayName ,
GameAction<PreDamageContext>.ExecuteWithOwnerAsync(PreDamageContext, MonoBehaviour, CancellationToken) ,
GameAction<PreDamageContext>.ExecuteAsyncForUnityEvent(PreDamageContext) ,
GameAction<PreDamageContext>.OnOperationCanceled() ,
GameAction<PreDamageContext>.RunFireAndForget(PreDamageContext, MonoBehaviour)

Namespace ElectricDrill.AstraHealth.Heal

Classes

HealAmountContext

Represents the different numeric stages of a heal amount as it is processed: the raw requested amount, the amount after applying a heal modifier, and the final net amount applied.

HealSourceSO

Defines a heal source asset that identifies how a heal was produced (e.g. spell, potion, resurrection, system, ...). Create instances via Assets -> Create -> Astra Health / Heal Source.

LifestealAmountSelector

Selects which damage amount will be used as the basis for lifesteal calculations.

LifestealStatConfig

Configuration that associates a lifesteal Stat and HealSource with an optional amount selector.

PreHealContext

Mutable pre-heal context raised via the pre-heal game event before the heal pipeline runs. Listeners may mutate Amount, HealSource, IsCritical, CriticalMultiplier and Ignore to transform the incoming heal — e.g. amplify a heal via a buff, reroute it through a different HealSourceSO, or negate it entirely by setting Ignore (temporary heal immunity). The pipeline reads these fields after the event is dispatched.

PreHealContext.PreHealInfoStepBuilder

Concrete builder implementing the fluent step interfaces to construct a PreHealContext.

ReceivedHealContext

Immutable result of a heal operation as actually received by the target. Contains the resolved heal amount, source, involved entities and whether it was critical.

ReceivedHealContext.ReceivedHealInfoStepBuilder

Concrete fluent builder used to assemble a ReceivedHealContext. Use Builder to start.

Interfaces

IHealable

Interface for entities that can receive healing. Implementers apply healing based on the provided PreHealContext.

PreHealContext.HealInfoAmount

Builder step interface for specifying the heal amount.

PreHealContext.HealInfoSource

Builder step interface for specifying the heal source category.

PreHealContext.HealInfoTarget

Builder step interface for specifying the target entity.

ReceivedHealContext.HealInfoAmount

Builder step: provide the heal amount.

ReceivedHealContext.HealInfoSource

Builder step: provide the heal source category.

ReceivedHealContext.HealInfoTarget

Builder step: provide the target entity and finalize options.

Enums

HealOutcome

The overall result of a heal attempt on a target. Use this in ReceivedHealContext to determine whether any healing was actually applied or if the attempt was prevented (e.g. via heal-immunity).

LifestealBasisMode

Specifies which damage value should be used as the basis for lifesteal calculations.

StepValuePoint

Selects whether to use the pre-step or post-step value when Step is selected.

Class HealAmountContext

Namespace: ElectricDrill.AstraHealth.Heal

Assembly: com.electricdrill.astra-health.Runtime.dll

Represents the different numeric stages of a heal amount as it is processed: the raw requested amount, the amount after applying a heal modifier, and the final net amount applied.

```
public class HealAmountContext
```

Inheritance

object ← HealAmountContext

Constructors

HealAmountContext(long, long, long)

Create a new instance describing the provided heal values.

```
public HealAmountContext(long rawAmount, long afterModifierAmount, long netAmount)
```

Parameters

`rawAmount` long

The initial requested heal amount.

`afterModifierAmount` long

The amount after applying modifier/stat adjustments.

`netAmount` long

The final amount actually applied to the target's health.

Properties

AfterModifierAmount

Gets the healing amount after being modified by the associated heal modifier statistic.

```
public long AfterModifierAmount { get; }
```

Property Value

long

NetAmount

Gets the actual amount of health added to the entity, considering the entity's maximum health.

```
public long NetAmount { get; }
```

Property Value

long

RawAmount

Gets the initial healing amount derived directly from PreHealContext.

```
public long RawAmount { get; }
```

Property Value

long

Enum HealOutcome

Namespace: ElectricDrill.AstraHealth.Heal

Assembly: com.electricdrill.astra-health.Runtime.dll

The overall result of a heal attempt on a target. Use this in ReceivedHealContext to determine whether any healing was actually applied or if the attempt was prevented (e.g. via heal-immunity).

```
public enum HealOutcome
```

Fields

`Applied = 0`

The heal was applied to the target's health. See HealAmount for the actual amount gained.

`Prevented = 1`

The heal attempt was prevented — typically because a listener set Ignore to `true` during the pre-heal event.

Class HealSourceSO

Namespace: ElectricDrill.AstraHealth.Heal

Assembly: com.electricdrill.astra-health.Runtime.dll

Defines a heal source asset that identifies how a heal was produced (e.g. spell, potion, resurrection, system, ...). Create instances via Assets -> Create -> Astra Health / Heal Source.

```
public class HealSourceSO : ScriptableObject, ITaggable
```

Inheritance

object ← Object ← ScriptableObject ← HealSourceSO

Implements

ITaggable

Properties

FlatHealModificationStat

The stat representing the flat heal modifier for this source. This stat serves as an accumulator for multiple flat modifiers (positive or negative) that affect healing from this source linearly.

```
public StatSO FlatHealModificationStat { get; }
```

Property Value

StatSO

PercentageHealModificationStat

The stat representing the percentage heal modifier for this source. This stat serves as an accumulator for multiple percentage modifiers (positive or negative) that affect healing from this source. Percentage heal modifiers are applied after flat modifiers.

```
public StatSO PercentageHealModificationStat { get; }
```

Property Value

StatSO

Tags

The tags associated with this object.

```
public GameTagSet Tags { get; }
```

Property Value

GameTagSet

Methods

ToString()

Returns the asset name of this heal source.

```
public override string ToString()
```

Returns

string

Interface IHealable

Namespace: ElectricDrill.AstraHealth.Heal

Assembly: com.electricdrill.astra-health.Runtime.dll

Interface for entities that can receive healing. Implementers apply healing based on the provided PreHealContext.

```
public interface IHealable
```

Methods

Heal(PreHealContext)

Apply a heal described by `context` and return information about the received heal.

```
ReceivedHealContext Heal(PreHealContext context)
```

Parameters

`context` PreHealContext

Pre-heal context used to compute the actual heal applied.

Returns

ReceivedHealContext

A ReceivedHealContext describing what was actually applied to the target.

Class LifestealAmountSelector

Namespace: ElectricDrill.AstraHealth.Heal

Assembly: com.electricdrill.astra-health.Runtime.dll

Selects which damage amount will be used as the basis for lifesteal calculations.

```
[Serializable]
public class LifestealAmountSelector
```

Inheritance

object ← LifestealAmountSelector

Constructors

LifestealAmountSelector()

Initializes a new instance of LifestealAmountSelector using default values.

```
public LifestealAmountSelector()
```

LifestealAmountSelector(LifestealBasisMode, string, StepValuePoint)

Initializes a new instance of LifestealAmountSelector with the specified selection mode, optional pipeline step type name and step value point.

```
public LifestealAmountSelector(LifestealBasisMode mode, string stepTypeName,
StepValuePoint stepPoint)
```

Parameters

`mode` LifestealBasisMode

Which basis to use (Initial, Step, Final).

`stepTypeName` string

Assembly-qualified or simple type name of the pipeline step to sample when Step is used.

`stepPoint` StepValuePoint

Whether to use the pre-step or post-step value when sampling a step.

Properties

Mode

Gets or sets the basis mode used to select the damage amount for lifesteal.

```
public LifestealBasisMode Mode { get; set; }
```

Property Value

LifestealBasisMode

StepPoint

Gets or sets whether to use the pre-step or post-step recorded value when sampling a pipeline step.

```
public StepValuePoint StepPoint { get; set; }
```

Property Value

StepValuePoint

StepType

Gets or sets the System.Type of the pipeline step to sample when Mode is Step. Setting this property stores the assembly-qualified name; getting the property resolves the stored name to a runtime System.Type.

```
public Type StepType { get; set; }
```

Property Value

Type

Methods

Evaluate(DamageAmountContext)

Evaluates the damage amounts and returns the long value that should be used for lifesteal according to this selector.

```
public long Evaluate(DamageAmountContext amounts)
```

Parameters

`amounts` DamageAmountContext

DamageAmountContext instance containing initial, current and per-step records.

Returns

long

The selected damage value to compute lifesteal from.

Enum LifestealBasisMode

Namespace: ElectricDrill.AstraHealth.Heal

Assembly: com.electricdrill.astra-health.Runtime.dll

Specifies which damage value should be used as the basis for lifesteal calculations.

```
[Serializable]
public enum LifestealBasisMode
```

Fields

`Final = 2`

Use the final damage amount (after all pipeline steps).

`Initial = 0`

Use the initial damage amount (before any pipeline modifications).

`Step = 1`

Use the damage recorded at a specific pipeline step (requires a step type).

Class LifestealStatConfig

Namespace: ElectricDrill.AstraHealth.Heal

Assembly: com.electricdrill.astra-health.Runtime.dll

Configuration that associates a lifesteal Stat and HealSource with an optional amount selector.

```
[Serializable]
public class LifestealStatConfig
```

Inheritance

object ← LifestealStatConfig

Properties

AmountSelector

Selector that determines which damage amount (initial/step/final) is used to compute the heal.

```
public LifestealAmountSelector AmountSelector { get; }
```

Property Value

LifestealAmountSelector

LifestealSource

The HealSource that will be used when applying lifesteal.

```
public HealSourceSO LifestealSource { get; }
```

Property Value

HealSourceSO

LifestealStat

The Stat that will receive lifesteal when this mapping is used.

```
public StatSO LifestealStat { get; }
```

Property Value

StatSO

Class PreHealContext

Namespace: ElectricDrill.AstraHealth.Heal

Assembly: com.electricdrill.astra-health.Runtime.dll

Mutable pre-heal context raised via the pre-heal game event before the heal pipeline runs. Listeners may mutate Amount, HealSource, IsCritical, CriticalMultiplier and Ignore to transform the incoming heal — e.g. amplify a heal via a buff, reroute it through a different HealSourceSO, or negate it entirely by setting Ignore (temporary heal immunity). The pipeline reads these fields after the event is dispatched.

```
public sealed class PreHealContext : IHasEntity, IHasTarget, IHasPerformer, IHasAmount,
IHasInstigator, IReactable
```

Inheritance

object ← PreHealContext

Implements

IHasEntity, IHasTarget, IHasPerformer, IHasAmount, IHasInstigator, IReactable

Properties

Amount

The requested heal amount (base value before modifiers). Listeners may mutate this.

```
public long Amount { get; set; }
```

Property Value

long

Builder

Fluent builder entry point for creating a PreHealContext. Use the returned builder to set amount, source, and target. The healer entity (WithPerformer(EntityCore)) is optional.

```
public static PreHealContext.HealInfoAmount Builder { get; }
```

Property Value

PreHealContext.HealInfoAmount

CriticalMultiplier

Multiplier applied when this is a critical heal. A value of 1.0 means no change. Values <= 0 are normalized to 1.0 by the constructor. Listeners may mutate this.

```
public double CriticalMultiplier { get; set; }
```

Property Value

double

Entity

The entity receiving the healing. Same as Target in this case.

```
public EntityCore Entity { get; }
```

Property Value

EntityCore

HealSource

The HealSource identifying the origin/type of the healing. Listeners may mutate this.

```
public HealSourceSO HealSource { get; set; }
```

Property Value

HealSourceSO

Ignore

If true, the heal will be ignored and not applied to the target. Use this to model heal-immunity buffs or listener-driven cancellation. Listeners may mutate this.

```
public bool Ignore { get; set; }
```

Property Value

bool

Instigator

The specific thing that caused this heal — for example an ability definition, a modifier instance, or a passive. Use pattern matching to identify the concrete type. May be `null` if no specific instigator is relevant beyond Performer and HealSource.

```
public IEffectInstigator Instigator { get; }
```

Property Value

IEffectInstigator

IsCritical

Whether this heal is flagged as a critical heal. Listeners may mutate this.

```
public bool IsCritical { get; set; }
```

Property Value

bool

IsReactable

When `true`, reactive systems may respond to this heal event. When `false`, reactive systems should skip execution to prevent chain reactions. Defaults to `true`. Set to `false` when building contexts for secondary effects such as a passive that heals in response to another heal or damage event.

```
public bool IsReactable { get; }
```

Property Value

bool

Performer

The entity that performed the heal (may be null for system-initiated or non-entity sources).

```
public EntityCore Performer { get; }
```

Property Value

EntityCore

Target

The target entity receiving the healing.

```
public EntityCore Target { get; }
```

Property Value

EntityCore

Methods

Create(long, HealSourceSO, EntityCore, EntityCore, bool, double,

IEffectInstigator, bool, bool)

Allocation-free alternative to Builder: constructs a PreHealContext directly, without the intermediate step-builder object. The fluent Builder stays the ergonomic default; prefer this overload on high-frequency heal paths (passive regeneration, lifesteal). Critical multiplier <= 0 is normalized to 1.0.

```
public static PreHealContext Create(long amount, HealSourceSO healSource, EntityCore target,
EntityCore performer = null, bool isCritical = false, double criticalMultiplier = 1,
IEffectInstigator instigator = null, bool isReactable = true, bool ignore = false)
```

Parameters

`amount` long

Base heal amount, before modifiers.

`healSource` HealSourceSO

Heal source category.

`target` EntityCore

Entity receiving the heal.

`performer` EntityCore

Entity that performed the heal. Pass `null` for system-initiated heals.

`isCritical` bool

Whether the heal is flagged as critical.

`criticalMultiplier` double

Multiplier applied when critical (1.0 means no change).

`instigator` IEffectInstigator

Specific cause of the heal (ability, modifier, passive). May be `null`.

`isReactable` bool

When `false`, reactive systems skip this event (use for secondary effects).

`ignore` bool

Initial value of Ignore. Listeners may still mutate it after dispatch.

Returns

PreHealContext

Interface PreHealContext.HealInfoAmount

Namespace: ElectricDrill.AstraHealth.Heal

Assembly: com.electricdrill.astra-health.Runtime.dll

Builder step interface for specifying the heal amount.

```
public interface PreHealContext.HealInfoAmount
```

Methods

WithAmount(long)

Specify the base heal amount.

```
PreHealContext.HealInfoSource WithAmount(long amount)
```

Parameters

`amount` long

Returns

PreHealContext.HealInfoSource

Interface PreHealContext.HealInfoSource

Namespace: ElectricDrill.AstraHealth.Heal

Assembly: com.electricdrill.astra-health.Runtime.dll

Builder step interface for specifying the heal source category.

```
public interface PreHealContext.HealInfoSource
```

Methods

WithSource(HealSourceSO)

Specify the HealSourceSO for the heal and advance to the next step.

```
PreHealContext.HealInfoTarget WithSource(HealSourceSO healSource)
```

Parameters

`healSource` HealSourceSO

Returns

PreHealContext.HealInfoTarget

Interface PreHealContext.HealInfoTarget

Namespace: ElectricDrill.AstraHealth.Heal

Assembly: com.electricdrill.astra-health.Runtime.dll

Builder step interface for specifying the target entity.

```
public interface PreHealContext.HealInfoTarget
```

Methods

WithTarget(EntityCore)

Specify the target entity and return the concrete builder. The healer (WithPerformer(EntityCore)) is optional and can be omitted when no specific entity performed the heal (e.g. system-initiated regeneration).

```
PreHealContext.PreHealInfoStepBuilder WithTarget(EntityCore target)
```

Parameters

`target` EntityCore

Returns

PreHealContext.PreHealInfoStepBuilder

Class PreHealContext.PreHealInfoStepBuilder

Namespace: ElectricDrill.AstraHealth.Heal

Assembly: com.electricdrill.astra-health.Runtime.dll

Concrete builder implementing the fluent step interfaces to construct a PreHealContext.

```
public sealed class PreHealContext.PreHealInfoStepBuilder : PreHealContext.HealInfoAmount,
PreHealContext.HealInfoSource, PreHealContext.HealInfoTarget
```

Inheritance

object ← PreHealContext.PreHealInfoStepBuilder

Implements

PreHealContext.HealInfoAmount, PreHealContext.HealInfoSource, PreHealContext.HealInfoTarget

Methods

Build()

Build and return the configured PreHealContext.

```
public PreHealContext Build()
```

Returns

PreHealContext

WithAmount(long)

Set the base heal amount.

```
public PreHealContext.HealInfoSource WithAmount(long amount)
```

Parameters

`amount` long

Returns

PreHealContext.HealInfoSource

WithCriticalMultiplier(double)

Set the critical heal multiplier. Values <= 0 will be normalized to 1.0 by the constructor.

```
public PreHealContext.PreHealInfoStepBuilder WithCriticalMultiplier(double multiplier)
```

Parameters

`multiplier` double

Returns

PreHealContext.PreHealInfoStepBuilder

WithIgnore(bool)

Set the initial Ignore flag for the built context. Defaults to `false`. Listeners may still mutate the flag after dispatch.

```
public PreHealContext.PreHealInfoStepBuilder WithIgnore(bool ignore)
```

Parameters

`ignore` bool

Returns

PreHealContext.PreHealInfoStepBuilder

WithInstigator(IEffectInstigator)

Set the specific cause of this heal — for example an ability definition, a modifier definition or instance, or a passive. Use this to allow downstream listeners to identify the exact origin beyond entity (With Performer(EntityCore)) and category (WithSource(HealSourceSO)).

```
public PreHealContext.PreHealInfoStepBuilder WithInstigator(IEffectInstigator instigator)
```

Parameters

`instigator` IEffectInstigator

Returns

PreHealContext.PreHealInfoStepBuilder

WithIsCritical(bool)

Mark the built PreHealContext as critical or not.

```
public PreHealContext.PreHealInfoStepBuilder WithIsCritical(bool isCritical)
```

Parameters

`isCritical` bool

Returns

PreHealContext.PreHealInfoStepBuilder

WithIsReactable(bool)

Set the reactability flag for the built context. Pass `false` when building a context for a secondary effect (a passive that heals in reaction to another event) so that reactive systems do not respond to it and infinite chains are prevented. Defaults to `true`.

```
public PreHealContext.PreHealInfoStepBuilder WithIsReactable(bool isReactable)
```

Parameters

`isReactable` bool

Returns

PreHealContext.PreHealInfoStepBuilder

WithPerformer(EntityCore)

Set the entity that performed the heal. Optional — pass `null` or omit entirely when no specific entity is the performer (e.g. system-initiated resurrection or passive regeneration without an actor entity).

```
public PreHealContext.PreHealInfoStepBuilder WithPerformer(EntityCore performer)
```

Parameters

`performer` EntityCore

Returns

PreHealContext.PreHealInfoStepBuilder

WithSource(HealSourceSO)

Set the heal source category.

```
public PreHealContext.HealInfoTarget WithSource(HealSourceSO healSource)
```

Parameters

`healSource` HealSourceSO

Returns

PreHealContext.HealInfoTarget

WithTarget(EntityCore)

Set the target entity.

```
public PreHealContext.PreHealInfoStepBuilder WithTarget(EntityCore target)
```

Parameters

`target` EntityCore

Returns

PreHealContext.PreHealInfoStepBuilder

Class ReceivedHealContext

Namespace: ElectricDrill.AstraHealth.Heal

Assembly: com.electricdrill.astra-health.Runtime.dll

Immutable result of a heal operation as actually received by the target. Contains the resolved heal amount, source, involved entities and whether it was critical.

```
public sealed class ReceivedHealContext : IHasEntity, IHasTarget, IHasPerformer, IHasAmount,
IHasInstigator, IReactable
```

Inheritance

object ← ReceivedHealContext

Implements

IHasEntity, IHasTarget, IHasPerformer, IHasAmount, IHasInstigator, IReactable

Constructors

ReceivedHealContext(HealAmountContext, PreHealContext, EntityCore)

Create a ReceivedHealContext from the computed heal amount and the pre-heal context.

```
public ReceivedHealContext(HealAmountContext healAmount, PreHealContext preHealContext,
EntityCore target)
```

Parameters

`healAmount` HealAmountContext

Resolved heal amount information.

`preHealContext` PreHealContext

Pre-heal context that produced metadata such as source and healer.

`target` EntityCore

Entity that will receive the heal.

Properties

Builder

Entry point to build a ReceivedHealContext using a step builder.

```
public static ReceivedHealContext.HealInfoAmount Builder { get; }
```

Property Value

ReceivedHealContext.HealInfoAmount

Entity

The entity that received the healing. Same as Target in this case.

```
public EntityCore Entity { get; }
```

Property Value

EntityCore

HealAmount

The calculated heal amount that will be applied to the target.

```
public HealAmountContext HealAmount { get; }
```

Property Value

HealAmountContext

HealSource

The HealSource that produced this heal.

```
public HealSourceSO HealSource { get; }
```

Property Value

HealSourceSO

Instigator

The specific thing that caused this heal — propagated from the originating PreHealContext. May be `null` if unspecified.

```
public IEffectInstigator Instigator { get; }
```

Property Value

IEffectInstigator

IsCritical

True if the heal was a critical heal.

```
public bool IsCritical { get; }
```

Property Value

bool

IsReactable

Propagated from IsReactable. When `false`, reactive systems should not respond to this heal event.

```
public bool IsReactable { get; }
```

Property Value

bool

Outcome

The overall result of this heal attempt. Prevented when the heal was ignored (e.g. a listener set Ignore to `true`); Applied otherwise.

```
public HealOutcome Outcome { get; }
```

Property Value

HealOutcome

Performer

The entity that performed the heal (may be null for system heals).

```
public EntityCore Performer { get; }
```

Property Value

EntityCore

Target

The entity that received the healing.

```
public EntityCore Target { get; }
```

Property Value

EntityCore

Interface ReceivedHealContext.HealInfoAmount

Namespace: ElectricDrill.AstraHealth.Heal

Assembly: com.electricdrill.astra-health.Runtime.dll

Builder step: provide the heal amount.

```
public interface ReceivedHealContext.HealInfoAmount
```

Methods

WithAmount(HealAmountContext)

Continue building by specifying the resolved HealAmountContext.

```
ReceivedHealContext.HealInfoSource WithAmount(HealAmountContext healAmount)
```

Parameters

`healAmount` HealAmountContext

Returns

ReceivedHealContext.HealInfoSource

Interface ReceivedHealContext.HealInfoSource

Namespace: ElectricDrill.AstraHealth.Heal

Assembly: com.electricdrill.astra-health.Runtime.dll

Builder step: provide the heal source category.

```
public interface ReceivedHealContext.HealInfoSource
```

Methods

WithSource(HealSourceSO)

Continue building by specifying the HealSourceSO.

```
ReceivedHealContext.HealInfoTarget WithSource(HealSourceSO healSource)
```

Parameters

`healSource` HealSourceSO

Returns

ReceivedHealContext.HealInfoTarget

Interface ReceivedHealContext.HealInfoTarget

Namespace: ElectricDrill.AstraHealth.Heal

Assembly: com.electricdrill.astra-health.Runtime.dll

Builder step: provide the target entity and finalize options.

```
public interface ReceivedHealContext.HealInfoTarget
```

Methods

WithTarget(EntityCore)

Final mandatory step: specify the target, which returns the concrete builder for optional fields. The healer (WithHealer(EntityCore)) is optional and can be omitted for system heals.

```
ReceivedHealContext.ReceivedHealInfoStepBuilder WithTarget(EntityCore target)
```

Parameters

`target` EntityCore

Returns

ReceivedHealContext.ReceivedHealInfoStepBuilder

Class ReceivedHealContext.ReceivedHealInfoStepBuilder

Namespace: ElectricDrill.AstraHealth.Heal

Assembly: com.electricdrill.astra-health.Runtime.dll

Concrete fluent builder used to assemble a ReceivedHealContext. Use Builder to start.

```
public sealed class ReceivedHealContext.ReceivedHealInfoStepBuilder :
ReceivedHealContext.HealInfoAmount, ReceivedHealContext.HealInfoSource,
ReceivedHealContext.HealInfoTarget
```

Inheritance

object ← ReceivedHealContext.ReceivedHealInfoStepBuilder

Implements

ReceivedHealContext.HealInfoAmount, ReceivedHealContext.HealInfoSource, ReceivedHealContext.HealInfoTarget

Properties

Builder

Starts a new builder for ReceivedHealContext.

```
public static ReceivedHealContext.HealInfoAmount Builder { get; }
```

Property Value

ReceivedHealContext.HealInfoAmount

Methods

Build()

Build the final ReceivedHealContext instance.

```
public ReceivedHealContext Build()
```

Returns

ReceivedHealContext

WithAmount(HealAmountContext)

Set the resolved HealAmountContext.

```
public ReceivedHealContext.HealInfoSource WithAmount(HealAmountContext healAmount)
```

Parameters

`healAmount` HealAmountContext

Returns

ReceivedHealContext.HealInfoSource

WithHealer(EntityCore)

Set the entity that performed the heal. Optional — pass `null` or omit entirely for system heals with no specific performer entity.

```
public ReceivedHealContext.ReceivedHealInfoStepBuilder WithHealer(EntityCore healer)
```

Parameters

`healer` EntityCore

Returns

ReceivedHealContext.ReceivedHealInfoStepBuilder

WithInstigator(IEffectInstigator)

Optionally set the specific cause of this heal.

```
public ReceivedHealContext.ReceivedHealInfoStepBuilder WithInstigator(IEffectInstigator
instigator)
```

Parameters

`instigator` IEffectInstigator

Returns

ReceivedHealContext.ReceivedHealInfoStepBuilder

WithIsCritical(bool)

Optionally mark the heal as critical.

```
public ReceivedHealContext.ReceivedHealInfoStepBuilder WithIsCritical(bool isCritical)
```

Parameters

`isCritical` bool

Returns

ReceivedHealContext.ReceivedHealInfoStepBuilder

WithIsReactable(bool)

Set the reactability flag for the built context. Pass `false` when building a context for a secondary effect so that reactive systems do not respond to it. Defaults to `true`.

```
public ReceivedHealContext.ReceivedHealInfoStepBuilder WithIsReactable(bool isReactable)
```

Parameters

`isReactable` bool

Returns

ReceivedHealContext.ReceivedHealInfoStepBuilder

WithOutcome(HealOutcome)

Set the HealOutcome for the built context. Defaults to Applied. Use Prevented when building a context that represents an ignored or cancelled heal.

```
public ReceivedHealContext.ReceivedHealInfoStepBuilder WithOutcome(HealOutcome outcome)
```

Parameters

`outcome` HealOutcome

Returns

ReceivedHealContext.ReceivedHealInfoStepBuilder

WithSource(HealSourceSO)

Set the HealSourceSO for the heal.

```
public ReceivedHealContext.HealInfoTarget WithSource(HealSourceSO healSource)
```

Parameters

`healSource` HealSourceSO

Returns

ReceivedHealContext.HealInfoTarget

WithTarget(EntityCore)

Set the entity that will receive the heal. Returns the builder for optional flags.

```
public ReceivedHealContext.ReceivedHealInfoStepBuilder WithTarget(EntityCore target)
```

Parameters

`target` EntityCore

Returns

ReceivedHealContext.ReceivedHealInfoStepBuilder

Enum StepValuePoint

Namespace: ElectricDrill.AstraHealth.Heal

Assembly: com.electricdrill.astra-health.Runtime.dll

Selects whether to use the pre-step or post-step value when Step is selected.

```
[Serializable]
public enum StepValuePoint
```

Fields

`Post = 1`

Use the value recorded after the pipeline step executes.

`Pre = 0`

Use the value recorded before the pipeline step executes.

Namespace ElectricDrill.AstraHealth.ReductionFunctions

Classes

ReductionFnSO

Common abstract base for all reduction-function ScriptableObjects. Provides ElectricDrill.AstraRpgFramework.Tags.ITaggable support shared by DamageMitigationFnSO and DefensePenetrationFnSO.

Structs

ReductionContext

The entities involved in a hit, handed to damage mitigation and defense penetration functions that depend on more than the stat values: a formula whose constant grows with the attacker's level, or one driven by another stat of the attacker or the target.

Struct ReductionContext

Namespace: ElectricDrill.AstraHealth.ReductionFunctions

Assembly: com.electricdrill.astra-health.Runtime.dll

The entities involved in a hit, handed to damage mitigation and defense penetration functions that depend on more than the stat values: a formula whose constant grows with the attacker's level, or one driven by another stat of the attacker or the target.

```
public readonly struct ReductionContext
```

Remarks

It carries references and derives what it exposes only when asked, so a function that needs nothing from it (flat, percentage, logarithmic) costs nothing. Future inputs are added as properties here, not as new parameters on the functions.

Entity-backed or value-backed. The damage pipeline builds an entity-backed context. Tools and tests that have no entities can state levels directly with ReductionContext(int, int), and the full constructor lets an explicit level take precedence over the entity's. `default` is a valid context: no entities, target level 1, no performer (the same as Default).

Contract for functions. The context is valid only during the call: do not store it or its references. Functions are pure, so treat the entities as read-only even though they are reachable. Read a stat with `TryGet` on TargetStats / PerformerStats and treat a missing stat as 0: those readers are `null` when the context was built from levels alone.

What is not a stat. TargetStats and PerformerStats exist to read stats the way the pipeline does: the attacker's through the ownership fallback chain, which an entity cannot derive on its own (hence the reader the pipeline injects for it; the target has no chain and is read directly). Anything else is reached through Target and Performer. The target's maximum health, for example, is not a stat but `EntityHealth.MaxHp`: `context.Target.GetEntityHealth()` returns it, or `null` when the context has no target entity or the target has no health component.

It is a readonly struct passed by `in`: building and passing it does not allocate, which matters on the per-hit damage path.

Constructors

ReductionContext(EntityCore, EntityCore, IStatReader, int, int)

Builds a context from the entities of a hit.

```
public ReductionContext(EntityCore target, EntityCore performer, IStatReader performerStats
= null, int targetLevel = 0, int performerLevel = 0)
```

Parameters

`target` EntityCore

The entity taking the damage. May be `null`.

`performer` EntityCore

The entity the damage is attributed to: the performer itself, or its resolved owner under an ownership attribution. `null` for environmental damage.

`performerStats` IStatReader

The reader used for the attacker's stats, which under an ownership attribution falls back from the performer to its owner. When `null`, `performer` is used.

`targetLevel` int

An explicit target level that takes precedence over the entity's. 0 or less reads it from the entity.

`performerLevel` int

An explicit performer level that takes precedence over the entity's. 0 or less reads it from the entity.

ReductionContext(int, int)

Builds a context from levels alone, with no entities (so no stat readers).

```
public ReductionContext(int targetLevel, int performerLevel)
```

Parameters

`targetLevel` int

Level of the damage target. Values below 1 are raised to 1.

`performerLevel` int

Level of the attacker, or 0 (or less) when there is no performer.

Fields

Default

Context used when no entity is known (tools, callers of the context-free overloads): the target is level 1 and there is no performer, so every level source resolves to level 1.

```
public static readonly ReductionContext Default
```

Field Value

ReductionContext

Properties

HasPerformer

`true` when the hit has an attacker: a performer entity or an explicit performer level.

```
public bool HasPerformer { get; }
```

Property Value

bool

Performer

The entity the hit is attributed to: the performer itself, or its resolved owner under an ownership attribution (a level 1 weapon owned by a level 50 hero attacks as the hero). `null` for environmental damage, or when the context was built from levels alone.

```
public EntityCore Performer { get; }
```

Property Value

EntityCore

PerformerLevel

Level of the entity the hit is attributed to: the explicit one if given, otherwise the performer's, otherwise 0 when there is no performer.

```
public int PerformerLevel { get; }
```

Property Value

int

PerformerStats

Reader for the attacker's stats, consistent with the stats the piercing stat is read from: under an ownership attribution the performer's own stats come first and its owner's fill the gaps. `null` when there is no performer entity.

```
public IStatReader PerformerStats { get; }
```

Property Value

IStatReader

Target

The entity taking the damage, or `null` when the context was built from levels alone.

```
public EntityCore Target { get; }
```

Property Value

EntityCore

TargetLevel

Level of the entity taking the damage: the explicit one if given, otherwise the target's, otherwise 1. Never below 1.

```
public int TargetLevel { get; }
```

Property Value

int

TargetStats

Reader for the target's stats, or `null` when there is no target entity. Always the target entity itself: unlike the attacker's, there is no fallback chain.

```
public IStatReader TargetStats { get; }
```

Property Value

IStatReader

Class ReductionFnSO

Namespace: ElectricDrill.AstraHealth.ReductionFunctions

Assembly: com.electricdrill.astra-health.Runtime.dll

Common abstract base for all reduction-function ScriptableObjects. Provides ElectricDrill.AstraRpg Framework.Tags.ITaggable support shared by DamageMitigationFnSO and DefensePenetrationFnSO.

```
public abstract class ReductionFnSO : ScriptableObject, ITaggable
```

Inheritance

object ← Object ← ScriptableObject ← ReductionFnSO

Implements

ITaggable

Derived

DamageMitigationFnSO, DefensePenetrationFnSO

Properties

Tags

The tags associated with this object.

```
public GameTagSet Tags { get; }
```

Property Value

GameTagSet

Namespace ElectricDrill.AstraHealth.Resurrection

Interfaces

IResurrectable

Interface IResurrectable

Namespace: ElectricDrill.AstraHealth.Resurrection

Assembly: com.electricdrill.astra-health.Runtime.dll

```
public interface IResurrectable
```

Methods

Resurrect(PreHealContext)

Resurrects the entity using the provided PreHealContext. All heal modifiers (generic and HealSourceSO-specific) defined in the context are applied.

```
void Resurrect(PreHealContext preHealContext)
```

Parameters

`preHealContext` PreHealContext

The pre-heal context describing amount, source and healer for the resurrection heal.

Resurrect(Percentage)

Convenience overload. Resurrects the entity with a percentage of its maximum HP using the default resurrection source from config and no healer entity.

```
void Resurrect(Percentage withHpPercent)
```

Parameters

`withHpPercent` Percentage

The percentage of maximum HP to restore on resurrection (before heal modifiers).

Resurrect(long)

Convenience overload. Resurrects the entity with a base amount of HP using the default resurrection source from config and no healer entity.

```
void Resurrect(long withHp)
```

Parameters

`withHp` long

The base HP amount to restore on resurrection (before heal modifiers).

Namespace ElectricDrill.AstraHealth.Scaling.ScalingComponents

Classes

HealthScalingComponentSO

A scaling component that calculates a value based on an entity's health values.

Structs

HealthScalingComponentSO.HealthScalingMetricValue

Represents a mapping between a health metric and its scaling multiplier.

Enums

HealthScalingComponentSO.HealthScalingMetrics

Defines the available health metrics for scaling calculations.

Class HealthScalingComponentSO

Namespace: ElectricDrill.AstraHealth.Scaling.ScalingComponents

Assembly: com.electricdrill.astra-health.Runtime.dll

A scaling component that calculates a value based on an entity's health values.

```
public class HealthScalingComponentSO : ScalingComponentSO, ITaggable
```

Inheritance

object ← Object ← ScriptableObject ← ScalingComponentSO ← HealthScalingComponentSO

Implements

ITaggable

Inherited Members

ScalingComponentSO.Tags

Properties

ScalingMetricValues

Read-only view of every metric/multiplier pair defined on this component. Intended for tooling and UI display (e.g. tooltips) — not for runtime calculations.

```
public IReadOnlyList<HealthScalingComponentSO.HealthScalingMetricValue> ScalingMetricValues
{ get; }
```

Property Value

IReadOnlyList<HealthScalingComponentSO.HealthScalingMetricValue>

Methods

CalculateValue(EntityCore)

Calculates the scaling value based on the entity's health component.

```
public override long CalculateValue(EntityCore entity)
```

Parameters

`entity` EntityCore

The entity to calculate the value for.

Returns

long

The calculated scaling value.

Struct HealthScalingComponentSO.HealthScalingMetricValue

Namespace: ElectricDrill.AstraHealth.Scaling.ScalingComponents

Assembly: com.electricdrill.astra-health.Runtime.dll

Represents a mapping between a health metric and its scaling multiplier.

```
[Serializable]
public struct HealthScalingComponentSO.HealthScalingMetricValue
```

Fields

metric

The health metric to scale by.

```
public HealthScalingComponentSO.HealthScalingMetrics metric
```

Field Value

HealthScalingComponentSO.HealthScalingMetrics

value

The multiplier applied to the metric (e.g. 0.3 = 30%).

```
public double value
```

Field Value

double

Enum HealthScalingComponentSO.HealthScalingMetrics

Namespace: ElectricDrill.AstraHealth.Scaling.ScalingComponents

Assembly: com.electricdrill.astra-health.Runtime.dll

Defines the available health metrics for scaling calculations.

```
public enum HealthScalingComponentSO.HealthScalingMetrics
```

Fields

`Hp = 0`

`MaxHp = 1`

`MissingHp = 2`

Namespace ElectricDrill.AstraHealth.Triggers

Classes

DamageResolutionGameEventTrigger

ElectricDrill.AstraRpgFramework.Triggers.GameEventTrigger<TContext> that fires when damage resolution completes. Payload: DamageResolutionContext — use `payload.Target` or `payload.Performer` to filter by entity in an associated condition.

EntityDiedGameEventTrigger

ElectricDrill.AstraRpgFramework.Triggers.GameEventTrigger<TContext> that fires when an entity dies. Payload: EntityDiedContext — use `payload.Entity` or `payload.Victim` to filter by target entity in an associated condition.

EntityHealedGameEventTrigger

ElectricDrill.AstraRpgFramework.Triggers.GameEventTrigger<TContext> that fires when an entity is healed. Payload: ReceivedHealContext — use `payload.Entity` or `payload.Target` to filter by entity in an associated condition.

EntityHealthChangedGameEventTrigger

ElectricDrill.AstraRpgFramework.Triggers.GameEventTrigger<TContext> that fires when an entity's health changes. Payload: EntityHealthChangedContext — use `payload.Entity` or `payload.Target` to filter by entity in an associated condition.

EntityMaxHealthChangedGameEventTrigger

ElectricDrill.AstraRpgFramework.Triggers.GameEventTrigger<TContext> that fires when an entity's max health changes. Payload: EntityMaxHealthChangedContext — use `payload.Entity` to filter by entity in an associated condition.

EntityResurrectedGameEventTrigger

ElectricDrill.AstraRpgFramework.Triggers.GameEventTrigger<TContext> that fires when an entity is resurrected. Payload: ResurrectionContext — use the heal-related fields on the payload to filter by entity in an associated condition.

HealthRatioChangedGameEventTrigger

PreDamageGameEventTrigger

ElectricDrill.AstraRpgFramework.Triggers.GameEventTrigger<TContext> that fires before damage is processed. Payload: PreDamageContext — use `payload.Target` or `payload.Performer` to filter by entity in an associated condition.

PreHealGameEventTrigger

ElectricDrill.AstraRpgFramework.Triggers.GameEventTrigger<TContext> that fires before a heal is processed. Payload: PreHealContext — use `payload.Entity` or `payload.Target` to filter by entity in an associated condition.

Class DamageResolutionGameEventTrigger

Namespace: ElectricDrill.AstraHealth.Triggers

Assembly: com.electricdrill.astra-health.Runtime.dll

ElectricDrill.AstraRpgFramework.Triggers.GameEventTrigger<TContext> that fires when damage resolution completes. Payload: DamageResolutionContext — use `payload.Target` or `payload.Performer` to filter by entity in an associated condition.

```
[Serializable]
[TypeSelectableMenu("Damage")]
public sealed class DamageResolutionGameEventTrigger :
GameEventTrigger<DamageResolutionContext>, IReactiveTrigger
```

Inheritance

object ← GameEventTrigger<DamageResolutionContext> ← DamageResolutionGameEventTrigger

Implements

IReactiveTrigger

Inherited Members

GameEventTrigger<DamageResolutionContext>.PayloadType ,
GameEventTrigger<DamageResolutionContext>.EventSource ,
GameEventTrigger<DamageResolutionContext>.Subscribe(EntityCore, Action<object>) ,
GameEventTrigger<DamageResolutionContext>.Unsubscribe(EntityCore, Action<object>)

Properties

Event

The typed event SO that this trigger listens to. Provided by the concrete subclass.

```
protected override GameEventGeneric1<DamageResolutionContext> Event { get; }
```

Property Value

GameEventGeneric1<DamageResolutionContext>

Class EntityDiedGameEventTrigger

Namespace: ElectricDrill.AstraHealth.Triggers

Assembly: com.electricdrill.astra-health.Runtime.dll

ElectricDrill.AstraRpgFramework.Triggers.GameEventTrigger<TContext> that fires when an entity dies. Payload: EntityDiedContext — use `payload.Entity` or `payload.Victim` to filter by target entity in an associated condition.

```
[Serializable]
[TypeSelectableMenu("Lifecycle")]
public sealed class EntityDiedGameEventTrigger : GameEventTrigger<EntityDiedContext>,
IReactiveTrigger
```

Inheritance

object ← GameEventTrigger<EntityDiedContext> ← EntityDiedGameEventTrigger

Implements

IReactiveTrigger

Inherited Members

GameEventTrigger<EntityDiedContext>.PayloadType ,
GameEventTrigger<EntityDiedContext>.EventSource ,
GameEventTrigger<EntityDiedContext>.Subscribe(EntityCore, Action<object>) ,
GameEventTrigger<EntityDiedContext>.Unsubscribe(EntityCore, Action<object>)

Properties

Event

The typed event SO that this trigger listens to. Provided by the concrete subclass.

```
protected override GameEventGeneric1<EntityDiedContext> Event { get; }
```

Property Value

GameEventGeneric1<EntityDiedContext>

Class EntityHealedGameEventTrigger

Namespace: ElectricDrill.AstraHealth.Triggers

Assembly: com.electricdrill.astra-health.Runtime.dll

ElectricDrill.AstraRpgFramework.Triggers.GameEventTrigger<TContext> that fires when an entity is healed. Payload: ReceivedHealContext — use `payload.Entity` or `payload.Target` to filter by entity in an associated condition.

```
[Serializable]
[TypeSelectableMenu("Healing")]
public sealed class EntityHealedGameEventTrigger : GameEventTrigger<ReceivedHealContext>,
IReactiveTrigger
```

Inheritance

object ← GameEventTrigger<ReceivedHealContext> ← EntityHealedGameEventTrigger

Implements

IReactiveTrigger

Inherited Members

GameEventTrigger<ReceivedHealContext>.PayloadType ,
GameEventTrigger<ReceivedHealContext>.EventSource ,
GameEventTrigger<ReceivedHealContext>.Subscribe(EntityCore, Action<object>) ,
GameEventTrigger<ReceivedHealContext>.Unsubscribe(EntityCore, Action<object>)

Properties

Event

The typed event SO that this trigger listens to. Provided by the concrete subclass.

```
protected override GameEventGeneric1<ReceivedHealContext> Event { get; }
```

Property Value

GameEventGeneric1<ReceivedHealContext>

Class EntityHealthChangedGameEventTrigger

Namespace: ElectricDrill.AstraHealth.Triggers

Assembly: com.electricdrill.astra-health.Runtime.dll

ElectricDrill.AstraRpgFramework.Triggers.GameEventTrigger<TContext> that fires when an entity's health changes. Payload: EntityHealthChangedContext — use `payload.Entity` or `payload.Target` to filter by entity in an associated condition.

```
[Serializable]
[TypeSelectableMenu("Stats")]
public sealed class EntityHealthChangedGameEventTrigger :
GameEventTrigger<EntityHealthChangedContext>, IReactiveTrigger
```

Inheritance

object ← GameEventTrigger<EntityHealthChangedContext> ← EntityHealthChangedGameEventTrigger

Implements

IReactiveTrigger

Inherited Members

GameEventTrigger<EntityHealthChangedContext>.PayloadType ,
GameEventTrigger<EntityHealthChangedContext>.EventSource ,
GameEventTrigger<EntityHealthChangedContext>.Subscribe(EntityCore, Action<object>) ,
GameEventTrigger<EntityHealthChangedContext>.Unsubscribe(EntityCore, Action<object>)

Properties

Event

The typed event SO that this trigger listens to. Provided by the concrete subclass.

```
protected override GameEventGeneric1<EntityHealthChangedContext> Event { get; }
```

Property Value

GameEventGeneric1<EntityHealthChangedContext>

Class EntityMaxHealthChangedGameEventTrigger

Namespace: ElectricDrill.AstraHealth.Triggers

Assembly: com.electricdrill.astra-health.Runtime.dll

ElectricDrill.AstraRpgFramework.Triggers.GameEventTrigger<TContext> that fires when an entity's max health changes. Payload: EntityMaxHealthChangedContext — use `payload.Entity` to filter by entity in an associated condition.

```
[Serializable]
[TypeSelectableMenu("Stats")]
public sealed class EntityMaxHealthChangedGameEventTrigger :
GameEventTrigger<EntityMaxHealthChangedContext>, IReactiveTrigger
```

Inheritance

object ← GameEventTrigger<EntityMaxHealthChangedContext> ← EntityMaxHealthChangedGameEventTrigger

Implements

IReactiveTrigger

Inherited Members

GameEventTrigger<EntityMaxHealthChangedContext>.PayloadType , GameEventTrigger<EntityMaxHealthChangedContext>.EventSource , GameEventTrigger<EntityMaxHealthChangedContext>.Subscribe(EntityCore, Action<object>) , GameEventTrigger<EntityMaxHealthChangedContext>.Unsubscribe(EntityCore, Action<object>)

Properties

Event

The typed event SO that this trigger listens to. Provided by the concrete subclass.

```
protected override GameEventGeneric1<EntityMaxHealthChangedContext> Event { get; }
```

Property Value

GameEventGeneric1<[EntityMaxHealthChangedContext]>

Class EntityResurrectedGameEventTrigger

Namespace: ElectricDrill.AstraHealth.Triggers

Assembly: com.electricdrill.astra-health.Runtime.dll

ElectricDrill.AstraRpgFramework.Triggers.GameEventTrigger<TContext> that fires when an entity is resurrected. Payload: ResurrectionContext — use the heal-related fields on the payload to filter by entity in an associated condition.

```
[Serializable]
[TypeSelectableMenu("Lifecycle")]
public sealed class EntityResurrectedGameEventTrigger :
GameEventTrigger<ResurrectionContext>, IReactiveTrigger
```

Inheritance

object ← GameEventTrigger<ResurrectionContext> ← EntityResurrectedGameEventTrigger

Implements

IReactiveTrigger

Inherited Members

GameEventTrigger<ResurrectionContext>.PayloadType ,
GameEventTrigger<ResurrectionContext>.EventSource ,
GameEventTrigger<ResurrectionContext>.Subscribe(EntityCore, Action<object>) ,
GameEventTrigger<ResurrectionContext>.Unsubscribe(EntityCore, Action<object>)

Properties

Event

The typed event SO that this trigger listens to. Provided by the concrete subclass.

```
protected override GameEventGeneric1<ResurrectionContext> Event { get; }
```

Property Value

GameEventGeneric1<ResurrectionContext>

Class HealthRatioChangedGameEventTrigger

Namespace: ElectricDrill.AstraHealth.Triggers

Assembly: com.electricdrill.astra-health.Runtime.dll

```
[Serializable]
[TypeSelectableMenu("Stats")]
public sealed class HealthRatioChangedGameEventTrigger :
GameEventTrigger<HealthRatioChangedContext>, IReactiveTrigger
```

Inheritance

object ← GameEventTrigger<HealthRatioChangedContext> ← HealthRatioChangedGameEventTrigger

Implements

IReactiveTrigger

Inherited Members

GameEventTrigger<HealthRatioChangedContext>.PayloadType ,
GameEventTrigger<HealthRatioChangedContext>.EventSource ,
GameEventTrigger<HealthRatioChangedContext>.Subscribe(EntityCore, Action<object>) ,
GameEventTrigger<HealthRatioChangedContext>.Unsubscribe(EntityCore, Action<object>)

Properties

Event

The typed event SO that this trigger listens to. Provided by the concrete subclass.

```
protected override GameEventGeneric1<HealthRatioChangedContext> Event { get; }
```

Property Value

GameEventGeneric1<HealthRatioChangedContext>

Class PreDamageGameEventTrigger

Namespace: ElectricDrill.AstraHealth.Triggers

Assembly: com.electricdrill.astra-health.Runtime.dll

ElectricDrill.AstraRpgFramework.Triggers.GameEventTrigger<TContext> that fires before damage is processed. Payload: PreDamageContext — use `payload.Target` or `payload.Performer` to filter by entity in an associated condition.

```
[Serializable]
[TypeSelectableMenu("Damage")]
public sealed class PreDamageGameEventTrigger : GameEventTrigger<PreDamageContext>,
IReactiveTrigger
```

Inheritance

object ← GameEventTrigger<PreDamageContext> ← PreDamageGameEventTrigger

Implements

IReactiveTrigger

Inherited Members

GameEventTrigger<PreDamageContext>.PayloadType ,
GameEventTrigger<PreDamageContext>.EventSource ,
GameEventTrigger<PreDamageContext>.Subscribe(EntityCore, Action<object>) ,
GameEventTrigger<PreDamageContext>.Unsubscribe(EntityCore, Action<object>)

Properties

Event

The typed event SO that this trigger listens to. Provided by the concrete subclass.

```
protected override GameEventGeneric1<PreDamageContext> Event { get; }
```

Property Value

GameEventGeneric1<PreDamageContext>

Class PreHealGameEventTrigger

Namespace: ElectricDrill.AstraHealth.Triggers

Assembly: com.electricdrill.astra-health.Runtime.dll

ElectricDrill.AstraRpgFramework.Triggers.GameEventTrigger<TContext> that fires before a heal is processed. Payload: PreHealContext — use `payload.Entity` or `payload.Target` to filter by entity in an associated condition.

```
[Serializable]
[TypeSelectableMenu("Healing")]
public sealed class PreHealGameEventTrigger : GameEventTrigger<PreHealContext>,
IReactiveTrigger
```

Inheritance

object ← GameEventTrigger<PreHealContext> ← PreHealGameEventTrigger

Implements

IReactiveTrigger

Inherited Members

GameEventTrigger<PreHealContext>.PayloadType , GameEventTrigger<PreHealContext>.EventSource , GameEventTrigger<PreHealContext>.Subscribe(EntityCore, Action<object>) , GameEventTrigger<PreHealContext>.Unsubscribe(EntityCore, Action<object>)

Properties

Event

The typed event SO that this trigger listens to. Provided by the concrete subclass.

```
protected override GameEventGeneric1<PreHealContext> Event { get; }
```

Property Value

GameEventGeneric1<PreHealContext>

Namespace ElectricDrill.AstraHealth.Utils

Classes

EntityCoreHealthExtensions

Class EntityCoreHealthExtensions

Namespace: ElectricDrill.AstraHealth.Utils

Assembly: com.electricdrill.astra-health.Runtime.dll

```
public static class EntityCoreHealthExtensions
```

Inheritance

object ← EntityCoreHealthExtensions

Methods

GetEntityHealth(EntityCore)

Gets the EntityHealth component associated with this EntityCore. Result is cached for performance similar to internal fields.

```
public static EntityHealth GetEntityHealth(this EntityCore entity)
```

Parameters

`entity` EntityCore

Returns

EntityHealth